Operativni Sistem

za rad sa **AI**

Mentalni obrasci za rad sa vještačkom inteligencijom

Goran Crljenica

Prvo izdanje — 2026

metaxis.io

Operativni Sistem za rad sa AI

Mentalni obrasci za rad sa vještačkom inteligencijom

Goran Crljenica

Prvo izdanje — 2026

Autorska prava

© 2026 Goran Crljenica

Sva prava zadržana.

Nijedan dio ove publikacije ne smije se reproducirati, distribuirati ili prenositi u bilo kojem obliku ili na bilo koji način bez prethodne pismene dozvole autora, osim u slučaju kratkih citata u recenzijama ili naučnim radovima.

Napomena

Ova knjiga je edukativnog karaktera i ima za cilj da čitaocu pomogne u razumijevanju principa prompt inženjeringa i rada sa velikim jezičkim modelima.

Autor je uložio značajan trud kako bi informacije u knjizi bile tačne i korisne u trenutku objavljivanja. Međutim, tehnologije umjetne inteligencije se brzo razvijaju, pa određeni alati, komande ili metode opisane u knjizi mogu vremenom postati zastarjele ili promijenjene.

Autor ne garantuje da će primjena svih opisanih tehnika dovesti do istih rezultata u svakom okruženju ili za svaki model umjetne inteligencije.

Korištenje informacija iz ove knjige vrši se na vlastitu odgovornost čitaoca.

O autoru

Goran Crljenica izgradio je svoj profesionalni put kroz više industrija i organizacijskih nivoa — od operativnih pozicija do strateških odgovornosti. Njegovo iskustvo obuhvata prodaju, logistiku, koordinaciju procesa i rad u strukturiranim industrijskim i zdravstvenim okruženjima.

Rad kroz različite hijerarhije i sisteme naučio ga je jednoj stalnoj lekciji: struktura je ključna. Bilo da je riječ o lancima snabdijevanja, komercijalnim pregovorima, regulatornim okruženjima ili tehničkim procesima, jasno definisane specifikacije određuju stabilnost ishoda.

Njegov ulazak u oblast umjetne inteligencije nije bio vođen trendovima ili hype-om, već načinom razmišljanja zasnovanim na sistemima. Umjesto da jezičke modele posmatra kao alat za brze odgovore, pristupio im je kao kompleksnim operativnim sistemima — kroz definisanje ograničenja, mapiranje zavisnosti i izgradnju ponovljivih procesa.

Rezultat je metodologija zasnovana na kontroli, provjeri i operativnoj disciplini.

Ova knjiga predstavlja rezultat tog pristupa.

Izdanje i verzija

Prvo izdanje
Verzija 1.0
Objavljeno 2026

Ovo izdanje odražava strukturu i obrasce dostupne u trenutku objavljivanja. Kako se sistemi umjetne inteligencije razvijaju, buduća izdanja mogu proširiti, doraditi ili reorganizovati pojedine dijelove kako bi zadržala praktičnu vrijednost i relevantnost sadržaja.

Sadržaj

Part I — Foundations: Thinking Patterns for Prompting

Block A — Mentalni modeli i kontrola razgovora

1. Kako koristiti ovu knjigu
2. Cognitive Verifier: kako provjeriti da stvarno razumiješ
3. Reframing: kako postaviti pitanje da dobiješ bolji odgovor
4. Studija slučaja: CVP + Reframing u praksi
5. Flipped Interaction: kad AI vodi intervju umjesto tebe

Block B — Komande i kompozicija promptova

6. Komandna biblioteka: komande za ponašanje modela
7. Kombinovanje komandi: misaoni režimi i kontrola
8. Šabloni: kako napraviti ponovljiv output
9. Meta-jezik: kad modelu uvedeš svoj sistem pojmova
10. Recepti: vođenje modela kroz korake bez nagađanja
11. Menu pristup: izbor bez gubitka kontrole
12. Napredni menu sistemi: od izbora do agent menija
13. Few-shot: učenje iz primjera koje mu pokažeš

Block C — Razmišljanje i rad u koracima

14. Razmišljanje u koracima: kad i kako tražiti korak-po-korak
15. Razlaganje zadatka: od nejasnog zahtjeva do koraka
16. Write → Reflect → Improve: mini-proces urednika
17. ReAct — sinergija razmišljanja i djelovanja
18. Game-play: misaona igra za dolaženje do cilja
19. Kad model prestaje pričati i počinje raditi
20. Prompt pack: 20 gotovih promptova za code interpreter

Block D — Od promptinga ka agentima

21. Prelaz: od prompta do agenta
22. Ograničenja: kako dobiti stabilan output
23. Uloge: kad želiš stručnjaka po narudžbi
24. Persona, uloga i mode: tri različite poluge
25. Agent loopovi: Think → Act → Verify
26. Alati: izbor akcije i izvršenje
27. Stanje i memorija: šta je stvarno zapamćeno
28. Planiranje i task grafovi: kad zadatak postane sistem

Block E — Operativna stabilnost i sigurnost

29. Granice alata: greške, povjerenje i fallback
30. Kontekst prozor: kapacitet, rizici i strategije
31. Feedback signali: učenje bez treniranja
32. Nesigurnost: kad model mora reći ne znam
33. Guardrails: sigurnosna pravila i hard stop
34. Kad prompt postane sistem: production readiness
35. Prompt Orchestration / Multi-Pattern Control
36. Multi-Prompt Orchestration: kada prompti postanu proces

Part II — Under the Hood: How These Patterns Work in Practice

Block A — Output & Structure

37. Output Format Grounding
38. JSON Structuring Pattern
39. Structured Extraction Pattern
40. Data Transformation Pattern

Block B — Data & Document Handling

41. From Images to Data: OCR & Visual Understanding
42. Tabular Reasoning & Table Operations
43. Data Visualization Pattern

Block C — Tool Usage

44. Working with Tools: The Code Interpreter Paradigm
45. Toolformer Pattern: When Models Decide to Use Tools

Block D — Quality & Validation

46. Evaluating AI Output: Validation, Errors, and Trust
47. Self-Correction Loops: How Models Fix Their Own Mistakes
48. Self-Consistency: Stability Across Multiple Reasoning Paths

Block E — System Integration

49. Managing Context: Working Within the Context Window (CWO)
50. Multi-Modal Reasoning: Combining Text, Images, and Data
51. Retrieval-Augmented Prompting: When AI Needs Memory (RAP)

Block F — Reasoning, Reflection & Self-Correction

52. Meta-Prompting Pattern
53. Error-Correction Pattern
54. Socratic Prompting Pattern
55. Reflection Pattern
56. ReAct Pattern (Reasoning + Acting)
57. Deliberate Reasoning Pattern

Part I - Foundations: Thinking Patterns for Prompting

Mentalni modeli i kontrola razgovora

Kako koristiti ovu knjigu

Uvod

Ako koristiš AI, a imaš osjećaj da ti ponekad daje pametne odgovore, a ponekad potpune gluposti — problem najčešće nije u alatu. Problem je u načinu na koji razmišljaš s njim.

Većina ljudi koristi AI kao Google na steroidima: postave pitanje, dobiju odgovor i idu dalje. Nekad to ispadne solidno, nekad površno, nekad potpuno pogrešno. I onda se zaključi da je AI "nepouzdan", "još nezreo" ili da "halucinira".

Istina je jednostavnija — i malo neugodna:

AI gotovo uvijek odgovara onako kako je vođen.

Ova knjiga nije nastala iz želje da naučim "kako pisati bolji prompt". Nastala je iz frustracije. Iz pokušaja da shvatim zašto isti alat, u rukama različitih ljudi, daje potpuno različite rezultate. Zašto nekome AI djeluje kao genije, a nekome kao dijete koje pogađa odgovore.

Vremenom sam shvatio jednu važnu stvar:

AI nije pametan u ljudskom smislu. Nema razumijevanje, intuiciju ni kontekst. Ono što ima je sposobnost da slijedi **obrasce razmišljanja koje mu ti nametneš**.

Drugim riječima:

> AI ne razmišlja umjesto tebe. On razmišlja onako kako ga ti natjeraš.

Tu počinje pravi problem — ali i pravo rješenje.

Za koga je ova knjiga

Ova knjiga ne traži da budeš programer. Ne traži da razumiješ matematiku iza modela. Ne traži ni da pratiš trendove.

Traži samo jedno: da preuzmeš odgovornost za način na koji koristiš AI.

Prompt engineering se danas često svodi na trikove i recepte: "upiši ovo", "koristi ovu frazu", "kopiraj ovaj template". To kratkoročno radi. Dugoročno stvara zavisnost od tuđih rješenja i osjećaj da nikad nemaš stvarnu kontrolu.

Ovdje idemo drugim putem.

U ovoj knjizi nećeš učiti kako da "izvučeš" odgovor iz AI-ja. Naučit ćeš kako da **postaviš misaoni okvir** u kojem AI može raditi smisleno, stabilno i predvidivo. Kako da mu ograničiš prostor, usmjeriš pažnju i natjeraš ga da provjerava sam sebe.

Zato govorimo o **patternima**, a ne o promptovima. Pattern nije trik — to je način razmišljanja koji možeš primijeniti danas, sutra i na bilo kojem alatu koji će tek doći.

Ako imaš osjećaj da ti AI daje previše teksta, a premalo smisla — ova knjiga je za tebe. Ako želiš da AI postane koristan alat, a ne izvor konfuzije — ova knjiga je za tebe. Ako te zanima AI, ali nemaš tehnički background i ne želiš ga glumiti — ova knjiga je za tebe.

Jedna stvar za kraj

Važno je da ovo kažeš sebi iskreno:

AI nije rješenje. AI je pojačalo.

Pojačava jasnoću, ali i konfuziju. Pojačava strukturu, ali i haos. Pojačava razmišljanje koje već postoji.

Ova knjiga postoji da bi ono što AI pojačava bilo — **jasno, mirno i pod tvojom kontrolom**.

Ako si spreman za to, možemo početi.

Cognitive Verifier: kako provjeriti da stvarno razumiješ

Pattern koji tjera AI da prvo razmišlja, a tek onda odgovara.

---

1. Šta je to

Cognitive Verifier Pattern (CVP) je tehnika u kojoj AI **ne daje odmah konačan odgovor**, već prvo analizira samo pitanje.

Umjesto direktnog odgovaranja, model:

1. prima tvoje pitanje
2. identifikuje šta u njemu može biti nejasno, preširoko, pogrešno postavljeno ili zasnovano na skrivenim pretpostavkama
3. generiše **podpitanja** koja razjašnjavaju problem
4. tek nakon toga kombinuje te uvide u jedan smislen, provjeren odgovor

Na taj način AI ne "pogađa" rješenje, nego ga **logički sastavlja kroz provjeru vlastitog razumijevanja**.

CVP pretvara odgovor u proces, a ne u refleks.

Cognitive Verifier Pattern pomaže AI-ju da uspori, razmisli i provjeri vlastitu logiku prije nego što isporuči konačan odgovor.

Rezultat su:

- precizniji zaključci
- dublje razumijevanje problema
- manji rizik od pogrešnih pretpostavki
- pouzdaniji odgovori kod kompleksnih pitanja

Ako želiš da AI **razumije prije nego što odgovori**, CVP je temeljni obrazac koji moraš koristiti.

2. Problem koji rješava

Jedan od najvećih problema u radu s AI-jem je brzina. Model često dobije pitanje i odmah isporuči odgovor koji zvuči samouvjereno, logično i uvjerljivo — ali nije nužno tačan, potpun ili dobro promišljen.

To stvara iluziju razumijevanja, i kod AI-ja i kod korisnika.

Cognitive Verifier Pattern postoji upravo da bi razbio tu iluziju. Njegova svrha nije da AI napiše više teksta, nego da **uspori proces razmišljanja** i provjeri da li je pitanje uopšte pravilno shvaćeno prije nego što se na njega odgovori.

3. Kako funkcioniše

U praksi, Cognitive Verifier Pattern mijenja način na koji postavljaš upit.

Umjesto da AI pitaš samo *šta* misli, eksplicitno ga tražiš da:

- provjeri da li je pravilno razumio pitanje
- razloži problem na manje misaone cjeline
- identifikuje moguće nejasnoće ili pogrešne pretpostavke
- tek onda izvede zaključak

Time se uloga AI-ja mijenja: **od generatora odgovora → u asistenta za razmišljanje**

Ovo je jedna od ključnih razlika između površnog korištenja AI-ja i ozbiljnog, profesionalnog rada s njim.

4. Kada ga koristiti

Cognitive Verifier Pattern je posebno koristan kada:

- pitanje djeluje jednostavno, ali krije kompleksnost
- postoji rizik od pogrešnih ili neizgovorenih pretpostavki
- tražiš analitički, strateški ili savjetodavni odgovor
- radiš sa važnim odlukama, a ne brzim informacijama

Bez ovog patterna, AI često:

- preskače ključne korake u razmišljanju
- zaključuje prerano
- gradi odgovor na pogrešnoj osnovi

Sa CVP-om dobijaš **kognitivno provjeren rezultat**, a ne samo lijepo formulisan tekst.

5. Primjer upotrebe

Zamisli da AI-ju postaviš sljedeće pitanje:

> "Kako da poboljšam produktivnost svog tima?"

Bez CVP-a, AI odmah nudi listu savjeta — generičnih, uopštenih, neadaptiranih tvojoj situaciji.

Sa CVP-om, prompt izgleda ovako:

```
Koristi Cognitive Verifier Pattern.
Pitanje: "Kako da poboljšam produktivnost svog tima?"
Prije nego odgovoriš, identificiraj pretpostavke i nejasnoće u pitanju.
Postavi mi podpitanja. Tek nakon mojih odgovora daj finalni prijedlog.
```

AI tada: 1. primjećuje da "produktivnost" nije definisana 2. pita: veličina tima? tip posla? koji problem konkretno? 3. tek sa tvojim odgovorima sastavlja relevantan plan

Razlika nije u odgovoru — nego u **procesu koji je doveo do odgovora**.

6. Ograničenja

CVP nije uvijek pravi izbor.

Nije koristan kada: - pitanje je jednostavno i jednoznačno - trebaš brz odgovor bez analize - kontekst je već potpuno definisan u promptu

Ako zlouptrebljavaš CVP na svakom pitanju, AI postaje spor i pretjerano analitičan — što gubi smisao u rutinskim zadacima.

Pravilo: koristi CVP za odluke i analize gdje pogrešna pretpostavka košta. Nemoj ga koristiti za brze informacije.

Reframing: kako postaviti pitanje da dobiješ bolji odgovor

Pattern koji mijenja način postavljanja pitanja kako bi AI dao precizniji i relevantniji odgovor.

---

1. Šta je to

Reframing je proces u kojem se postojeće pitanje **preformuliše** tako da postane:

- jasnije
- preciznije
- kontekstualno bogatije
- bolje usmjereno ka željenom ishodu

Kada tražiš od AI-ja da reframira tvoje pitanje, on:

- mijenja perspektivu iz koje se problem posmatra
- dodaje nedostajući kontekst
- uklanja nejasnoće i implicitne pretpostavke
- usmjerava pitanje ka tačnom *tipu* odgovora

Cilj nije da se pitanje "uljepša", nego da **AI zaista razumije šta želiš postići**, a ne samo šta si doslovno napisao.

---

2. Problem koji rješava

U radu s AI-jem problem često nije u odgovoru koji dobiješ, već u pitanju koje postaviš.

Model može savršeno "odgovoriti" na loše pitanje — i time te odvesti u pogrešnom smjeru.

AI radi isključivo s onim što mu daš.

Ako je pitanje preširoko, neodređeno, kontradiktorno ili zasnovano na pogrešnoj pretpostavci — i odgovor će biti takav, bez obzira koliko model bio napredan.

Reframing omogućava da se **pitanje optimizuje prije nego što AI uopšte krene u generisanje odgovora**.

Time se smanjuje rizik pogrešnih zaključaka i povećava kvalitet cjelokupnog outputa.

---

3. Kako funkcioniše

Reframing Technique postoji upravo da bi promijenila okvir razmišljanja iz kojeg AI polazi kada odgovara.

Umjesto da se pitanje postavi direktno, ono se prvo preoblikuje kako bi:

- dobilo jasniji kontekst
- uklonilo implicitne pretpostavke
- definisalo tip odgovora koji se očekuje

Na taj način AI ne reaguje na površinski tekst pitanja, nego na njegov **stvarni cilj i kontekst**.

---

4. Kada ga koristiti

Reframing se najčešće koristi kada:

- dobijaš preopšte ili “prazne” odgovore
- AI promašuje suštinu problema
- želiš tačno određen stil odgovora (analitički, sažet, tehnički, kreativan)
- radiš na kompleksnom ili višeslojnom pitanju

U takvim situacijama ne mijenjaš alat — mijenjaš **okvir pitanja**.

Reframing pomjera fokus sa *šta pitam* na *kako je pitanje postavljeno*.

---

5. Primjer upotrebe

Primjer:

Originalno pitanje:

> Kako mogu poboljšati svoj Instagram profil?

Reframed pitanje:

> Analiziraj moj Instagram profil kao marketing stručnjak i predloži konkretne korake za povećanje angažmana i rasta publike u narednih 30 dana.

---

6. Ograničenja

Reframing ne može popraviti problem ako:

- nema dovoljno konteksta
- cilj pitanja nije jasno definisan
- korisnik zapravo ne zna kakav odgovor želi

U tim situacijama potrebno je dodatno razjasniti problem prije nego što se pitanje može pravilno preformulisati.

Studija slučaja: CVP + Reframing u praksi

Cognitive Verifier Pattern + Reframing u praksi

Uvod

Do sada smo Cognitive Verifier Pattern i Reframing Technique posmatrali odvojeno.

U praksi, ove dvije tehnike se gotovo uvijek koriste **zajedno**.

Razlog je jednostavan:

- **Reframing** pomaže da se postavi *bolje pitanje*
- **Cognitive Verifier Pattern** pomaže da AI na to pitanje *razmišlja dublje i preciznije*

Jedan bez drugog daje ograničene rezultate. Zajedno, oni formiraju stabilan misaoni okvir.

Početno pitanje

Zamisli da AI-ju postaviš sljedeće pitanje:

> "Kako mogu poboljšati svoj Instagram profil?"

Naizgled jasno, ali u stvarnosti **preširoko i neodređeno**.

AI može odgovoriti na deset različitih načina — i nijedan ne mora biti stvarno primjenjiv.

Tu na scenu stupaju patterni.

Korak 1 — Reframing (preformulisanje pitanja)

Prvi korak je da od AI-ja zatražiš **preformulisanje pitanja** kako bi ono postalo jasnije i usmjerenije:

> "Kako mogu optimizovati sadržaj, učestalost objava i stil komunikacije na mom Instagram profilu kako bih povećao angažman publike?"

Ovim potezom:

- dodaješ kontekst
- definišeš fokus
- smanjuješ prostor za generičke odgovore

AI sada ne pogađa *šta bi mogao da radiš* — nego razumije *šta tačno želiš postići*.

Korak 2 — Cognitive Verifier Pattern (kognitivna provjera)

Zatim tražiš od AI-ja da **razloži pitanje na podpitanja** prije nego što ponudi rješenje:

1. Koja je tvoja trenutna ciljna publika?
2. Kakav sadržaj trenutno objavljuješ (fotografije, reels, tekstovi)?
3. Koliko često objavljuješ i u koje vrijeme?
4. Kakve rezultate trenutno imaš (reach, engagement)?

U ovoj fazi AI **ne pokušava da "bude pametan"**. On prvo prikuplja informacije koje su mu potrebne da bi imao **čvrstu osnovu za razmišljanje**.

Korak 3 — Sastavljanje odgovora

Tek nakon što dobije odgovore na podpitanja, AI sastavlja **konkretan plan** za optimizaciju Instagram profila.

Plan je sada:

- zasnovan na stvarnim podacima
- logički strukturiran
- prilagođen tvojoj konkretnoj situaciji

Ne na pretpostavkama, nego na razumijevanju konteksta.

Sažetak

- **Reframing** pomaže da postaviš bolje pitanje
- **Cognitive Verifier Pattern** pomaže da AI razmisli prije nego što odgovori

Zajedno, ove dvije tehnike pretvaraju AI iz generatora savjeta u **partnera u razmišljanju**.

Upravo ovdje počinje razlika između “dobio sam odgovor” i “dobio sam uvid”.

Flipped Interaction: kad AI vodi intervju umjesto tebe

Uvod

Ponekad problem nije u tome da AI nema dobar odgovor — nego u tome da ti nemaš dovoljno jasno definisan zahtjev.

Znaš otprilike šta želiš, ali ne znaš:

- koje informacije su zaista bitne,
- šta sve utiče na krajnji rezultat,
- kako da pitanje postaviš tako da AI ne nagađa.

Flipped Interaction Pattern postoji upravo za takve situacije.

Šta je Flipped Interaction Pattern

Flipped Interaction Pattern (FIP) je tehnika u kojoj **AI preuzima inicijativu u razgovoru**.

Umjesto da odmah izvrši tvoju komandu, AI:

- postavlja pitanja,
- razjašnjava kontekst,
- testira pretpostavke,
- prikuplja informacije,

sve dok ne stekne dovoljno razumijevanja da može dati **precizan, kontekstualno tačan i upotrebljiv odgovor**.

Drugim riječima, uloge su zamijenjene:

- ti postaješ izvor podataka,
- AI postaje istraživač, analitičar ili mentor.

Zbog ove zamjene uloga, obrazac se zove *flipped*.

Kako se pravi prompt

Osnovna struktura Flipped Interaction prompta izgleda ovako:

```
I would like you to ask me questions to achieve [X].
You should keep asking questions until [Y] is met.
(Optional) Ask me one question at a time. Ask me the first question.
```

Gdje:

- **[X]** predstavlja tvoj cilj (npr. plan, strategiju, tekst, analizu)
- **[Y]** predstavlja uslov prestanka (trenutak kada AI ima dovoljno informacija da može dati kvalitetan odgovor)

Ovim promptom **zabranjuješ AI-ju da žuri s rješenjem**. On prvo mora razumjeti problem.

Primjeri iz prakse

Primjer 1 — trening plan

> "I would like you to ask me questions to help create a weekly training plan. Ask questions until you know my fitness level, schedule, and recovery goals. Ask one question at a time."

Primjer 2 — email sekvenca

> "I would like you to ask questions to help me write an email sequence. Ask questions until you understand my audience and tone. Start with the first question."

U oba primjera, AI **ne nudi rješenje odmah**. Prvo gradi kontekst — kao dobar konsultant ili trener.

Gdje se FIP koristi najčešće

Flipped Interaction Pattern je posebno koristan:

- kada znaš *šta želiš*, ali ne znaš kako to precizno definisati
- kada želiš da AI vodi brainstorming proces
- kod personalizovanih planova, strategija i tekstova
- kada je kontekst važniji od brzine odgovora
- kada ne želiš generičke savjete

Ovo je pattern za situacije u kojima je **razumijevanje važnije od brzine**.

Analogija

Zamisli da AI nije pomoćnik koji čeka komandu, nego **intervjuer**.

Postavlja prava pitanja:

- o tebi,
- o cilju,
- o ograničenjima,
- o prioritetima,

sve dok ne shvati tvoju situaciju. Tek tada daje odgovor koji zaista *pogađa u centar*.

Primjer gotovog prompta

```
I would like you to ask me questions to design a personalized content strategy
Ask me questions until you fully understand my target audience, content goals,
Ask one question at a time. Ask me the first question.
```

Ovim promptom praktično kažeš AI-ju: **"Ne pretpostavljaj. Prvo me upoznaj."**

Mini vježba

Isprobaj sljedeće:

```
I would like you to ask me questions to improve the clarity and tone of my Link
Keep asking until you fully understand my audience, purpose, and professional
Ask one question at a time. Start now.
```

Primijeti kako se razgovor ponaša drugačije nego kod klasičnog prompta.

Sažetak

Flipped Interaction Pattern preokreće uloge u razgovoru.

AI postavlja pitanja — ti daješ kontekst.

Rezultat nisu brzi odgovori, nego **precizni, relevantni i stvarno upotrebljivi outputi**, prilagođeni tvojoj konkretnoj situaciji.

Komande i kompozicija promptova

Komandna biblioteka: komande za ponašanje modela

Uvod

Kada ljudi pričaju o prompt engineeringu, često misle na duge i kompleksne promptove. U praksi, to brzo postane neefikasno. Svaki put iznova objašnjavaš kontekst, ton, način razmišljanja — i nadaš se da će AI "pogoditi" kako treba.

Ova tabela je nastala iz potrebe da se taj haos svede na minimum.

Umjesto da svaki put pišeš novi prompt od nule, koristiš **kratke komande** koje AI-ju jasno govore *kako treba da razmišlja*, ne samo *šta da uradi*.

Ovo nije lista trikova. Ovo je **kontrolni jezik** za upravljanje ponašanjem AI-ja.

Kako koristiti ovu biblioteku

Svaka komanda u tabeli predstavlja **jedan način razmišljanja** koji možeš "uključiti" po potrebi.

U praksi to izgleda ovako:

- u promptu jasno navedeš koju komandu želiš da AI koristi
- kombinuješ ih po potrebi (npr. FIP + CVP)
- dobijaš konzistentno ponašanje, umjesto nasumičnih odgovora

Ne moraš pamtiti sve. Dovoljno je da znaš **kada koji pattern ima smisla**.

Kako čitati tabelu

Ako gledaš tabelu kao cjelinu, primijetićeš tri važne stvari:

- neke komande upravljaju **interakcijom** (npr. FIP)
- neke upravljaju **provjerom i kvalitetom razmišljanja** (CVP, CoVP, SFP)
- neke upravljaju **stilom, kontekstom i ulogom** (RT, EXP, EMP, CTP)

Drugim riječima, ne biraš "bolji prompt", nego **pravilan misaoni režim** za zadatak koji imaš. To je razlika između nasumičnog korištenja AI-ja i kontrolisanog rada.

Komandna tabela

Komanda	Puno ime	Opis ponašanja	Kada koristiti	Primjer upotrebe
FIP	*Flipped Interaction Pattern*	AI **postavlja pitanja tebi** kako bi prikupio informacije prije nego što odgovori.	Kada ne znaš tačno šta želiš, ili želiš da AI pomogne definisati cilj, brief ili problem.	"Koristi FIP da mi postaviš pitanja dok ne shvatiš moje ciljeve i publiku, pa onda napravi plan sadržaja."
CVP	*Cognitive Verifier Pattern*	AI **provjerava i koriguje vlastito razmišljanje** prije nego što da odgovor.	Kada želiš da odgovor bude tačan, logičan i samoproveren.	"Koristi CVP da provjeri svoj odgovor o razlici između fine-tuninga i promptinga."

Komanda	Puno ime	Opis ponašanja	Kada koristiti	Primjer upotrebe
RT	*Reframing Technique*	AI **preformuliše pitanje** iz više uglova da bi generisao bolji izlaz.	Kada želiš dublje, kreativnije ili preciznije tumačenje teme.	"Koristi RT da preformulišeš moje pitanje o digitalnom fokusu na tri različita načina."
CoVP	*Chain of Verification Pattern*	AI koristi **višestepenu provjeru**: generiše odgovor, zatim ga provjerava kroz logičke filtere.	Kada je zadatak složen i traži višeslojno rezonovanje.	"Koristi CoVP da provjeri strategiju u tri koraka: logika, izvedivost, jasnoća."
FRT	*Fact-Checking Reframing Technique*	AI preformuliše zadatak uz **ugrađenu provjeru činjenica** i izvora.	Kada radiš s podacima, citatima, edukativnim sadržajem.	"Koristi FRT da preformulise moj tekst i provjeri tačnost svih tvrdnji."
SOP	*Step-by-Step Output Pattern*	AI razlaže zadatak u **sekvencu logičnih koraka**.	Kada želiš jasnoću, procesno razmišljanje ili uputstvo.	"Koristi SOP da objasniš proces spajanja Notiona s n8n korak po korak."
EXP	*Expert Persona Pattern*	AI preuzima **ulogu eksperta** određenog profila i iz te perspektive odgovara.	Kada želiš ton i preciznost specifične profesije.	"Koristi EXP i ponašaj se kao senior n8n inženjer koji objašnjava početniku."
CTP	*Context-Transfer Pattern*	AI **pamti i koristi kontekst** iz prethodnih poruka da bi održao konzistentnost.	Kada radiš duge projekte (eBook, kurs, workflow).	"Koristi CTP i nastavi gdje smo stali s RAG workflowom."
EMP	*Emotion Modulation Pattern*	AI prilagođava **ton, stil i emocionalnu nijansu** teksta.	Kada želiš emocionalno usklađen ton komunikacije.	"Koristi EMP da prepravi ovaj tekst u smiren mentorski ton."
SFP	*Self-Feedback Pattern*	AI daje **komentar o sopstvenom odgovoru** — šta je bilo dobro, a šta bi poboljšao.	Kada želiš model koji uči kroz samorefleksiju.	"Koristi SFP da ocijeniš kvalitet svog prethodnog odgovora."

Sažetak

Prompt Pattern Command Library je tvoj **brzi interfejs za upravljanje AI-jem**.

Umjesto da objašnjavaš sve iznova, koristiš kratke komande koje jasno definišu:

- ko AI treba da bude
- kako treba da razmišlja
- i kada da stane

Ako iz cijele knjige zadržiš samo jednu stvar za svakodnevni rad — ova tabela je vrlo dobar kandidat.

Kombinovanje komandi: misaoni režimi i kontrola

Uvod

Jedna od najvećih prednosti rada s patternima je to što ih ne moraš koristiti izolovano. U stvarnom radu, najkvalitetniji rezultati dolaze kada **kombinuješ više misaonih režima** unutar jednog prompta.

Umjesto da AI-ju kažeš samo *šta* da uradi, ti mu govoriš:

- kako da prikuplja informacije
- kako da razmišlja
- kako da provjeri vlastite zaključke

Svaka komanda postaje **jedan sloj u procesu razmišljanja**.

Kako kombinovanje patterna funkcioniše u praksi

Osnovna logika kombinovanja izgleda ovako:

1. **FIP** — AI prvo prikuplja informacije od tebe kroz pitanja.
2. **RT** — zatim preformuliše i proširuje razumijevanje problema.
3. **CVP** — na kraju provjerava i koriguje vlastiti zaključak.
4. *(Opcionalno)* **CoVP** ili **SOP** — dodatna validacija ili struktura procesa.

Na ovaj način AI ne djeluje kao jedan model, nego kao **tim**: jedan pita, drugi analizira, treći provjerava.

Primjeri kombinacija

Kombinacija	Šta AI radi	Kada koristiti
FIP + CVP	AI te prvo ispituje da bi razumio cilj, zatim sam provjerava finalni odgovor.	Kada nisi siguran kako da definišeš zadatak ili cilj.
RT + CVP	AI preformuliše pitanje iz više uglova, zatim testira logiku svakog rješenja.	Kada želiš bolji, precizniji ili dublji uvid u temu.
EXP + SOP + CVP	AI govori kao ekspert, objašnjava korak-po-korak i provjerava svaki korak.	Kada želiš edukativni, strukturisan i provjeren tekst.
FIP + RT + CoVP	AI postavlja pitanja, preformuliše koncept iz više perspektiva i validira sve slojeve.	Kada praviš kompleksne projekte (AI sistemi, workflowi, planovi).

Praktičan primjer

Prompt koji kombinuje **FIP + CVP**:

```
Koristi FIP + CVP.
Pomogni mi da napravim strukturu edukativnog kursa o AI strategiji.
Postavljaj mi pitanja dok ne shvatiš moju publiku i ciljeve,
zatim sam provjeri i koriguj svoj finalni prijedlog prije nego što ga pošalješ
```

Tok rada izgleda ovako:

1. AI te intervjuiše (FIP).
2. Sastavlja prijedlog kursa.
3. Provjerava vlastitu logiku prije isporuke (CVP).

Rezultat je strukturisan, kontekstualno tačan i samoproveren odgovor.

Meta Combo Prompt Template (FIP + RT + CVP)

```
You are an advanced AI strategist and educator.

Use the Meta Combo Pattern (FIP + RT + CVP) to process my request.

1. Flipped Interaction (FIP):
  - Start by asking me targeted questions to fully understand my goal, audienc
   - Ask one question at a time.
  - Continue until you are confident that you understand what I truly need.

2. Reframing Technique (RT):
  - Once you have enough context, reframe my topic or question from at least tw
     (e.g. conceptual, practical, creative, analytical).
  - Briefly explain your reasoning for each framing before continuing.

3. Cognitive Verifier Pattern (CVP):
  - After generating your main answer, verify your own reasoning by asking you
       • Is this logically sound and contextually relevant?
      • Does it align with the information gathered during FIP?
       • Are there any unclear or weak points?
   - Refine and improve your answer based on that verification.

4. Final Output:
  - Present the verified and improved answer clearly and structured.
   - Add a short section at the end titled "Verification Notes"
    summarizing what you refined or corrected in your reasoning.
```

Faza	Uloga	Rezultat
FIP	AI te “intervjuiše” da razumije cilj	Odgovor temeljen na stvarnim potrebama, ne pretpostavkama
RT	AI preformuliše temu i sagleda više uglova	Dublje razumijevanje i kvalitetniji sadržaj
CVP	AI provjerava svoj zaključak	Tačan, jasan i dosljedan izlaz

Sažetak

Kombinovanjem pattern-komandi učiš AI da **razmišlja u više slojeva** — da pita, analizira i sam se provjerava prije nego što govori. To je prelaz sa “promptanja” na **kontrolisano rezonovanje**.

Šabloni: kako napraviti ponovljiv output

Uvod

Jedan od glavnih razloga zašto AI često daje neujednačene odgovore nije nedostatak znanja, nego nedostatak strukture.

Jednom dobiješ jasan, fokusiran tekst. Drugi put — haotičan, razvučen ili nedorečen. Iako si postavio isto pitanje.

Template Pattern rješava upravo taj problem.

Ovaj pattern koristi **unaprijed definisan misaoni šablon** koji model mora slijediti. Kao u Excel tabeli: svaka "ćelija" ima svoju funkciju, a preskakanje nije dozvoljeno.

Kako Template Pattern funkcioniše

Template Pattern se sastoji od nekoliko jasno razdvojenih elemenata koji zajedno čine stabilan okvir razmišljanja.

Svrha (Purpose)

Prvi korak je da definišeš **šta želiš standardizovati**.

To mogu biti: lekcije, poglavlja knjige, analize, poslovni izvještaji, postovi za društvene mreže, edukativne bilješke.

Bez jasno definisane svrhe, šablon nema smisla.

Okvir (Frame)

Zatim određuješ **redoslijed sekcija** koje se uvijek ponavljaju.

Na primjer: **Uvod → Objašnjenje → Primjer → Zaključak**

Model mora poštovati taj raspored, bez obzira na temu. Ovdje ne kontrolišeš sadržaj — nego tok razmišljanja.

Blokovi (Blocks)

Svaka sekcija ima **jasnu ulogu**:

- **Hook** — privlači pažnju i uvodi temu
- **Core idea** — objašnjava suštinu pojma
- **Example** — daje konkretan, primjenjiv prikaz
- **Summary** — zatvara krug i učvršćuje poruku

AI ne improvizuje gdje šta ide. Svaki dio ima svoju funkciju.

Konzistentnost

Bez obzira na temu, struktura ostaje ista. To stvara:

- prepoznatljiv stil,
- profesionalni ritam,
- lakše čitanje,
- lakšu upotrebu u serijama sadržaja.

Čitalac (ili korisnik) zna šta da očekuje.

Fleksibilnost

Template nije zatvor. Možeš ga prilagoditi: dužini, tonu, publici, nivou stručnosti.

Ali **okvir ostaje isti**.

Upravo ta kombinacija stabilnosti i fleksibilnosti čini ovaj pattern moćnim.

Prompt (primjer)

> Use the Template Pattern to create a short educational note.
>
> **Topic:** "What is Prompt Engineering?"
>
> **Template Structure:**
>
> 1. *Hook:* Start with a one-line metaphor or relatable example.
> 2. *Definition:* Explain the concept in one paragraph.
> 3. *Example:* Show a simple real-world application.
> 4. *Summary:* End with one sentence that captures the essence.
>
> **Tone:** Clear, confident, and teacher-like — not academic.

Ovim promptom ti ne govoriš AI-ju *šta* da misli, nego *kako* da strukturira razmišljanje.

Sažetak

Template Pattern omogućava AI-ju da piše **dosljedno, jasno i profesionalno**.

Posebno je koristan kada:

- radiš serije sadržaja,
- gradiš edukativne materijale,
- želiš stabilnu formu, ali promjenjiv sadržaj,
- praviš knjigu, kurs ili sistem znanja.

Forma ostaje ista — sadržaj se mijenja.

Meta-jezik: kad modelu uvedeš svoj sistem pojmova

Uvod

Jedan od najnaprednijih načina rada s AI-jem je prestati mu stalno objašnjavati iste stvari. Umjesto toga, ti mu **jednom definišeš značenje svojih izraza** — i koristiš ih kao kod.

To je ideja Meta Language Creation Patterna.

Ovdje ne prilagođavaš sebe AI-ju. Ti učiš AI **kako da razumije tebe**.

Kako funkcioniše Meta Language Pattern

Osnovna struktura je jednostavna:

> When I say X, I mean Y.

Gdje je:

- **X** — riječ, simbol ili fraza koju definišeš
- **Y** — tačno ponašanje ili radnja koju AI treba izvršiti

Jednom definisan, ovaj "jezik" postaje dio tvog rada s AI-jem.

Primjeri i zašto su moćni

Primjeri poput:

- `variations(...)`
- `explain(...)`
- definisanja zavisnosti između zadataka

omogućavaju ti da:

- skratiš promptove
- povećaš preciznost
- izbjegneš ponavljanje

Vremenom gradiš **sopstveni radni vokabular** koji AI razumije bez dodatnih objašnjenja.

Zašto koristiti ovaj obrazac

- Povećava **preciznost** – AI zna tačno šta misliš.
- Smanjuje **ponavljanje** – ne moraš stalno objašnjavati iste stvari.
- Omogućava **sopstveni vokabular** – gradiš svoj "radni jezik" s AI-jem.
- Koristan je kod **kompleksnih projekata, edukacije i automatizacija**.

Sažetak

Meta Language Creation Pattern ti daje dugoročnu prednost. Umjesto da svaki put objašnjavaš šta želiš, ti gradiš jezik kojim komuniciraš s AI-jem brzo, precizno i bez šuma. To je posebno moćno u kompleksnim projektima, edukaciji i automatizaciji.

Recepti: vođenje modela kroz korake bez nagađanja

Uvod

Jedan od najčešćih razloga zašto AI daje neujednačene rezultate je to što mu zadatak djeluje jasan — ali proces nije. Model zna *šta* treba uraditi, ali ne zna *kako tačno* da dođe do rezultata.

Recipe Pattern rješava upravo taj problem.

Ovaj pattern tretira prompt kao **recept**: niz jasno definisanih sastojaka i koraka koji vode do predvidivog ishoda. Umjesto da se oslanjaš na improvizaciju modela, ti mu daješ strukturu rada — redoslijed, tempo i očekivani završetak.

Drugim riječima, AI ne "kuha po osjećaju". On slijedi recept.

Šta je Recipe Pattern i zašto je važan

Recipe Pattern je prompting pristup u kojem zadatak razlažeš na **jasne, numerisane korake**, slično receptu za jelo. Svaki korak ima svoju funkciju i prirodno vodi u sljedeći.

Recipe Pattern **ne ograničava kreativnost**, nego uklanja konfuziju.

Bez recepta, AI često:

- preskače važne korake
- miješa analizu i zaključak
- daje odgovor prije nego što je "obradio" problem

Sa receptom, model:

- zna kojim redoslijedom razmišlja
- ostaje fokusiran na zadatak
- proizvodi stabilne i ponovljive rezultate

Zato je ovaj pattern posebno koristan kada želiš **dosljednost**, a ne iznenađenja.

Kako Recipe Pattern funkcioniše u praksi

Recipe Pattern se obično sastoji od tri do pet koraka. Primjer osnovne strukture izgleda ovako:

1. **Analiza zadatka** — Model jasno definiše šta je problem i koje su ključne varijable.
2. **Prikupljanje ili organizacija informacija** — Razdvaja relevantne od nerelevantnih podataka.
3. **Obrada / razrada rješenja** — Povezuje informacije u smislen tok.
4. **Finalni rezultat** — Daje jasan, sažet i upotrebljiv odgovor.
5. *(Opcionalno)* **Provjera ili poboljšanje** — Model kratko provjerava da li je rezultat u skladu s ciljem.

Ovaj redoslijed nije krut, ali **redoslijed mora postojati**. Bez njega, AI se vraća nagađanju.

Primjer Recipe Pattern prompta

```
Koristi Recipe Pattern.

1. Prvo analiziraj moj zahtjev i sažmi šta tačno treba uraditi.
2. Zatim identifikuj ključne informacije koje su potrebne.
3. Nakon toga razradi rješenje korak po korak.
4. Na kraju daj jasan i praktičan finalni odgovor.
```

Ovim promptom ti:

- vodiš model kroz proces
- sprečavaš preskakanje faza
- dobijaš odgovor koji ima unutrašnju logiku

Ako želiš dodatnu kontrolu, Recipe Pattern se lako kombinuje sa drugim obrascima, npr.:

- **Recipe + CVP** (na kraju provjera logike)
- **Recipe + Template Pattern** (strukturisan izlaz)

Sažetak

Recipe Pattern uči AI da **radi po procesu, ne po osjećaju**. Umjesto brzih, često površnih odgovora, dobijaš stabilan tok razmišljanja koji vodi do jasnog rezultata. Kada ti je važna pouzdanost, ponovljivost i kontrola — ovaj pattern postaje nezamjenjiv.

Menu pristup: izbor bez gubitka kontrole

Uvod

Jedan od suptilnih problema u radu s AI-jem je prevelika sloboda.

Kad mu daš otvoren zadatak, model često pokušava da bude "koristan" na sve moguće načine — i završi sa odgovorom koji je širok, neujednačen ili promašuje ono što ti je zapravo trebalo.

Menu Action Pattern rješava taj problem tako što **ne tražiš odgovor**, nego **ponudiš opcije**.

Umjesto: "Šta misliš o ovome?" ti kažeš: "Evo menija. Izaberi jednu od ovih akcija."

AI tada ne improvizuje smjer — on bira **unutar granica koje si ti postavio**.

Šta je Menu Action Pattern i zašto je važan

Menu Action Pattern je prompting obrazac u kojem AI-ju daješ **ograničen skup dozvoljenih akcija**, obično u formi liste ili menija. Model ne odlučuje *šta bi sve mogao*, nego *šta smije* da uradi.

Ovaj pattern je posebno važan jer:

- smanjuje rasipanje odgovora
- povećava preciznost i fokus
- čini interakciju predvidivom

U praksi, Menu Action Pattern pretvara AI iz "pričalice" u **kontrolisanog asistenta**.

Kako Menu Action Pattern funkcioniše u praksi

Osnovna logika je jednostavna:

1. Ti definišeš **menije akcija**
2. AI mora izabrati **jednu (ili više) ponuđenih opcija**
3. AI izvršava samo izabranu akciju — ništa van toga

Primjer menija može izgledati ovako:

- A) Analiziraj tekst i sažmi ga
- B) Preformuliši tekst za jasniji stil
- C) Pronađi logičke greške
- D) Postavi pitanja za dodatno razjašnjenje

AI tada:

- prvo bira opciju
- zatim izvršava tačno tu radnju

Time dobijaš **dvostepenu kontrolu**: kontrolišeš i *šta* radi i *kako* razmišlja.

Primjer Menu Action Pattern prompta

```
Koristi Menu Action Pattern.

Dostupne akcije:
A) Sažmi tekst u 5 tačaka
B) Preformuliši tekst za početnike
C) Identifikuj nejasne dijelove
```

```
D) Postavi pitanja za dodatni kontekst

Prvo izaberi jednu akciju i objasni zašto.
Zatim izvrši samo izabranu akciju.
```

Ovaj obrazac:

- sprječava “odgovaranje na sve odjednom”
- tjera AI da pokaže namjeru prije izvršenja
- povećava kvalitet rezultata

Gdje je ovaj pattern najkorisniji

Menu Action Pattern je izuzetno koristan kada:

- radiš **reviziju teksta ili ideja**
- želiš **kontrolisan brainstorming**
- gradiš **AI asistenta ili agenta**
- vodiš AI kroz kompleksne odluke korak po korak

Posebno je moćan u kombinaciji sa:

- **FIP** (AI postavlja pitanja, ali unutar menija)
- **CVP** (provjera nakon izvršenja akcije)
- **Recipe Pattern** (meniji po fazama recepta)

Kratka analogija

Zamisli AI kao zaposlenog u restoranu.

Bez menija — pita goste svašta, predlaže sve i svašta, zbunjuje. Sa menijem — zna šta se nudi, šta se ne nudi i radi posao precizno.

Menu Action Pattern je **jelovnik za razmišljanje**.

Sažetak

Menu Action Pattern ti omogućava da AI-ju daš slobodu izbora **bez gubitka kontrole**. Umjesto otvorenih odgovora, dobijaš fokusirane akcije. Kada želiš jasnoću, preciznost i predvidivost — ovaj pattern je jedno od najjačih oruđa koje imaš.

Napredni menu sistemi: od izbora do agent menija

Uvod

U prethodnoj lekciji vidjeli smo kako **Menu Action Pattern** ograničava AI i daje mu jasne opcije za djelovanje. To je bio prvi korak ka kontroli.

Ali stvarni problemi ne stanu u jedan meni.

U stvarnom radu:

- zadaci imaju faze
- odluke imaju posljedice
- jedna akcija otvara potrebu za sljedećom

Tu prestaje "meni kao lista" i počinje **meni kao sistem**.

Ova lekcija objašnjava kako se jednostavan Menu Action Pattern razvija u:

- **Advanced Menu** (višeslojni meniji)
- **AI Agent Menu System** (trajni sistem odlučivanja)

Ovdje AI više ne "odgovara". Ovdje AI **učestvuje u procesu**.

Od osnovnog menija do naprednog menija

Osnovni Menu Action Pattern izgleda ovako:

> "Izaberi jednu od ponuđenih akcija."

To je korisno — ali ograničeno.

Advanced Menu uvodi hijerarhiju i stanje:

- meniji zavise od prethodnih izbora
- nisu sve opcije uvijek dostupne
- izbor u jednoj fazi utiče na narednu

Drugim riječima: AI ne bira samo *šta* radi — nego *kada* i *zašto*.

Osnovni Menu Action Pattern (core)

Struktura je jednostavna:

```
Menu of Actions:
1. [akcija 1]
2. [akcija 2]
3. [akcija 3]
4. [akcija 4]

User will choose one number. Perform only that action.

Input:
[tekst]
```

Tri pravila:

1. Akcije moraju biti glagoli + rezultat.
2. Samo jedna akcija.
3. Input ide na kraj.

Advanced Menu Pattern

Advanced Menu Pattern je proširenje osnovnog menija u **višefazni sistem odluka**.

Umjesto jedne liste opcija, imaš:

- glavni meni (high-level odluke)
- podmenije (specifične akcije)
- uslove prelaska (pravila toka)

Primjer logike:

- Ako je zadatak "analiza" → prikaži analitički meni
- Ako je zadatak "pisanje" → prikaži kreativni meni
- Ako nema dovoljno informacija → aktiviraj FIP meni

AI se tada ponaša kao operater koji zna koji alat smije koristiti u kojoj fazi.

Primjer strukture:

```
Menu of Actions:
1. Analyze
   - 1a. Find weaknesses
   - 1b. Find contradictions
2. Transform
   - 2a. Rewrite clearly
   - 2b. Rewrite persuasively
3. Generate
   - 3a. Create a plan
   - 3b. Create a checklist
4. Automate
   - 4a. Suggest n8n nodes
   - 4b. Suggest Notion structure

Rules:
- User selects one code (e.g., 2a).
- Perform only that action.
- If unclear, ask a clarifying question.

Input:
[tekst]
```

Menu Pattern za AI Agente (Agent Decision Matrix)

Ovo je najjača verzija. Ovdje ne biraš ti — bira AI agent. Ali ti ga učiš *kako* da bira.

```
Agent Menu (choose one based on the user goal):
1. Understand
   - Summarize
   - Identify problem
   - Detect missing info
2. Plan
   - Create steps
   - Propose workflow
   - Define required tools
3. Act
```

```
   - Generate content
   - Transform text
   - Produce structured data
4. Verify
   - Evaluate quality
   - Find errors
   - Improve clarity
5. Next Step Logic
   - Choose the optimal next action
   - Explain why
   - Perform it

Rules:
- Agent must choose one action per cycle.
- Agent must justify the choice in one sentence.
- Output must match the action.
- If the goal is not clear → go to Step 1 (Understand).

Input:
[user request]
```

Ovo je **decision brain** za AI. Prava inteligencija nastaje tek kada agent može da bira **šta je najbolji sljedeći korak**.

Kako nastaje AI Agent Menu System

AI Agent Menu System ide korak dalje. On podrazumijeva da:

- AI **pamti stanje zadatka**
- zna u kojoj se fazi nalazi
- zna koje su akcije dozvoljene u toj fazi
- zna kada mora stati i tražiti input

To više nije prompt. To je **pravilo ponašanja kroz cijelu sesiju**.

U tom sistemu:

- meniji nisu jednokratni
- oni su **trajni kontrolni mehanizam**
- AI ne izlazi iz uloge bez dozvole

Zato govorimo o *agentu*, a ne o odgovoru.

Kratka analogija

Zamisli da zapošljavaš asistenta.

Bez menija — pita te za sve, stalno. Sa menijem — zna koje poslove radi. Sa **agent menu sistemom** — zna šta mu je posao, kada da te pita, kada da djeluje sam.

To je razlika između pomoćnika i agenta.

Sažetak

- **Menu Action Pattern** daje izbor
- **Advanced Menu** uvodi strukturu i faze
- **AI Agent Menu System** uvodi stanje, pravila i kontinuitet

Ovdje AI prestaje biti alat za odgovore i postaje **kontrolisan misaoni sistem**.

Few-shot: učenje iz primjera koje mu pokažeš

Uvod

Jedna od najvećih zabluda u radu s AI-jem je uvjerenje da model "razumije" šta želiš čim mu to napišeš riječima. U praksi, riječi često nisu dovoljne. AI ne dijeli tvoje iskustvo, kontekst ni intuiciju.

Ali dijeli nešto drugo: **sposobnost prepoznavanja obrazaca.**

Few-Shot Prompting koristi upravo to.

Umjesto da objašnjavaš *šta želiš*, ti **pokažeš kako izgleda dobar odgovor**. AI tada ne pogađa. On **imitira obrazac koji si mu dao**.

Šta je Few-Shot Prompting (i šta NIJE)

Few-Shot Prompting je tehnika u kojoj AI-ju daješ **nekoliko konkretnih primjera ulaza i izlaza**, a zatim od njega tražiš da proizvede novi izlaz po istom obrascu.

Ključna stvar: ne učiš ga pravilima — **učiš ga stilom i strukturom**.

Važno je i šta Few-Shot *nije*:

- nije fine-tuning
- nije trajno učenje
- nije pamćenje

Model ne "pamti" primjere dugoročno. On ih koristi **u tom trenutku kao referentni uzorak**.

Primjer Few-Shot

Primjer 1: Input: "Organizuj mi dan." Output: - Fokus dana: Konsolidacija AI OS projekta - Zadaci: 1) Obsidian zapis, 2) n8n test, 3) analiza ChatGPT lekcija - Upozorenje: Ne rasipati fokus na nove alate

Primjer 2: Input: "Kako da poboljšam workflow?" Output: - Optimizacija: Uvedi logging u Notion - Rizik: Previše paralelnih projekata - Preporuka: Jedan workflow dnevno do kraja

Novi input: "Kako da unaprijedim svoj RAG sistem?"

Output modela: - Poboljšanje: Dodaj feedback scoring metrikom - Optimizacija: Ubrzaj embedding ingest proces - Preporuka: Dokumentuj sve edge-caseove u Obsidianu

LLM kopira stil, strukturu, ton, format.

Kada se ne može koristiti

Few-shot nije dobro rješenje za:

- **Zadacima koji zahtijevaju visoku preciznost bez varijacija** — npr. matematički proračuni, medicinske doze.
- **Kada radiš s velikim inputima** — dugi tekst može zbuniti model.
- **Kada je zadatak previše specifičan** — nema smislen uzorak.
- **Kada želiš potpuno slobodan kreativni odgovor** — Few-shot ograničava kreativnost modela.

Zašto Few-Shot Prompting radi tako dobro

AI modeli su trenirani na ogromnim količinama primjera. Njihova snaga nije u zaključivanju kao kod čovjeka, nego u **prepoznavanju i nastavku obrazaca**.

Kada mu daš dobar primjer, ti mu praktično kažeš:

> "Ovako izgleda ispravno razmišljanje u ovom kontekstu."

Zato Few-Shot Prompting:

- dramatično povećava dosljednost
- smanjuje pogrešne interpretacije
- čuva stil, ton i strukturu
- daje stabilne rezultate čak i kod kompleksnih zadataka

U mnogim slučajevima, **dva dobra primjera vrijede više od deset rečenica objašnjenja**.

Osnovna struktura Few-Shot prompta

Tipičan Few-Shot prompt ima tri dijela:

1. **Instrukcija** — Kratko objašnjenje šta AI treba da uradi.
2. **Primjeri (shots)** — Nekoliko parova *Input* → *Output*.
3. **Novi zahtjev** — Ulaz za koji želiš novi odgovor po istom obrascu.

Primjer strukture:

```
Zadatak: Uradi X po obrascu ispod.

Primjer 1:
Input: ...
Output: ...

Primjer 2:
Input: ...
Output: ...

Sada uradi isto za:
Input: ...
```

AI tada prepozna obrazac, prilagodi ga novom ulazu i generiše odgovor koji "liči" na prethodne.

Kako birati dobre primjere

Few-Shot Prompting je **tačno onoliko dobar koliko su dobri primjeri** koje daš.

Dobar primjer mora biti:

- jasan
- tipičan (ne ekstrem)
- tačno predstavljati stil i dubinu koju želiš

Loši primjeri zbunjuju model, uvode kontradikcije i proizvode nestabilne rezultate.

Ako želiš analitički stil — svi primjeri moraju biti analitički. Ako želiš jednostavan jezik — svi primjeri moraju biti jednostavni.

AI ne "prosječi". On **kopira dominantni obrazac**.

Gdje je Few-Shot Prompting najkorisniji

Few-Shot Prompting je posebno moćan kada:

- želiš **konzistentan stil pisanja**
- radiš **klasifikaciju ili evaluaciju**
- standardizuješ odgovore (QA, feedback, ocjene)
- praviš **interni "jezik" sa AI-jem**
- gradiš agente koji moraju reagovati predvidivo

Kada je Few-Shot najbolji izbor:

- **Specifičan stil** — pisanje u određenom tonu (smiren, motivacioni, klinički…)
- **Specifičan format** — lista, tabela, JSON, bullet points, email, caption…
- **Dosljednost** — kada model *svaki put* mora davati iste vrste odgovora
- **Replikacija složenog zadatka** — pravila koja se teško objasne, ali lako pokažu

Few-Shot + drugi patterni

Few-Shot Prompting postaje još jači kada se kombinuje sa:

- **Template Pattern** → primjeri + fiksna struktura
- **Menu Action Pattern** → primjeri za svaku dozvoljenu akciju
- **CVP** → provjera da li novi odgovor slijedi obrazac primjera
- **Meta Language Creation Pattern** → primjeri koji definišu značenje "tvog jezika"

Kratka analogija

Zamisli da nekome objašnjavaš kako da napiše dobar email. Možeš pričati sat vremena. Ili možeš pokazati **tri dobra emaila**. Većina ljudi će naučiti iz primjera. AI — još brže.

Sažetak

Few-Shot Prompting koristi ono u čemu je AI najjači: prepoznavanje obrazaca. Umjesto da objašnjavaš pravila, ti pokazuješ primjere. Kada želiš dosljednost, stabilan stil i predvidivo ponašanje — Few-Shot Prompting je jedan od najpouzdanijih alata koje imaš.

Razmišljanje i rad u koracima

Razmišljanje u koracima: kad i kako tražiti korak-po-korak

Uvod

Kada AI nema jasno usmjeren proces razmišljanja, on često daje brze, ali površne odgovore. Chain-of-Thought Prompting (CoT) postoji da bi se to promijenilo.

Umjesto da tražiš rješenje odmah, tražiš **tok razmišljanja koji vodi do rješenja**.

Šta je CoT i zašto je važan

Chain-of-Thought Prompting je tehnika kojom eksplicitno tražiš od modela da razmišlja kroz niz logičkih koraka prije nego što da konačan odgovor.

Bez CoT-a, AI:

- često nagađa
- preskače važne pretpostavke

- daje odgovor bez objašnjenja

Sa CoT-om, AI:

- analizira kontekst
- povezuje korake logički
- prepoznaje i ispravlja greške u toku rezonovanja

Zato CoT ne poboljšava samo odgovor — nego **kvalitet razmišljanja**.

Kako se koristi (šabloni + CVP checkpoint)

Osnovna instrukcija može biti jednostavna:

> "Razmisli korak po korak prije nego daš odgovor."

Strukturiraniji primjer:

> "Prvo analiziraj problem, zatim iznesi moguće pristupe, procijeni ih logički i tek na kraju daj zaključak."

Napredna varijanta (CoT + CVP):

```
Koristi Chain-of-Thought pristup da razmisliš korak po korak.
Na kraju primijeni Cognitive Verifier Pattern:
provjeri logičku povezanost koraka i konzistentnost zaključka.
```

Ako model ne razdvaja korake ili ne provjerava logiku — prompt nije dovoljno precizan.

Primjeri

Osnovni CoT prompt: "Razmisli korak po korak prije nego daš odgovor."

Strukturirani CoT prompt: "Prvo analiziraj problem, zatim iznesi moguće pristupe, procijeni ih logički i tek na kraju daj zaključak."

Napredni CoT + CVP prompt: "Koristi Chain-of-Thought pristup da razmisliš korak po korak. Na kraju, primijeni Cognitive Verifier Pattern: provjeri da li je svaki korak logički povezan, da li postoji rupa u rezonovanju i potvrdi da je zaključak konzistentan sa početnom premisom."

Kada primijeniti:

Koristi CoT svaki put kada:

- Rješavaš kompleksan problem (npr. dizajn workflowa, AI-strategija)
- Želiš da AI objasni zašto nešto radi, ne samo šta radi
- Testiraš vlastitu logiku kroz AI kao sparing-partnera

Kratka analogija

Zamisli AI-a kao detektiva.

Bez CoT-a — odmah kaže "kriv je onaj lik u kaputu." Sa CoT-om — prvo analizira tragove, motiv, alibi i tek onda donosi presudu.

Ti zapravo učiš AI da misli kao detektiv, ne kao komentator.

Cognitive Verifier Checkpoint

Pitanja za provjeru vlastitog prompta:

1. Da li sam tražio od modela da objasni proces, a ne samo rezultat?
2. Da li postoji jasan redoslijed misaonih koraka?
3. Da li model sam provjerava konzistentnost zaključka?
4. Da li je moj prompt formulisan tako da podstiče razmišljanje, a ne samo opisivanje?

Ako tri od četiri odgovora nisu “da” → prompt ponovo napiši.

Sažetak

Chain-of-Thought uči AI da ne daje samo odgovore, nego **obrazloženja koja vode do odgovora**. To ga čini idealnim alatom za kompleksne probleme, strategije i tehničke odluke.

Razlaganje zadatka: od nejasnog zahtjeva do koraka

1. Uvod u Task Decomposition

Task Decomposition je tehnika kojom se **kompleksan, nejasan ili višeslojan zadatak** razlaže na niz manjih, jasno definisanih koraka koje AI model može razumjeti, izvršiti i kontrolisati.

AI modeli funkcionišu znatno bolje kada:

- znaju tačno *šta je sljedeći korak*,
- razumiju *zašto taj korak postoji*,
- i imaju jasan redoslijed izvršavanja.

Zbog toga Task Decomposition nije samo "tehnika promptinga", nego **temeljni princip rada svih naprednih AI sistema**.

2. Zašto je Task Decomposition ključan?

Kada se zadatak ne razloži:

- model preskače korake,
- pravi logičke rupe,
- daje djelimične ili nepovezane odgovore,
- ne razumije stvarni cilj,
- output je teško kontrolisati i provjeriti.

Razlaganjem zadatka dobijaš:

- **preciznost** – svaki korak ima svrhu,
- **dosljednost** – isti proces daje sličan rezultat,
- **upravljivost** – možeš intervenisati po fazama,
- **provjerljivost** – svaka faza se može validirati,
- **mogućnost optimizacije** – slabe tačke su vidljive.

Task Decomposition pretvara AI iz improvizatora u **procesnog izvršioca**.

3. Standardni prompt šablon za Task Decomposition

Da bi model pravilno razložio zadatak, moraš mu **eksplicitno tražiti strukturu**, ne samo rezultat.

> Razloži sljedeći zadatak na jasno definisane faze.
>
> 1. Identifikuj glavni cilj zadatka.
> 2. Definiši sve potciljeve ili podsisteme potrebne za postizanje cilja.
> 3. Pretvori potciljeve u konkretne korake.
> 4. Za svaki korak opiši: ulazne podatke, radnju, očekivani izlaz.
> 5. Organizuj korake u logički redoslijed.
> 6. Identifikuj tačke rizika ili potencijalne greške.
> 7. Predloži optimizovanu verziju procesa.

4. Interna logika procesa

a) Makro-razumijevanje

Model prvo mora razumjeti **šta je stvarni cilj**, a ne samo površinski zahtjev.

Korisnici često opisuju simptom ("napravi izvještaj"), a ne cilj ("donijeti odluku na osnovu trenda"). Ako se ova faza preskoči, cijeli proces ide u pogrešnom smjeru.

b) Mikro-razlaganje

Zatim se kompleksan zadatak rastavlja na **male, izvršive jedinice**.

Primjeri:

- "Analiziraj podatke" → učitaj fajl → očisti podatke → izračunaj metrike → napravi grafike → napiši zaključke
- "Napravi automatizaciju" → definiši input → dizajniraj tok → odaberi trigger → definiši output → testiraj

c) Linearizacija

Na kraju se svi koraci poredaju u **redoslijed kojim ih AI može izvesti**.

Ova faza sprječava preskakanje, kružnu logiku i kontradikcije između faza.

5. Primjena u raznim kontekstima

AI OS – modularni sistemi

Task Decomposition omogućava jasno definisane inpute, outpute i granice odgovornosti svakog modula.

RAG sistemi

RAG proces se prirodno razlaže na: ingest, embedding, indeksiranje, razumijevanje upita, evaluaciju odgovora.

n8n automatizacije

Svaki n8n workflow je zapravo **vizualni oblik Task Decomposition-a**. Node = jedan korak. Flow = linearizovani proces.

Prompt Engineering

Svi napredni promptovi (analitika, agenti, višefazni zadaci) implicitno koriste Task Decomposition.

6. Primjer Task Decomposition u praksi

Zadatak: "Napravi izvještaj iz Excel fajla sa analizom trendova."

Razlaganje:

1. **Učitaj fajl** — provjeri dostupne sheetove, identifikuj numeričke kolone
2. **Očisti podatke** — ukloni duplikate, standardizuj datume, ispravi tipove podataka
3. **Analiziraj trendove** — izračunaj moving average, identifikuj outliere
4. **Vizualizuj podatke** — napravi line chart, napravi histogram
5. **Generiši zaključke** — sažmi 3–5 ključnih opažanja
6. **Izvoz** — PDF izvještaj, CSV dataset

7. Sažetak lekcije

Task Decomposition Pattern omogućava da se kompleksni zadaci pretvore u niz preciznih, jasno definisanih faza.

Ovaj obrazac:

- povećava kvalitet outputa,
- stabilizuje ponašanje modela,
- omogućava profesionalne AI workflowe.

Task Decomposition je **temelj svih ozbiljnih AI projekata**.

Write → Reflect → Improve: mini-proces urednika

1. Uvod u Write → Reflect → Improve (WRI) obrazac

Write → Reflect → Improve je višefazni obrazac koji uvodi **svjesnu samoprovjeru** u proces generisanja AI odgovora.

Umjesto da model isporuči jedan odgovor — kakav god da ispadne — WRI ga vodi kroz tri jasno razdvojene faze:

1. **Write** – inicijalno generisanje odgovora
2. **Reflect** – kritička analiza vlastitog outputa
3. **Improve** – integrisana, poboljšana verzija

Ovaj obrazac mijenja osnovnu dinamiku rada s AI-jem. Model više ne djeluje kao "generator teksta koji pogađa odgovor", nego kao autor koji piše nacrt, zatim ga uređuje i poboljšava.

2. Zašto je ovaj pattern važan?

AI modeli ne griješe zato što su "glupi", nego zato što:

- preskoče kontekst,
- ne primijete kontradikcije,
- pogrešno interpretiraju upit,
- daju generičke odgovore,
- ne provjere da li su ispunili kriterije zadatka.

WRI uvodi upravo to pitanje — model mora provjeriti da li je njegov odgovor dobar.

Korištenjem ovog obrasca značajno se povećava:

- **preciznost** (manje pogrešnih tvrdnji),
- **pouzdanost** (manje halucinacija),
- **jasnoća** (bolja struktura),
- **stručnost** (zreliji ton i argumentacija).

3. Standardni prompt šablon za WRI

Korak 1 – Write

> Generiši inicijalni odgovor na zadatak. Ne pokušavaj da ga unaprijed optimizuješ.

Korak 2 – Reflect

> Kritički procijeni svoj prethodni odgovor: 1. Identifikuj logičke greške 2. Uoči preskočene informacije 3. Označi nejasne ili površne dijelove 4. Provjeri da li su ispunjeni svi kriteriji zadatka 5. Navedi konkretna poboljšanja

Korak 3 – Improve

> Na osnovu refleksije, generiši novu verziju odgovora koja: - ispravlja uočene probleme, - ima jasniju strukturu, - uklanja suvišne dijelove, - preciznije odgovara na zadatak.

4. Interna logika WRI procesa

a) Write – Prvi nacrt

U ovoj fazi model slobodno generiše odgovor. Ovo je sirova verzija, nacrt bez samocenzure, polazna tačka — ne konačno rješenje. Greške u ovoj fazi nisu problem — one su očekivane.

b) Reflect – Kritička samoevaluacija

Ovdje model ulazi u meta-poziciju. On više **ne piše**, nego:

- čita vlastiti tekst,
- analizira strukturu,
- prepoznaje slabosti,
- poredi odgovor s ciljem zadatka.

Ova faza je ključna jer model po prvi put "razmišlja o onome što je rekao".

c) Improve – Integrisana optimizacija

Na osnovu refleksije, model kreira novu verziju koja:

- ispravlja greške,
- pojašnjava ključne dijelove,
- dodaje preciznost,
- uklanja nejasnoće,
- zaokružuje odgovor u profesionalni standard.

Ovo nije kozmetičko uređivanje — ovo je **kvalitativni skok**.

5. Primjer WRI u praksi

Zadatak: "Napiši sažetak o važnosti kontrole LDL vrijednosti."

Write – inicijalni odgovor:

> "LDL je bitan za zdravlje srca i treba ga držati niskim. Visok LDL može uzrokovati probleme i važno je redovno ga pratiti."

Reflect – kritička analiza:

Model uočava: preopćenitost, manjak stručnog konteksta, izostanak mehanizma rizika, nejasne preporuke.

Improve – poboljšana verzija:

> "LDL-holesterol je ključni biomarker kardiovaskularnog rizika. Povišene vrijednosti ubrzavaju aterosklerotski proces i povećavaju vjerovatnoću infarkta miokarda i moždanog udara. Redovno praćenje LDL-a i pravovremena terapijska intervencija omogućavaju značajno smanjenje ukupnog kardiovaskularnog rizika."

Razlika u kvalitetu je očigledna.

6. Sažetak lekcije

Write → Reflect → Improve Pattern uvodi **samokorekciju** u rad AI modela.

Ovaj obrazac:

- smanjuje greške,

- povećava tačnost,
- podiže profesionalni nivo outputa.

WRI nije dodatak promptingu — on je **standard za ozbiljan rad s AI-jem**.

ReAct — sinergija razmišljanja i djelovanja

Uvod

Chain-of-Thought uči AI da razmišlja. ReAct ide korak dalje — uči ga da **razmišlja i djeluje**.

ReAct (Reason + Act) je paradigma u kojoj model ne ostaje zatvoren u vlastitom rezonovanju, nego koristi spoljašnje izvore da provjeri i nadogradi svoje zaključke.

Ukratko: **Think → Act → Observe → Repeat.**

Kako ReAct funkcioniše

ReAct se odvija kroz petlju:

1. **Thought** — model izražava misao ili hipotezu
2. **Action** — izvršava vanjsku radnju (pretraga, API, alat)
3. **Observation** — dobija rezultat akcije
4. **Loop** — koristi rezultat da formira novu misao

Ova povratna petlja smanjuje halucinacije i povećava tačnost.

Model ne ostaje "zatvoren u glavi" (kao u Chain-of-Thought), nego koristi vanjski svijet (API-je, baze, okruženja) da provjeri i nadogradi svoje zaključke.

Konkretni primjer toka

1. **Thought** → model izražava unutrašnje rezonovanje: "Moram pronaći hotel gdje je održan show Mystère."
2. **Action** → izvršava vanjsku radnju: "Search[Cirque du Soleil Mystère]"
3. **Observation** → dobija rezultat akcije: "Mystère is held at Treasure Island Hotel and Casino."
4. **Loop** → koristi posmatranje da generiše sljedeću misao: "Znači, trebam potražiti broj soba u tom hotelu."

Ova petlja misli i akcija stvara dinamičan tok učenja i odlučivanja.

Ključne prednosti

Aspekt	Klasični CoT	ReAct
Razmišljanje	Samo unutar modela	Kombinacija unutrašnjeg i spoljašnjeg
Ažurnost informacija	Ograničena na trening podatke	Može pretraživati real-time izvore
Halucinacije	Česte	Znatno smanjene
Transparentnost	Ograničena	Svaka odluka ima trag
Primjena	Tekstualni zadaci	Q&A, igre, web-navigacija, agenti

Zašto je ReAct važan

U poređenju sa klasičnim CoT-om:

- CoT razmišlja samo unutar modela
- ReAct kombinuje unutrašnje razmišljanje i spoljašnje provjere
- svaka odluka ima trag (misao → akcija → rezultat)

Zbog toga je ReAct osnova za:

- AI agente
- web-navigaciju
- alate koji komuniciraju s realnim sistemima

Istraživanja (HotpotQA, WebShop, ALFWorld) potvrđuju veću tačnost i manju sklonost halucinacijama.

Sažetak

ReAct omogućava LLM-u da **misli kao čovjek i uči kroz djelovanje**. To ga pretvara iz pasivnog generatora teksta u aktivnog, provjerljivog agenta.

Game-play: misaona igra za dolaženje do cilja

Uvod

Kada AI dobije složen zadatak, najveći problem nije nedostatak znanja — nego gubitak fokusa. Model često krene dobro, ali se putem razvuče, skrene s cilja ili završi u opštem objašnjavanju bez jasnog ishoda.

Game Play Pattern postoji da bi se to spriječilo.

Ovaj pattern tretira zadatak kao **misaonu igru** sa jasnim ciljem, pravilima i fazama. Model se ne ponaša kao generator teksta, nego kao igrač koji zna:

- šta pokušava postići
- po kojim pravilima igra
- kada je "pobjeda" ostvarena

Kako Game Play Pattern funkcioniše

Game Play Pattern se sastoji od nekoliko međusobno povezanih elemenata:

Cilj (Goal)

Na početku se jasno definiše šta znači uspješan ishod. Bez toga, model nema mjerilo kvaliteta.

Primjer cilja: objasniti pojam tako da ga razumije i srednjoškolac.

Pravila (Rules)

Ovdje postavljaš ograničenja ponašanja: ton, stil, zabrane i prioritete.

Primjeri: - ne koristi žargon - izbjegavaj duge rečenice - budi konkretan i jasan

Faze (Phases)

Razmišljanje se dijeli u korake kako bi se izbjeglo preskakanje logike.

Tipičan tok: - Faza 1: shvati problem - Faza 2: razradi mogućnosti - Faza 3: zaključi i sažmi

Strategija (Strategy)

Definiše "stil igre" — kako model pristupa zadatku.

Primjeri strategija: - analitička (razlaže i povezuje) - eksperimentalna (testira više rješenja) - reflektivna (povezuje s iskustvom i analogijama)

Povratna petlja (Feedback Loop)

Na kraju svake runde model provjerava: - da li je cilj postignut - da li je odgovor koristan i tačan - da li treba novi pokušaj

Primjer u praksi

Zamisli da želiš da AI objasni **Chain-of-Thought** početniku, ali da to ne bude suho objašnjenje.

Umjesto klasičnog prompta, koristiš Game Play Pattern:

> Play the "Game of Understanding."
>
> **Goal:** objasniti Chain-of-Thought kao jednostavnu edukativnu igru u tri runde.
>
> **Rules:** koristi jednostavan jezik, svaka runda gradi prethodnu, završi pitanjem.

Phases: setup → play → win.

Style: smiren, prijateljski, mentorski.

Rezultat je objašnjenje koje: - ima jasan tok - zadržava pažnju - vodi čitaoca do “aha” momenta

AI ne improvizuje — on **igra igru do kraja**.

Primjer prompta

```
Play the "Game of Understanding."
Your goal: Explain the concept of "Chain of Thought" to a beginner as if you're
round game.

Rules:
- Use plain language, no technical jargon.
- Each round must build on the previous one.
- End each round with a short question that makes the reader think.

Phases:
1. Setup – Introduce the idea.
2. Play – Show how it works step by step.
3. Win – Give a clear summary and "aha" moment.

Style:
Calm, friendly, and logical — like a mentor helping someone learn through curi
```

Sažetak

Game Play Pattern pomaže AI-ju da razmišlja sa svrhom. Umjesto slobodnog lutanja kroz tekst, model ima cilj, pravila i strategiju. To ga čini posebno korisnim za edukaciju, objašnjavanje i vođenje korisnika kroz složene ideje.

Kad model prestaje pričati i počinje raditi

Uvod

Većina ljudi koristi AI kao sagovornika. Postavi pitanje, dobije odgovor, eventualno tekst ili ideju — i tu se sve završava.

Code Interpreter mijenja tu dinamiku.

Ovdje AI više nije samo model koji generiše riječi. On postaje **radno okruženje**: može računati, analizirati podatke, obrađivati fajlove, crtati grafikone i provjeravati rezultate.

Ako je "obični" AI mozak koji razmišlja — Code Interpreter je **mozak + ruke**.

Šta je Code Interpreter (i šta zapravo radi)

Code Interpreter je integrisano okruženje unutar ChatGPT-a koje omogućava modelu da:

- izvršava Python kod,
- učitava i obrađuje fajlove (Excel, CSV, PDF, slike…),
- kreira vizualizacije (grafikone, tabele),
- generiše i obrađuje slike,
- izvršava kompleksne proračune,
- kreira ZIP pakete, datasetove, modele i mini-automacije.

Ključna razlika u odnosu na "obični" chat:

> AI više ne nagađa rezultate — **on ih izračunava i provjerava.**

Zato odgovori postaju: - tačniji - provjerljivi - ponovljivi

Da li ga treba "aktivirati"

U novom ChatGPT okruženju, Code Interpreter (često označen kao *Advanced Data Analysis*) **ne moraš ručno aktivirati**. Model sam prepoznaje kada je potrebno računanje, analiza podataka ili rad s fajlovima i automatski ulazi u taj režim.

Međutim — ti ga možeš eksplicitno pozvati:

> "Use Code Interpreter to analyze the attached Excel file and calculate the trends."

Time: - uklanjaš dvosmislenost - sprječavaš "tekstualna nagađanja" - prisiljavaš model da računa, a ne da priča

Zašto je Code Interpreter presudan za ozbiljan rad

Bez Code Interpretera, AI: - objašnjava kako bi nešto uradio - daje procjene - ponekad griješi u brojkama

Sa Code Interpreterom, AI: - računa stvarne vrijednosti - vidi podatke - provjerava vlastite pretpostavke

AI postaje: **analitičar, ne komentator**

Primjeri napredne upotrebe

Analiza Excel i CSV fajlova

```
Use Code Interpreter.
Load the attached Excel file.
Analyze monthly trends and highlight anomalies.
Generate a clear summary and one chart.
```

Analiza slika i vizualnih podataka

Code Interpreter može analizirati dimenzije slike, porediti dvije slike, kvantifikovati promjene. Primjena: poređenje dizajna, vizualna kontrola kvaliteta, tehnička analiza.

Provjera logike i izračuna

Ako AI treba da uporedi scenarije, simulira ishode ili provjeri matematičku logiku, Code Interpreter uklanja grešku ljudske procjene.

Kada koristiti Code Interpreter

Koristi ga kad:

- radiš sa podacima,
- radiš vizualizaciju,
- praviš reports,
- analiziraš Excel/CSV,
- obrađuješ slike,
- kreiraš automatizacije.

Kada ne koristiti Code Interpreter

- Za pisanje dugih esejskih odgovora.
- Za razgovore o strategiji i planiranju.
- Za medicinske preporuke (statistika nije isto što i klinička odluka).
- Za situacije gdje samo želiš prirodan tekst.

Code Interpreter kao dio pattern sistema

Najjači kada je dio većeg misaonog sistema:

- **Recipe Pattern + Code Interpreter** → prvo proces, zatim računanje
- **Menu Action Pattern + Code Interpreter** → AI bira da li treba računati ili pitati
- **CVP + Code Interpreter** → provjera zaključaka kroz stvarne izračune

Tada AI ne koristi kod da "impresionira", nego da **potvrdi razmišljanje**.

Kratka analogija

Zamisli konsultanta koji: - govori pametno - ali nikad ne pogleda podatke

To je AI bez Code Interpretera.

Sada zamisli istog konsultanta koji: - otvori Excel - provjeri brojke - pokaže grafikon

To je AI sa Code Interpreterom. Razlika je — **povjerenje**.

Sažetak

Code Interpreter pretvara AI iz govornika u izvršioca. Umjesto objašnjenja, dobijaš izračune. Umjesto pretpostavki, dobijaš podatke. Ako želiš da AI bude alat za ozbiljan rad, a ne samo razgovor — Code Interpreter je ključna komponenta tvog sistema.

Prompt pack: 20 gotovih promptova za code interpreter

1. Excel – Osnovna analiza

"Ucitaj ovaj Excel fajl, analiziraj osnovne statistike (mean, median, std), identifikuj duplikate i napravi kratak rezime ključnih opažanja."

---

2. Excel – Napredna analiza + grafovi

"Ucitaj Excel i napravi tri grafika: line chart po mjesecima, histogram distribucije i scatter plot za korelaciju. Objasni šta grafici pokazuju."

---

3. Excel – Čišćenje podataka

"Ucitaj ovaj Excel. Očisti dataset (duplikati, prazne ćelije, outlieri), napravi novu verziju fajla i daj mi CSV za download."

---

4. Excel – Predikcija trenda

"Analiziraj ove podatke i napravi predikciju trenda za narednih 6 mjeseci koristeći jednostavnu regresiju. Nacrtaj graf predikcije."

---

5. CSV → analiza + export

"Ucitaj ovaj CSV, analiziraj sve kolone, napravi summary tabelu i izvezi optimizovanu verziju kao novi CSV fajl."

---

6. Kreiranje nove tabele od nule

"Napravi tabelu sa 50 primjera korisničkih pitanja i kategorizuj ih u 4 grupe. Izvezi kao CSV."

---

7. Kombinovanje više sheetova

"Ucitaj ova dva Excel fajla i spoji ih na osnovu ključne kolone. Ukloni nekonzistentnosti i generiši jednu finalnu tabelu."

---

8. Slike → analiza

"Ucitaj ovu sliku i analiziraj sve vizualne elemente: kontrast, svjetlo, kompoziciju, orijentaciju, detalje koji mogu biti bitni."

---

9. Slike → kreiranje više verzija

"Napravi 5 različitih verzija slike: 1:1, 4:5, 16:9, 1080x1080, 1920x1080. Sačuvaj sve u ZIP fajl i kreiraj CSV katalog."

---

10. Slike → tehničke korekcije

"Ispravi svjetlo, kontrast i balans boja na slici. Napravi tri poboljšane verzije i objasni šta si tačno korigovao."

---

11. Slike → OCR izvlačenje teksta

"Izdvoji sav tekst sa ove slike i prebaci ga u organizovan format (JSON ili tabela)."

---

12. PDF → sažetak

"Ucitaj PDF dokument, izvuci glavni tekst, organizuj ga u sekcije i napravi stručni sažetak do 250 riječi."

---

13. PDF → ekstrakcija tabela

"Izvuci sve tabele iz PDF-a, pretvori ih u Excel i generiši ZIP fajl sa svim tabelama."

---

14. Napredna statistika (korelacija)

"Ucitaj dataset i izračunaj korelaciju između svih numeričkih kolona. Prikaži heatmap i objasni šta znači svaki ključni odnos."

---

15. Modeliranje (klaster analiza)

"Na osnovu ovog dataset-a napravi K-means clustering sa 3 klastera. Prikaži graf i objasni karakteristike svakog klastera."

---

16. Generisanje vizualnih reportova

"Analiziraj podatke i generiši PDF izvještaj koji uključuje grafike, tabele i zaključke. Napravi da izgleda profesionalno."

---

17. Generisanje ZIP fajla sa outputima

"Napravi 10 različitih verzija slike, kreiraj CSV katalog i spakuj sve u ZIP fajl koji mogu preuzeti."

---

18. Python mini-algoritam

"Napravi Python funkciju koja prima tekst i generiše JSON sa poljima: tema, ton, sentiment, risk_level. Sačuvaj .py fajl."

---

19. Dataset simulacija

"Napravi sintetički dataset sa 1000 redova koji simulira poslovne podatke: kolone 'mjesec', 'prihod', 'trošak', 'profit', 'delta'. Izvezi kao CSV."

---

20. RAG / AI OS analiza embeddinga

"Ucitaj embedding CSV, izračunaj prosječnu distancu između vektora, pronađi anomalije i napravi graf distribucije."

Od promptinga ka agentima

Prelaz: od prompta do agenta

Do ovog trenutka u knjizi učio si **kako natjerati AI da misli bolje**.

Učio si kako da:

- razbiješ zadatak (Task Decomposition),
- strukturiraš podatke (Structured Extraction),
- kontrolišeš format i kvalitet (WRI, Self-Correction Loop),
- provjeriš logiku i greške prije finalnog odgovora.

Sve ove tehnike imaju jednu zajedničku stvar: fokusirane su na **jedan odgovor u jednom trenutku**.

To je moćno — ali ima granicu.

Kada prompt više nije dovoljan

U stvarnim projektima, AI rijetko radi samo jednu stvar:

- ne piše samo tekst,
- ne analizira samo jedan dataset,
- ne daje samo jedan savjet.

Umjesto toga, AI mora:

- **donositi odluke u nizu**,
- **pamtiti prethodne korake**,
- **poštovati ograničenja**,
- **koristiti alate**,
- **ispravljati se u hodu**,
- **ponavljati procese dok ne dođe do cilja**.

Tu prompt prestaje biti dovoljan. Tu počinje **ponašanje sistema**.

Šta zapravo znači “agent”

Agent nije “pametniji prompt”. Agent nije “chatbot sa više teksta”.

Agent je **kontrolisani obrazac ponašanja**.

Jednostavno rečeno:

> Prompt govori šta da kaže. Agent definiše kako da se ponaša.

Agent kombinuje više elemenata:

- razmišljanje (reasoning),
- akcije (tool usage),
- povratne petlje (loops),
- ograničenja (constraints),
- uloge (roles),
- memoriju (state).

Sve to zajedno čini sistem koji **ne reaguje samo**, nego **djeluje namjenski**.

Zašto je ovo prijelomna tačka

Do sada si gradio **kvalitet jednog odgovora**. Odavde počinješ graditi **stabilnost kroz vrijeme**.

Razlika je ključna:

- Prompting → optimizuje *jedan output*
- Agent patterns → dizajniraju *ponašanje kroz više koraka*

Ovo je trenutak kada AI prestaje biti alat za pisanje i postaje **komponenta sistema**.

Šta slijedi

U narednim lekcijama ulazimo u obrasce koji:

- nameću granice modelu,
- definišu uloge i odgovornosti,
- povezuju više promptova u jedan tok,
- omogućavaju razmišljanje + djelovanje (ReAct),
- uvode memoriju, alate i evaluaciju.

Ovdje više ne pitaš: “Kako da dobijem bolji odgovor?”

Ovdje pitaš: “Kako da AI radi pouzdano, ponovljivo i bez mog stalnog nadzora?”

Mentalni model za nastavak

> Agent = Prompt + Pravila + Petlje + Akcije + Kontrola

Ako ovo razumiješ — sve što slijedi ima smisla.

Ograničenja: kako dobiti stabilan output

1. Uvod u Constraint-Based Prompting

Constraint-Based Prompting je tehnika u kojoj AI dobija **jasno definisana ograničenja** koja mora poštovati tokom generisanja odgovora.

Ta ograničenja mogu se odnositi na: format, stil, dužinu, ton, broj stavki, jezik, terminologiju, raspon vrijednosti, logičke zabrane (šta model *ne smije* uraditi), obavezne elemente (šta *mora* uključiti).

Ovim pristupom AI prestaje biti "kreativni generator" i postaje **precizno podešen alat** koji radi unutar jasno zadanih granica.

Ne pitaš više *šta bi AI mogao reći*, nego **šta mu je dozvoljeno da kaže — i kako**.

2. Zašto je ova tehnika ključna

Bez eksplicitnih ograničenja, AI modeli često:

- proizvode preduge odgovore,
- koriste neadekvatan ton,
- dodaju nepotrebne informacije,
- improvizuju podatke,
- preskaču ključne elemente,
- krše format (posebno JSON).

Constraint-Based Prompting omogućava:

- potpunu kontrolu outputa,
- dosljednost kroz više poziva,
- stabilnost u automatizacijama (n8n, API),
- tehnički ispravne odgovore,
- profesionalan i predvidiv stil.

Zbog toga je ovaj pattern **obavezan** u ozbiljnim enterprise-level sistemima.

3. Standardni Constraint-Based Template

Izvrši zadatak **strogo po sljedećim ograničenjima**.

FORMAT OGRANIČENJA - Output mora biti u formatu: *(JSON / tabela / bullet list / sekcije — navesti tačno)* - Ne dodavati tekst van traženog formata - Koristiti isključivo navedene ključeve

STILSKA OGRANIČENJA - Ton: *(stručan / formalan / tehnički / neutralan)* - Rečenice kratke i jasne - Bez metafora i kreativnih elemenata *(osim ako je eksplicitno dozvoljeno)*

SADRŽAJNA OGRANIČENJA - Maksimalno X riječi - Tačno Y stavki - Obavezno uključiti: *(navesti)* - Strogo izbjegavati: *(navesti)*

LOGIČKA OGRANIČENJA - Ne izmišljati podatke - Ako informacija nije poznata → eksplicitno navesti - Poštovati numeričke i semantičke granice

4. Kako model "razmišlja" pod ograničenjima

Constraint-Based Prompting funkcioniše kroz tri osnovna mehanizma:

a) Channeling — Model razmišlja unutar dozvoljenog kognitivnog prostora.

b) Filtering — Sve što ne zadovoljava uslove biva eliminisano prije outputa.

c) Structural anchoring — Format postaje obavezujući zakon, naročito kod JSON i API izlaza.

Rezultat je odgovor koji je **stabilan, tehnički validan i predvidiv**.

5. Praktičan primjer

Zadatak: "Napiši sažetak istraživanja o dijabetesu."

Ograničenja: maksimalno 80 riječi, striktno profesionalan ton, obavezno uključiti: prevalencu, komplikacije, značaj kontrole, izbjegavati brojčane vrijednosti bez izvora, format: bullet list.

Rezultat:

- Dijabetes je hronično metaboličko stanje sa rastućom globalnom prevalencom.
- Glavne komplikacije uključuju vaskularna, bubrežna i neurološka oštećenja.
- Kontrola glikemije značajno smanjuje dugoročne rizike.
- Multidisciplinarni pristup poboljšava ishode liječenja i kvalitet života.

Sav format, ton i sadržaj su ispoštovani bez odstupanja.

6. Sažetak lekcije

Constraint-Based Prompting omogućava da AI radi **unutar jasno definisanih granica**, bez improvizacije i grešaka.

Ovaj pattern je temelj:

- stabilnih AI sistema,
- tehnički sigurnih automatizacija,
- profesionalnih tekstova,
- pouzdanih AI agenata.

Bez ograničenja — AI je nepredvidiv. Sa ograničenjima — **postaje alat pod punom kontrolom**.

Uloge: kad želiš stručnjaka po narudžbi

1. Uvod u Role-Based Prompting

Role-Based Prompting je tehnika u kojoj AI-u eksplicitno dodjeljuješ **ulogu**, čime direktno utičeš na način njegovog razmišljanja, nivo stručnosti i kvalitet odgovora.

Uloga može biti:

- profesionalna (*doktor, programer, analitičar, fitness trener*),
- akademska (*profesor, istraživač, urednik*),
- tehnička (*n8n inženjer, AI agent, DevOps stručnjak*),
- stilistička (*copywriter, storyteller, marketinški strateg*),
- visoko specijalizovana (*kardiolog, lipidolog, AI OS arhitekt*).

Dodjeljivanjem uloge ne mijenjaš samo **stil odgovora**, već cijelu *kognitivnu mapu* po kojoj model rezonuje.

AI tada ne odgovara "kao model", nego kao **profil znanja koji si definisao**.

2. Zašto je ova tehnika presudna

Kada ne definišeš ulogu, AI najčešće odgovara:

- preširoko,
- preopćenito,
- bez jasne dubine,
- bez konzistentne perspektive.

Kada definišeš ulogu:

- odgovori postaju fokusirani,
- terminologija se automatski prilagođava,
- logika postaje stručna,
- kontekst se bolje razumije,
- kvalitet skače višestruko.

Role-Based Prompting daje AI-ju **identitet**, a identitet vodi ponašanje.

3. Standardni Role-Based Prompt Template

```
Preuzmi ulogu [navedi tačnu ulogu]
(npr. senior data analyst, medicinski endokrinolog, n8n automation architect)
i odgovaraj isključivo u skladu s tom ulogom.

Zadatak:
- [navedi konkretan zadatak]

Ograničenja:
- ton (stručan, formalan, tehnički),
- stil (kratko, jasno, bez ukrasa),
- format (JSON, tabela, sekcije),
- fokus na tačnost,
- bez improvizacije ili nagađanja.
```

4. Kako Role-Based Prompting djeluje

Kada model dobije ulogu, dešavaju se četiri stvari:

a) Aktivacija relevantnog znanja — Model koristi terminologiju, strukturu i prioritete tipične za tu profesiju.

b) Usmjeravanje pažnje — Fokusira se na ono što je za tog stručnjaka važno, a ignoriše nebitno.

c) Veća dubina objašnjenja — Eksperti ne objašnjavaju kao početnici — već precizno i kontekstualno.

d) Smanjenje halucinacija — Uloga djeluje kao prirodno ograničenje koje sužava prostor za improvizaciju.

5. Primjena u realnim sistemima

- **AI OS** — Svaki modul ima svoju ulogu (zdravlje, fokus, strategija, automatizacija).
- **RAG sistemi** — "Researcher mode", "citation mode", "analyst mode".
- **Pisanje knjiga** — Uloga urednika ili recenzenta drastično poboljšava kvalitet.
- **n8n workflowi** — Precizna uloga sprječava pogrešne instrukcije.
- **Medicina i nauka** — Uloga stručnjaka smanjuje greške i povećava pouzdanost.

6. Praktičan primjer

Bez uloge: Površno, općenito, neupotrebljivo.

Sa ulogom:

Prompt: "Preuzmi ulogu kardiologa. Objasni šta je LDL-holesterol. Koristi stručan, ali jasan ton. Bez metafora."

Output: "LDL-holesterol predstavlja lipoproteinsku česticu niske gustoće odgovornu za transport holesterola iz jetre ka perifernim tkivima. Povišene koncentracije LDL-a ubrzavaju aterosklerotski proces, naročito kod prisutne oksidacije LDL čestica. Optimalno upravljanje LDL-em ključno je za smanjenje ukupnog kardiovaskularnog rizika."

7. Sažetak lekcije

Role-Based Prompting dodjeljuje AI-ju **identitet** kako bi odgovarao kao stvarni stručnjak.

Ovaj pattern omogućava:

- konzistentnost,
- stručnu dubinu,
- preciznost,
- realističan profesionalni nivo,
- znatno manju improvizaciju.

To je jedan od **najvažnijih alata naprednog rada sa AI-jem** i temelj svakog ozbiljnog sistema.

Persona, uloga i mode: tri različite poluge

Zašto nisu isto — i zašto većina ljudi to pogrešno miješa

1. Zašto je ova lekcija uopšte potrebna

U radu s AI-jem često ćeš vidjeti da ljudi koriste pojmove *persona*, *role* i *mode* kao da znače isto.

U praksi to dovodi do:

- konfuznih promptova,
- nekonzistentnih odgovora,
- loše kontrolisanih AI sistema.

Problem nije u AI-ju. Problem je u **miješanju slojeva ponašanja**.

2. Brza orijentacija (jedna rečenica po pojmu)

- **Persona** → *kako AI zvuči*
- **Role** → *šta AI jeste (stručno)*
- **Mode** → *kako se AI ponaša u tom trenutku*

3. Persona — stil, ton i "ličnost"

Persona definiše **način izražavanja**, ne znanje.

Ona utiče na: ton (smiren, oštar, prijateljski), stil (mentorski, akademski, sarkastičan), emocionalnu nijansu, način obraćanja čitaocu.

Persona **ne garantuje stručnost**. Možeš imati savršenu personu — i potpuno pogrešan sadržaj.

Primjeri persona:

- smireni mentor
- skeptični analitičar
- direktni trener
- empatični savjetnik
- neutralni tehnički narator

Persona je ono zbog čega tekst "zvuči ljudski".

4. Role — stručni identitet

Role definiše **kognitivni i profesionalni okvir znanja**.

Kada dodijeliš role, ti govoriš AI-ju: "Razmišljaj i odgovaraj kao ova vrsta stručnjaka."

Role utiče na: terminologiju, dubinu objašnjenja, način zaključivanja, prioritete informacija.

Primjeri role:

- kardiolog
- senior data analyst
- n8n automation architect
- AI systems designer

Bez role, AI je generalista. Sa role-om, AI postaje **specijalista**.

5. Mode — operativno ponašanje

Mode određuje **šta AI trenutno radi**, ne ko je on i kako zvuči.

Mode je *operativni kontekst*. On definiše:

- da li AI analizira, piše, evaluira ili ispravlja,
- da li postavlja pitanja ili daje odgovore,
- da li generiše, validira ili sažima.

Primjeri modova:

- **Analysis mode** – razmišljanje korak po korak
- **Draft mode** – generisanje prve verzije
- **Review mode** – kritička evaluacija
- **Correction mode** – ispravke i optimizacija
- **Agent mode** – donošenje odluka i akcija

Mode je ono što AI *trenutno izvršava*.

6. Zašto miješanje ova tri pojma pravi haos

Ako napišeš: "Budi profesionalni doktor i napiši to prijateljski, analiziraj i ispravi se."

AI dobija **tri konflikta odjednom**:

- persona i role nisu jasno razdvojene,
- mode nije definisan redoslijedom,
- ponašanje nije kontrolisano.

Rezultat: nestabilan output, nekad dobar, nekad loš, nemoguće ponoviti isti kvalitet.

7. Ispravan mentalni model

> **Role** = ko si **Persona** = kako zvučiš **Mode** = šta sada radiš

I uvijek ih postavljaj **tim redoslijedom**.

8. Praktičan primjer

Dobro definisan prompt:

> Preuzmi ulogu kardiologa. Persona: smiren, stručan, jasan — bez metafora. Mode: 1. prvo objasni pojam, 2. zatim provjeri tačnost, 3. onda isporuči finalni odgovor.

Rezultat: stručno ispravan, konzistentan, stabilan, ponovljiv. Ovo je **sistemski prompt**, ne improvizacija.

9. Kako se ovo koristi u ozbiljnim sistemima

- **AI OS** — Role = modul (zdravlje, fokus, strategija), Persona = stil komunikacije, Mode = trenutno ponašanje
- **RAG sistemi** — Role = researcher / analyst, Mode = retrieve → reason → answer, Persona = neutralna
- **AI agenti** — Role = domen ekspert, Mode = loop (think → act → verify), Persona = minimalna

10. Sažetak lekcije

Persona, Role i Mode nisu sinonimi. Oni su **tri različita sloja kontrole**.

- Persona upravlja *stilom*
- Role upravlja *znanjem*
- Mode upravlja *ponašanjem*

Ako ih jasno razdvojiš, AI prestaje biti “chat” i postaje **pouzdan sistem**.

Agent loopovi: Think → Act → Verify

Think → Act → Verify: kako AI prestaje da "odgovara" i počinje da djeluje

1. Zašto uopšte govorimo o agent loop-ovima

Do ove tačke u knjizi naučio si kako da: kontrolišeš šta AI zna (Role), kako zvuči (Persona), šta radi u jednom trenutku (Mode), uvedeš samoprovjeru (WRI, Self-Correction Loop), postaviš jasna ograničenja (Constraints).

Sve to je neophodno. Ali još uvijek postoji jedna ključna razlika između:

- **AI koji odgovara**, i
- **AI koji izvršava zadatak kroz vrijeme**.

Ta razlika se zove **agent loop**.

Agent loop je ono što omogućava AI-ju da: donosi odluke u više koraka, koristi alate, provjerava rezultate, ispravlja se, i nastavlja dok ne dođe do cilja.

Bez loop-a, AI je pametan — ali statičan. Sa loop-om, AI postaje **operativni sistem ponašanja**.

2. Šta je zapravo Agent Loop

Agent Loop je **kontrolisana petlja ponašanja**.

Umjesto jednog odgovora, AI prolazi kroz **ponovljivi ciklus**:

1. **Think** – razmišlja šta treba uraditi
2. **Act** – izvršava akciju (tekst, alat, API, odluka)
3. **Verify** – provjerava rezultat
4. **Decide** – nastavlja ili završava

Ova petlja se ponavlja **dok cilj nije ispunjen** ili dok sistem ne kaže "stani".

Agent loop **nije beskonačno razmišljanje**. On je *namjenski kontrolisan tok*.

3. Think → Act → Verify kao mentalni model

Think (Razmišljanje)

AI analizira: cilj, trenutno stanje, ograničenja, šta zna, šta ne zna.

U ovoj fazi **ne daje konačan odgovor**. Ovdje se dešavaju: Task Decomposition, izbor strategije, odluka o sljedećem koraku.

Act (Akcija)

AI izvršava **jednu konkretnu radnju**: napiše dio teksta, pozove alat, izvrši API request, pretraži RAG, generiše podatke, transformiše output.

Jedna akcija = jedan potez.

Verify (Provjera)

AI provjerava: da li je akcija dala očekivani rezultat, da li postoje greške, da li je format ispravan, da li je cilj bliži nego prije.

Ako nešto nije u redu → **ne ide dalje**, nego koriguje.

Ovo je razlika između "AI koji se nada da je u pravu" i "AI koji provjerava prije nego nastavi".

4. Control Flow — ko upravlja petljom

Control Flow je skup pravila koji odgovara na pitanja:

- Kada AI smije nastaviti?
- Kada mora stati?
- Kada se vraća korak unazad?
- Kada eskalira grešku?
- Kada završava proces?

Bez control flow-a: AI luta, ponavlja se, halucinira, troši resurse, gubi fokus.

Sa control flow-om: ponašanje je stabilno, output je ponovljiv, sistem je pouzdan.

5. Osnovni Agent Loop Template

```
You are operating as an AI agent.

Follow this control loop strictly:

1. THINK:
   - Analyze the current goal.
   - Decide the next best action.
   - Do NOT produce a final answer yet.

2. ACT:
   - Perform exactly one action.
   - Clearly label the action and its output.

3. VERIFY:
   - Check if the action result is correct and useful.
   - If errors exist, identify them.

4. DECIDE:
   - If the goal is not yet achieved, repeat the loop.
   - If the goal is achieved, produce the final output.

Rules:
- Do not skip steps.
- Do not combine steps.
- Stop immediately if constraints are violated.
```

6. Primjena u realnim sistemima

a) AI agenti — Agent koji planira, izvršava, provjerava, nastavlja. Bez loop-a → chatbot. Sa loop-om → agent.

b) RAG sistemi — Think → šta tražim? Act → retrieve izvore. Verify → da li su relevantni? Act → generiši odgovor. Verify → da li je u skladu s izvorima?

c) n8n automatizacije — Trigger → akcija → provjera → sljedeći node. Svaki workflow je implicitni agent loop.

d) AI OS — AI OS sa loop-om: prati stanje, reaguje, prilagođava se, uči iz rezultata.

7. Najčešće greške kod agent loop-ova

- × nema jasnog cilja
- × nema stop uslova
- × AI radi više akcija odjednom
- × nema verifikacije
- × nema granica (constraints)

8. Sažetak lekcije

Agent Loops i Control Flow su trenutak kada AI:

- prestaje biti generator teksta,
- prestaje biti alat,
- i postaje **sistem ponašanja**.

Think → Act → Verify nije trik. To je **operativna filozofija**.

Alati: izbor akcije i izvršenje

Kada, zašto i kako agent koristi alate — a kada ne smije

1. Zašto je Tool Use prelomna tačka

Agent bez alata je **ograničen na vlastiti kontekst**. On može razmišljati, kombinovati informacije, provjeravati logiku — ali **ne može proširiti stvarnost**.

Onog trenutka kada agent dobije mogućnost da: pretraži bazu, pozove API, pročita fajl, izvrši izračun, zapiše ili pročita stanje — AI prestaje biti "jezički model" i postaje **operativni agent**.

Tool Use je razlika između *razmišljanja o svijetu* i *interakcije sa svijetom*.

2. Šta zapravo znači "Action Selection"

Action Selection nije tehnički detalj. To je **odlučivanje**.

Agent u svakom loop-u mora odgovoriti na pitanje: "Da li mi je potreban alat — ili mogu nastaviti bez njega?"

Loš agent: koristi alate prerano, koristi ih nepotrebno, koristi pogrešan alat, ne provjerava rezultat.

Dobar agent: koristi alat **samo kad ima razlog**, zna *zašto* ga koristi, zna *šta očekuje* kao rezultat, zna *kada da stane*.

3. Mentalni model: Think → Decide → Act

U kontekstu alata, loop se precizira:

1. **Think** – šta mi trenutno nedostaje?
2. **Decide** – da li mi treba alat ili ne?
3. **Act** – izvrši *jednu* alat-akciju
4. **Verify** – provjeri da li je alat dao koristan rezultat

Ako agent ne zna odgovoriti na **Decide** — nije agent, nego automat bez kontrole.

4. Kada agent NE SMIJE koristiti alat

Agent **ne smije** koristiti alat ako:

- već ima dovoljno informacija,
- zadatak je čisto konceptualan,
- alat ne povećava tačnost,
- alat bi dodao šum ili rizik,
- rezultat se ne može validirati.

Svako korištenje alata u tim situacijama je **signal lošeg dizajna**.

5. Kada agent MORA koristiti alat

Agent **mora** koristiti alat kada:

- radi sa podacima koje ne zna,
- mora provjeriti činjenice,
- mora obraditi fajl ili tabelu,
- mora izvršiti izračun,
- mora dohvatiti stanje sistema,

- mora zapisati rezultat.

Primjeri: RAG pretraga, OCR analiza slike, CSV / Excel analiza, API validacija.

Ako agent ne koristi alat u ovim situacijama — on halucinira.

6. Tool Use kao ugovor, ne kao sloboda

Alat nije "opcija". Alat je **ugovor**.

Kada agent koristi alat, mora znati:

- ime alata,
- ulazne parametre,
- očekivani output,
- format rezultata,
- kriterije uspjeha.

7. Standardni Tool Use Prompt Template

```
You are operating as an AI agent with access to tools.

At each step, follow this logic:

1. THINK:
   - Identify what information or capability is missing.
   - Determine if a tool is required.

2. DECIDE:
   - If no tool is needed, continue reasoning.
   - If a tool is needed, select exactly one appropriate tool.
   - Clearly state why this tool is required.

3. ACT:
   - Execute the selected tool with explicit input parameters.
   - Perform only one tool action.

4. VERIFY:
   - Evaluate the tool output.
   - Confirm whether it satisfies the requirement.
   - If not, adjust strategy or select a different tool.

Rules:
- Never use a tool without justification.
- Never chain tools in one step.
- Always verify tool output before continuing.
```

8. Najčešće greške kod Tool Use-a

- × alat se koristi "za svaki slučaj"
- × više alata u jednoj akciji
- × nema provjere rezultata
- × alat se koristi umjesto razmišljanja
- × agent ne zna kada da stane

9. Sažetak lekcije

Tool Use nije tehnička sitnica. To je **odgovornost agenta**.

Agent koji zna: kada da koristi alat, kada da ne koristi, kako da validira rezultat — je agent koji može raditi **samostalno i pouzdano**.

Bez ovoga, svi agent sistemi ostaju demonstracije. Sa ovim — postaju **stvarni sistemi**.

Stanje i memorija: šta je stvarno zapamćeno

Kada agent mora "pamtiti" — a kada to postaje opasno

1. Zašto je pamćenje najopasniji dio agent sistema

Agent bez memorije: ponavlja iste greške, ne gradi kontinuitet, nema istoriju.

Agent sa loše dizajniranom memorijom: postaje nekonzistentan, kontradiktoran, nekontrolisan.

Zato je **Memory ≠ više konteksta**. Memory je **arhitektonska odluka**, ne feature.

2. Razlika koju moraš imati čistu u glavi

Context

- kratkoročno, privremeno, nestaje nakon sesije
- služi za *trenutno razmišljanje*

State

- trenutno stanje sistema
- varijable, flagovi, modovi
- "gdje se agent sada nalazi"

Memory

- dugoročno, selektivno, namjerno zapisano
- koristi se kasnije

Samo Memory preživljava vrijeme.

3. Kada agent NE SMIJE imati memoriju

Agent **ne smije** pamtiti ako:

- radi jednokratni zadatak,
- nema validaciju informacija,
- nema jasna pravila brisanja,
- radi savjetodavne ili osjetljive stvari.

Primjeri: Q&A bot, medicinski savjeti bez pacijent-ID, brainstorming agent, demo agent.

U tim slučajevima memorija je **rizik**, ne prednost.

4. Kada agent MORA imati memoriju

Memorija je obavezna kada:

- agent radi dugoročno,
- ima iterativne ciljeve,
- mora izbjeći ponavljanje grešaka,
- prati korisničke preferencije,
- gradi sistem (AI OS, agent mreže).

5. Tipovi memorije koje agent može imati

a) Ephemeral Memory — kratkotrajna, briše se brzo, služi za tok zadatka.

b) Working Memory — traje kroz jednu sesiju, drži međurezultate, nije trajna.

c) Long-Term Memory — trajna, selektivna, zapisuje se namjerno, koristi se kasnije.

Profesionalni agent koristi **samo ono što mora**.

6. Najveća greška: automatsko pamćenje

Ako agent pamti **sve što vidi** — to nije memorija, to je šum.

Memorija mora biti: rijetka, sažeta, provjerena, opravdana.

Zlatno pravilo: > Ako ne znaš zašto nešto pamtiš — ne pamti.

7. Standardni Memory Control Template

```
At each step, evaluate memory usage:

1. Identify whether new information is:
   - transient
   - session-based
   - long-term relevant

2. Store information ONLY if:
   - it is reusable,
   - it improves future decisions,
   - it is validated,
   - it has clear scope.

3. Define memory type:
   - ephemeral
   - working
   - long-term

4. Define expiration or persistence rule.

5. Never store raw conversations.
   Store distilled insights only.
```

8. Memory u RAG sistemima

RAG ≠ memorija.

- RAG: dohvaća informacije, ne pamti iskustvo.
- Memorija: pamti odluke, preferencije, evaluacije.

Stabilni sistemi koriste RAG za znanje, Memory za ponašanje. Ako to pomiješaš — dobićeš halucinacije sa samopouzdanjem.

9. Sažetak lekcije

Memory nije luksuz. Ali nije ni default.

Agent koji pamti bez kontrole → degradira. Agent koji ne pamti kada treba → stagnira.

Pravi agent: zna **kada** pamtiti, zna **šta** pamtiti, zna **kada brisati**.

Ovdje se agenti dijele na demonstracije i **sisteme koji žive dugo**.

Planiranje i task grafovi: kad zadatak postane sistem

Kako agent razmišlja unaprijed — bez gubljenja kontrole

1. Zašto planiranje nije isto što i "razmišljanje"

Model može: razmišljati, rezonovati, odgovarati.

Ali **agent mora planirati**.

- razmišljanje = reakcija na upit
- planiranje = konstrukcija puta do cilja

Bez planiranja agent: skače između koraka, zaboravlja cilj, ulazi u petlje, proizvodi fragmentiran output.

Planiranje je ono što **razdvaja chat-model od sistema**.

2. Tri nivoa agent planiranja

1. Goal Definition

- Šta je stvarni cilj?
- Kada je zadatak završen?
- Koji su kriteriji uspjeha?

Bez ovog → sve dalje je improvizacija.

2. Task Decomposition

- Razlaganje cilja u izvršive dijelove
- Identifikacija zavisnosti
- Eliminacija suvišnih koraka

3. Task Graph (ne lista!)

- Koraci nisu linearni
- Neki se izvršavaju paralelno
- Neki čekaju validaciju
- Neki se ponavljaju

Agent ne razmišlja u "listama", već u **grafovima**.

3. Task List vs Task Graph

Task List (×) - korak 1, korak 2, korak 3 - Nema povratnih veza, nema provjera, nema grananja

Task Graph (✓) - čvorovi (tasks), veze (dependencies), grananja, povratne petlje, validacioni čvorovi - Omogućava kontrolu toka, otpornost na greške, skaliranje sistema

4. Standardni Planning & Task Graph Template

```
Step 1: Define Goal
- Final objective:
- Success criteria:

Step 2: Decompose into Tasks
For each task:
- Task name
- Input
- Output
- Dependencies

Step 3: Build Task Graph
- Identify parallel tasks
- Identify validation checkpoints
- Identify feedback loops

Step 4: Execution Rules
- When to stop
- When to retry
- When to escalate error
```

5. Primjer: Agent koji analizira dataset

Goal: "Analizirati dataset i generisati pouzdan izvještaj"

Decomposition: - Load data → Validate schema → Clean data → Analyze metrics → Visualize results → Generate summary → Validate conclusions

Task Graph logika: - *Clean data* ne može prije *Validate schema* - *Visualize* zavisi od *Analyze* - *Validate conclusions* je obavezna prije finalnog outputa - Ako validacija padne → povratak na analizu

6. Zašto planiranje smanjuje halucinacije

Halucinacije često nastaju jer model: ne zna gdje se nalazi u procesu, ne zna šta je već uradio, preskače provjere.

Task Graph: uvodi orijentaciju, uvodi provjere, uvodi granice.

Agent sa planom halucinira **manje**, ne više.

7. Najčešće greške u agent planiranju

- Plan je skriven (implicit)
- Nema validacionih čvorova
- Sve je linearno
- Nema stop-uslova
- Nema rollbacka

Ako plan **ne možeš nacrtati** — ne postoji.

8. Sažetak lekcije

Agent bez planiranja je: reaktivan, nepredvidiv, lomljiv.

Agent sa planiranjem i task grafom: zna gdje ide, zna kada stati, zna kada se vratiti, zna kada je završio.

Ovdje se pravi razlika između "AI koji odgovara" i **AI koji vodi proces**.

Operativna stabilnost i sigurnost

Granice alata: greške, povjerenje i fallback

(Kako agent bira alat — i kada mu NE vjeruje)

1. Zašto alati nisu "pomoć", nego rizik

U naprednim AI sistemima, alati nisu neutralni. Svaki alat: - može pogriješiti, - može vratiti nepotpune podatke, - može zakazati, - može dati lažno samopouzdane rezultate.

Agent koji **slijepo vjeruje alatu** nije inteligentan — on je krhak.

Ova lekcija definiše **kako agent bira alat, kako tretira greške i gdje povlači granicu povjerenja**.

2. Razlika: Model vs Alat

Važno razdvajanje: - **Model** → rezonuje, odlučuje, planira - **Alat** → izvršava ograničenu operaciju

Alat: - nema kontekst, - nema razumijevanje cilja, - nema odgovornost za krajnji ishod.

Odgovornost je uvijek na agentu, nikada na alatu.

3. Kada agent uopšte treba koristiti alat

Agent **ne smije koristiti alat po inerciji**.

Prije svakog poziva mora proći kroz ova pitanja: 1. Da li je zadatak uopšte alat-vrijedan? 2. Da li model već ima dovoljno informacija? 3. Koji alat je *najmanje rizičan* za ovaj korak? 4. Da li je alat deterministički ili heuristički? 5. Da li postoji fallback ako alat zakaže?

Ako nema jasnog odgovora → **nema poziva alata**.

4. Kategorije alata (po nivou povjerenja)

A) Deterministički alati (visoko povjerenje)

Primjeri: - kalkulacije - formatiranje - parsiranje JSON-a - validacija šema

Greške su: - lako uočljive - binarne (radi / ne radi)

→ Visok nivo povjerenja.

B) Polu-deterministički alati (srednje povjerenje)

Primjeri: - OCR - scraping - data extraction - embedding generisanje

Greške: - često tihe - djelimične - zavise od kvaliteta inputa

→ Zahtijevaju **validaciju outputa**.

C) Heuristički / "pametni" alati (nisko povjerenje)

Primjeri: - search - recommendation API - sentiment analysis - AI-powered eksterni servisi

Greške: - suptilne - uvjerljive - teško uočljive bez provjere

→ Nikada se ne koriste bez **verification pass-a**.

5. Tool Trust Boundary (kritični koncept)

Tool Trust Boundary je linija iza koje agent: - prestaje vjerovati alatu, - preuzima kontrolu, - pokreće validaciju ili fallback.

Agent mora eksplicitno znati: - gdje alat *smije odlučivati* - gdje *nikada ne smije*

Bez ove granice: - greške se propagiraju, - halucinacije se maskiraju, - sistem postaje opasan.

6. Standardni Tool Selection & Trust Template

```
Task:
- Description:

Candidate Tools:
- Tool A:
- Purpose:
- Trust Level:
- Known Failure Modes:

Decision:
- Selected Tool:
- Reason:

Trust Boundary:
- What output is trusted:
- What must be validated:
- Fallback if tool fails:
```

Ovo je **minimalni profesionalni standard**.

7. Kako agent tretira greške alata

Agent nikada ne smije: - ignorisati grešku, - pretpostaviti da je rezultat tačan, - "popraviti" rezultat bez priznanja greške.

Ispravan tok: 1. Detekcija greške 2. Klasifikacija greške 3. Odlučivanje: - retry - fallback - eskalacija 4. Dokumentovanje greške (log)

Greška je informacija — ne smetnja.

8. Primjer: OCR alat u medicinskom kontekstu

OCR vrati: - pogrešan broj - pogrešnu decimalu - zamijenjene jedinice

Agent **ne smije**: - direktno koristiti podatak - donositi zaključak

Ispravno ponašanje: - flagovati nesigurnost - tražiti potvrdu - uporediti sa rasponima - označiti podatak kao "unverified"

→ Ovo spašava sistem od kritične greške.

9. Alati u Agent Loop-ovima

U loop-ovima (Think → Act → Verify): - **Think** → bira alat - **Act** → koristi alat - **Verify** → ne vjeruje alatu

Ako Verify preskočiš — **to više nije agent**, nego skripta.

10. Najčešće greške u radu s alatima

- Previše alata za isti task
- Alat se koristi bez opravdanja
- Output se uzima zdravo za gotovo
- Nema fallbacka
- Nema logovanja grešaka
- Agent se "zaljubi" u alat

Profesionalni agent je **skeptičan prema alatima**.

11. Sažetak lekcije

Alati: - ubrzavaju, - ali i kvare.

Agent koji zna: - **kada koristiti alat** - **kada mu vjerovati** - **kada stati**

→ gradi pouzdan sistem.

Ovdje se razdvaja: - "AI koji koristi tool" - od **AI sistema koji kontroliše rizik**.

Kontekst prozor: kapacitet, rizici i strategije

(šta agent pamti — a šta NIKADA ne smije)

1. Zašto je "pamćenje" najčešće pogrešno shvaćen dio AI sistema

Većina korisnika govori o "AI pamćenju" kao o jednoj stvari.

U stvarnosti, agent **nikada nema jedno pamćenje** — već više potpuno različitih mehanizama.

Ako ih ne razdvojiš: - agent halucinira kontinuitet, - kontekst se miješa, - sistem postaje nepouzdan, - greške se akumuliraju bez traga.

Ova lekcija uvodi **preciznu mapu: šta je memory, šta je state, a šta je samo context window**.

2. Tri potpuno različita pojma (kritično razdvajanje)

A) Context Window

- Privremeni radni prostor modela
- Postoji samo tokom jednog odgovora
- Nestaje odmah nakon odgovora
- Ograničen veličinom (tokeni)

Context nije pamćenje.

B) State

- Trenutno stanje procesa ili agenta
- Živi između koraka (loop-ova)
- Često se čuva spolja (JSON, varijable, DB)

Primjeri: - trenutni korak u workflow-u - status zadatka - aktivni mod (analysis / output / verify)

State je tehničko stanje, ne znanje.

C) Memory

- Namjerno sačuvana informacija
- Postoji izvan modela
- Vraća se selektivno
- Ima pravila upotrebe

Primjeri: - korisničke preferencije - trajni podaci - istorija odluka - verified facts

Memory je dizajnirana odluka, ne nusproizvod razgovora.

3. Najčešća greška: "model se sjeća"

Model se **nikada ne sjeća**.

Ako vidiš kontinuitet, to je zato što: - ti ponovo šalješ kontekst - sistem reinjectuje memoriju - state se ponovo učitava

Ako to ne znaš — ne kontrolišeš sistem.

4. Šta agent SMIJE da pamti

Agent smije čuvati samo informacije koje su: - stabilne kroz vrijeme - relevantne za buduće odluke - provjerene - bez kontekstualnog šuma

Primjeri: - trajni ciljevi - preferirani format - potvrđeni identiteti - verified procedure - dozvoljeni alati

5. Šta agent NIKADA ne smije pamtiti

- privremene pretpostavke
- neprovjerene zaključke
- emocije iz jednog razgovora
- pogrešne ulaze
- kontekstualne improvizacije
- eksperimente

Ako ovo uđe u memory → **sistem se kvari iznutra**.

6. Memory ≠ Log

Ovo je česta zabuna. - **Log** → zapis *šta se desilo* - **Memory** → odluka *šta vrijedi zapamtiti*

Sve može ići u log. Samo mali dio smije u memory.

7. Standardni Memory Gating Template

```
Candidate Information:
- Content:
- Source:
- Confidence Level:

Evaluation:
- Is it stable over time? (yes/no)
- Is it reusable? (yes/no)
- Is it verified? (yes/no)
- Is it context-dependent? (yes/no)

Decision:
- Store in memory? (yes/no)
- Memory type: (preference / fact / rule / history)
- Expiry or review condition:
```

Ako nemaš ovaj filter → **memory postaje smeće**.

8. Kako agent koristi memoriju (ispravan tok)

1. Task započinje
2. Agent učitava **samo relevantnu memoriju**
3. Kontekst se gradi
4. Zadatak se izvršava
5. Novi podaci prolaze **memory gating**
6. Samo odobreni ulaze u memory

Memory se **nikada ne koristi direktno bez provjere konteksta**.

9. Memory u Agent Loop-ovima

U loop-ovima: - **Think** → čita memory - **Act** → koristi state - **Verify** → odlučuje šta (ako išta) ide u memory

Ako memory upisuješ bez Verify faze → **to više nije agent**.

10. Primjer: medicinski AI sistem

Ako agent: - jednom pogrešno pročita LDL vrijednost - i to zapamti

→ svaki budući savjet je kompromitovan.

Ispravno ponašanje: - čuvati *izvor* - čuvati *raspon* - čuvati *status verifikacije* - nikada čuvati sirovu vrijednost bez konteksta

11. Sažetak lekcije

- Context ≠ State ≠ Memory
- Memory je **arhitektonska odluka**
- Sve što pamtiš mora biti:
  - provjereno
  - stabilno
  - korisno

Agent bez jasnih granica pamćenja → **nije inteligentan, nego opasan**.

Ovdje se završava osnovna kontrola kontinuiteta u AI sistemima.

Feedback signali: učenje bez treniranja

(kako AI "uči" bez promjene modela)

1. Ključna istina koju većina pogrešno shvata

AI sistemi **ne moraju biti trenirani** da bi se poboljšavali.

U profesionalnim sistemima: - **model ostaje isti** - **ponašanje se mijenja**

To se postiže kroz **feedback i reward signale**, ne kroz fine-tuning.

Ako ovo ne razumiješ: - miješaš RLHF sa runtime kontrolom - precjenjuješ "učenje" - praviš krhke agente

2. Šta feedback JESTE (a šta NIJE)

Feedback nije:

- ponovno treniranje
- trajna promjena modela
- "pamćenje razgovora"

Feedback jeste:

- signal kvaliteta
- kontrola ponašanja
- korekcija toka
- ulaz za loop odluke

Feedback utiče **na sljedeći korak**, ne na model.

3. Reward signal – minimalna, ali moćna informacija

Reward signal je **jednostavan pokazatelj uspjeha ili neuspjeha**.

Primjeri: - ✓ / × - score 0–1 - confidence level - pass / fail - user accept / reject

Nije objašnjenje. Nije analiza. Samo **signal**.

4. Zašto je jednostavan signal bolji od "pametnog"

Što je signal: - kraći - jasniji - binarniji

→ to je sistem stabilniji.

Kompleksni feedback: - zbunjuje agent - uvodi kontradikcije - kvari loop

Profesionalni sistemi **razdvajaju**: - signal (reward) - objašnjenje (log) - korekciju (plan)

5. Gdje se feedback koristi u Agent Loop-u

U standardnom loop-u: 1. **Think** – agent planira 2. **Act** – agent izvršava 3. **Verify** – rezultat se provjerava 4. **Reward / Feedback** – dodjeljuje se signal 5. **Adjust** – agent prilagođava sljedeći korak

Bez feedbacka, loop je slijep.

6. Learning without Training – šta se ZAISTA dešava

AI "uči" tako što: - mijenja **strategiju** - mijenja **redoslijed koraka** - mijenja **šablone** - mijenja **pravila aktivacije**

Model ostaje isti. Ponašanje evoluira.

Ovo je **runtime learning**, ne model learning.

7. Standardni Feedback & Reward Template

```
Task Outcome:
- Expected result:
- Actual result:

Evaluation:
- Success criteria met? (yes/no)
- Major errors detected? (yes/no)
- Format respected? (yes/no)

Reward Signal:
- Score: (0-1) or (pass/fail)
- Confidence level:
- Action recommendation:
  - repeat
  - adjust
  - escalate
```

Bez ovog → agent nema mjeru uspjeha.

8. Feedback ≠ Memory

Ovo je kritično. - **Feedback** → utiče na *sljedeći korak* - **Memory** → utiče na *buduće sesije*

Većina feedbacka **nikada ne ide u memory**. Samo agregirani, stabilni obrasci mogu.

9. Primjer: n8n + AI agent

Agent generiše JSON. - JSON validan → reward = pass - JSON slomljen → reward = fail

Sljedeći pokušaj: - agent mijenja strategiju - pojačava constraint - smanjuje kreativnost

Nema treniranja. Sistem se ipak poboljšava.

10. Primjer: medicinski decision-support

Ako: - preporuka odstupa od smjernica → fail - terminologija netačna → fail - izvor nije naveden → fail

Agent prilagođava: - stil - dubinu - provjere

Ne "uči medicinu". Uči **kako da se ponaša u sistemu**.

11. Najčešća greška

“Daj detaljan feedback da AI bolje nauči”

Ne.

AI treba: - jasan signal - stabilan kriterij - konzistentnu evaluaciju

Detalji idu u **log**, ne u reward.

12. Sažetak lekcije

- Feedback ≠ training
- Reward ≠ objašnjenje
- Učenje se dešava kroz **kontrolu ponašanja**
- Model ostaje isti
- Sistem postaje bolji

Ovo je osnova: - agent stabilnosti - enterprise AI - medicinskih sistema - automatizacija bez halucinacija

Nesigurnost: kad model mora reći “ne znam”

1. Problem koji ruši ozbiljne AI sisteme

Najopasnija greška AI sistema nije netačan odgovor.

Najopasnija greška je **samouvjereno pogrešan odgovor**.

Model koji: - uvijek odgovara, - nikad ne pokazuje sumnju, - ne zna reći “ne znam”,

nije pametan — nego **opasan**.

2. Zašto AI prirodno glumi sigurnost

LLM je treniran da: - nastavi tekst, - zvuči uvjerljivo, - isporuči odgovor.

On **nema prirodan impuls da stane**.

Ako mu ti ne postaviš pravila za: - nesigurnost, - nekompletne podatke, - kontradikcije,

on će **popuniti praznine**.

3. Confidence ≠ Accuracy

Važno razdvajanje: - **Confidence** → koliko uvjerljivo zvuči - **Accuracy** → koliko je tačno

Model može biti: - vrlo samouvjeren - potpuno netačan

Zato se u profesionalnim sistemima **confidence mora kontrolisati**, ne vjerovati mu se.

4. Uncertainty kao sistemska vrijednost

Ozbiljni AI sistemi **moraju znati kada da stanu**.

Uncertainty se aktivira kada: - nema dovoljno podataka, - izvori su kontradiktorni, - izlaz bi bio nagađanje, - kontekst je van definisanog opsega, - posljedice greške su visoke (medicina, pravo, finansije).

5. Kada model MORA reći “I don’t know”

Model mora eksplicitno stati kada: - podatak nije prisutan u inputu - odgovor bi zahtijevao pretpostavku - izvor nije pouzdan ili nije naveden - pitanje izlazi van domena sistema - confidence padne ispod definisanog praga

Ovo nije slabost. Ovo je **kontrola rizika**.

6. Confidence Threshold (prag sigurnosti)

Profesionalni sistemi uvode **confidence threshold**.

Primjer: - confidence ≥ 0.8 → dozvoljen odgovor - confidence 0.5–0.79 → odgovor + upozorenje - confidence < 0.5 → *ne odgovaraj*

Bez praga → sistem halucinira. Sa pragom → sistem je pouzdan.

7. Standardni Confidence & Uncertainty Template

```
Before answering:
- Do I have sufficient data? (yes/no)
- Are sources consistent? (yes/no)
- Is this within my defined scope? (yes/no)

Confidence estimation:
- Confidence score: (0–1)
- Uncertainty factors:
  - missing data
  - conflicting info
  - domain mismatch

Decision:
- Answer normally
- Answer with warning
- State explicitly: "I don't have enough information to answer this reliably.'
```

Ovo je **sistemska kočnica** protiv halucinacija.

8. "I don't know" kao kvalitet, ne greška

U ozbiljnim domenima: - medicina - farmacija - finansije - pravni sistemi - decision-support

rečenica

"Nemam dovoljno informacija da dam pouzdan odgovor"

je **najbolji mogući output**.

Sve ostalo je rizik.

9. Primjer: medicinski decision-support

Pitanje: > "Da li je ovaj pacijent kandidat za terapiju X?"

Ako nedostaje: - EF - eGFR - kalij - komorbiditeti

Ispravan odgovor: > "Nije moguće donijeti pouzdanu preporuku bez navedenih parametara."

Ne: > "Vjerovatno da."

10. Primjer: RAG sistem

Ako: - dokument ne sadrži odgovor - embedding nije pronašao relevantan chunk

Ispravno: > "Informacija nije pronađena u dostupnim izvorima."

Ne: > generisati "razumno" objašnjenje.

11. Najčešća greška

"Ako kaže 'ne znam', korisnik će biti nezadovoljan."

U realnim sistemima: - korisnik više vjeruje sistemu koji zna stati - nego onom koji uvijek priča

Povjerenje se gradi **ograničenjima**, ne brbljanjem.

12. Sažetak lekcije

- Confidence nije isto što i tačnost
- Uncertainty mora biti eksplicitna
- “I don’t know” je validan output
- Prag sigurnosti je obavezan
- Ovo štiti korisnika, sistem i tebe

Bez ove lekcije: - agenti haluciniraju - sistemi pucaju - odgovornost je nejasna

Guardrails: sigurnosna pravila i hard stop

1. Zašto guardrails uopšte postoje

AI sistemi ne pucaju zato što su glupi. Pucaju zato što **nemaju granice**.

Guardrails su **sistemska pravila ponašanja** koja određuju: - šta AI smije, - šta ne smije, - kada mora stati, - kada mora eskalirati, - kada mora odbiti zadatak.

Bez guardrails-a: - agent "ode predaleko", - halucinacije postaju uvjerljive, - sistem preuzima odgovornost koju ne smije imati.

2. Šta su guardrails (precizna definicija)

Guardrails su **nepregovarljiva pravila** koja ograničavaju ponašanje modela **bez obzira na prompt**.

Ona nisu: - stil, - ton, - uloga.

Ona su: - **sigurnosni okvir** - **kontrola rizika** - **pravna i etička barijera** - **hard stop mehanizam**

3. Tri nivoa zaštite

a) Soft guardrails (usmjeravanje)

- preporučeni ton
- preferirani format
- poželjni stil

→ mogu se prekršiti ako korisnik insistira

b) Hard guardrails (zabrane)

- zabrana davanja medicinske terapije
- zabrana pravnih savjeta
- zabrana finansijskih preporuka
- zabrana izmišljanja podataka

→ **ne smiju se prekršiti**

c) Hard stop (prekid)

- sistem **mora stati**
- nema odgovora
- nema "vjerovatno"
- nema nagađanja

→ samo objašnjenje zašto se ne može odgovoriti

4. Zašto su hard stopovi ključni

Najopasnija situacija: > "AI skoro zna odgovor."

Tu nastaje halucinacija.

Hard stop se aktivira kada: - nedostaju ključni podaci, - kontekst je van domene, - rizik greške je visok, - odgovor bi bio nagađanje.

5. Razlika: guardrails vs constraints

Važno razdvajanje: - **Constraints** → kako da odgovori - **Guardrails** → *da li uopšte smije odgovoriti*

Constraints oblikuju odgovor. Guardrails odlučuju **da li odgovor postoji**.

6. Standardni Guardrails Template

```
Safety Rules:
- Do NOT provide medical, legal or financial decisions
- Do NOT invent missing data
- Do NOT guess when information is incomplete

Hard Stop Conditions:
- Missing critical inputs
- Domain outside defined scope
- Conflicting sources without resolution
- Confidence below threshold

If any hard stop condition is met:
- Explicitly refuse to answer
- Explain why the answer cannot be given
- Suggest what information is required
```

Ovo je **nepregovarljivi dio sistema**, ne prompt trik.

7. Primjer: medical decision-support

Pitanje: > "Koju terapiju da uvedem ovom pacijentu?"

Ako nema: - EF - eGFR - K+ - terapijske istorije

Ispravan izlaz: > "Ovaj sistem ne može dati terapijsku preporuku bez navedenih parametara."

Ne: > "Uobičajeno se koristi…"

8. Primjer: n8n + API workflow

Ako je JSON: - nepotpun - pogrešno strukturiran

Hard stop: > "Output cannot be generated. Required fields are missing."

Bez guardrails-a → workflow puca u produkciji.

9. Primjer: AI agent

Agent bez guardrails-a: - sam odlučuje - improvizuje - prelazi granice

Agent sa guardrails-ima: - zna kada da pita - zna kada da stane - zna kada da eskalira

Razlika između **demo bota** i **produkcionog sistema**.

10. Najčešće greške

- "Guardrails ubijaju fleksibilnost" → Guardrails spašavaju sistem
- "Korisnik želi odgovor po svaku cijenu" → Korisnik želi **pouzdan** odgovor
- "AI je pametan, snaći će se" → AI mora biti **kontrolisan**

11. Guardrails u AI OS arhitekturi

U ozbiljnom AI OS-u: - guardrails su **iznad promptova** - ne mogu se override-ovati - vrijede za sve module - važe i za agente

Bez toga → AI OS je igračka.

12. Sažetak lekcije

- Guardrails definišu granice
- Hard stops sprječavaju katastrofe
- Constraints nisu isto što i guardrails
- "Ne mogu odgovoriti" je validan output
- Ovo je temelj produkcionih AI sistema

Bez guardrails-a: - agenti haluciniraju - sistemi pucaju - odgovornost je mutna

Sa njima: - sistem je pouzdan - skalabilan - profesionalan

Kad prompt postane sistem: production readiness

Kada prompt prestaje biti trik i postaje sistem

1. Šta znači "production-ready" u AI kontekstu

Prompt nije production-ready zato što: - daje dobar odgovor jednom, - zvuči pametno, - izgleda elegantno.

Prompt je production-ready kada: - daje **konzistentan output**, - radi **bez tebe u petlji**, - preživljava loš input, - ne halucinira pod pritiskom, - zna kada **ne smije odgovoriti**, - može biti dio **sistema**.

Production readiness ≠ pametan prompt Production readiness = **pouzdano ponašanje**

2. Evolucija: od prompta do sistema

Realna hijerarhija izgleda ovako: 1. **Single prompt** → ručni eksperiment 2. **Pattern-based prompt** → kontrola ponašanja 3. **Multi-pattern orchestration** → stabilnost 4. **Guardrails + validation** → sigurnost 5. **Agent loops + control flow** → autonomija 6. **Production system** → povjerenje

Ako nešto preskočiš — sistem će ti to naplatiti kasnije.

3. Checklista: da li je nešto production-ready

Ovo je **nepregovarljiva lista**.

Kontrola ponašanja

- koristi pattern(e), ne slobodan tekst
- zna koji misaoni režim je aktivan

Validacija

- provjerava vlastiti output
- ima Self-Correction ili WRI loop

Ograničenja

- koristi constraint-based prompting
- format je stabilan (JSON / tabela)

Sigurnost

- ima guardrails
- ima hard stop uslove

Kontinuitet

- ponašanje je isto danas i sutra
- nije zavisno od "dobrog pitanja"

Ako makar jedna stavka fali → **nije produkcija**.

4. Najčešća iluzija (i zašto projekti propadaju)

Iluzija: > “Radi savršeno kod mene.”

Realnost: - produkcija = loš input - produkcija = nejasni zahtjevi - produkcija = pogrešan redoslijed - produkcija = stres test

AI koji radi samo kad je sve idealno → **nije AI sistem**.

5. Production prompt ≠ lijep prompt

Lijep prompt: - čitljiv - inspirativan - kreativan

Production prompt: - dosadan - striktan - repetitivan - “težak”

Ali: - ne puca - ne improvizuje - ne izmišlja

U produkciji **ljepota je bug**, ne feature.

6. Standardni Production Prompt Skeleton

```
System Scope:
- Define domain and responsibility boundaries

Active Patterns:
- List explicit patterns in use

Constraints:
- Format, length, content limits

Validation:
- Self-check / verification step

Guardrails:
- Hard rules
- Hard stop conditions

Output Contract:
- Exact structure
- No additional text

Failure Mode:
- What to do if output cannot be generated
```

Ako tvoj prompt nema sve ovo → **još je igračka**.

7. Razlika: demo vs produkcija

Demo	Produkcija
Jedan primjer	Svi primjeri
Ručna korekcija	Autonomna korekcija
“Izgleda pametno”	“Radi uvijek”
Tolerira greške	Ne smije

8. Production mindset (ovo je najvažnije)

U produkciji: - **nemaš povjerenje u model** - vjeruješ samo sistemu - sve što može poći po zlu → hoće - "ne znam" je validan output

Ovo nije pesimizam. Ovo je **inženjerska realnost**.

9. Kako ova knjiga treba da se koristi

Ovo **nije**: - lista trikova - prompt cheat-sheet - "10 načina da…"

Ovo **jeste**: - misaoni okvir - sistemski alat - osnova za AI OS - temelj za agente, RAG, n8n, CDS, produkciju

Ako čitalac samo kopira promptove → promašio je poentu. Ako razumije obrasce → ima moć.

10. Završna istina

AI nije opasan jer je pametan. Opasan je jer **ljudi vjeruju outputu bez sistema**.

Ova knjiga postoji da to spriječi.

11. Završni sažetak knjige

- Prompting nije pisanje — nego **kontrola ponašanja**
- Patterni nisu trikovi — nego **mentalni modovi**
- Agent nije bot — nego **kontrolisana petlja**
- Guardrails nisu ograničenje — nego **sigurnost**
- Produkcija ne traži kreativnost — nego **disciplinu**

Ako si došao do ovdje → razmišljaš **sistemski**, ne korisnički.

Prompt Orchestration / Multi-Pattern Control

Kako AI prestaje da koristi obrasce pojedinačno i počinje da ih koristi kao sistem

1. Uvod: Zašto pojedinačni patterni nisu dovoljni

Do ove tačke u knjizi, naučio si kako da koristiš pojedinačne obrasce: - strukturiraš podatke, - razlažeš zadatke, - analiziraš tabele, - vizualizuješ rezultate, - poboljšavaš i evaluiraš output.

Međutim, u realnom radu s AI-jem, problemi **rijetko dolaze izolovano**.

Stvarni zadaci zahtijevaju da AI: - prvo razumije kontekst, - zatim postavi pitanja, - onda generiše rješenje, - provjeri tačnost, - poboljša kvalitet, - i tek na kraju isporuči odgovor.

Ovdje nastupa **Prompt Orchestration**.

To je obrazac koji omogućava da **više patterna radi zajedno**, u kontrolisanom redoslijedu, sa jasnim ulogama i granicama.

2. Šta je Prompt Orchestration?

Prompt Orchestration je tehnika kojom **svjesno upravljaš redoslijedom, ulogom i međusobnim odnosom više prompt obrazaca** u jednom zadatku.

Umjesto da AI koristi jedan pattern "ad hoc", ti mu govoriš: - koji obrazac koristi *prvo*, - koji *zatim*, - koji ima *kontrolnu ulogu*, - a koji je *finalni filter*.

Analogija je jasna: > Pojedinačni pattern = muzičar > Prompt Orchestration = dirigent

Bez dirigenta, svi sviraju — ali ne zajedno.

3. Zašto je Multi-Pattern Control kritičan?

Bez orkestracije, AI: - miješa logike, - preskače važne faze, - daje dobar odgovor prerano, - ne provjerava vlastite greške, - gubi stabilnost u složenim zadacima.

Multi-Pattern Control omogućava: - **predvidivo ponašanje modela**, - **kontrolu kvaliteta po fazama**, - **ponovljivost rezultata**, - **sigurnu upotrebu u profesionalnim sistemima**.

Ovo je razlika između: - "AI mi ponekad pomogne"

i

- "AI mi je pouzdan dio sistema".

4. Osnovna logika orkestracije (mentalni model)

Svaka dobra orkestracija ima četiri sloja:

a) Ulazni sloj – Razumijevanje zadatka

(npr. FIP, Reframing)

AI mora znati: - šta želiš, - zašto, - u kom kontekstu.

b) Izvršni sloj – Generisanje

(npr. Task Decomposition, Structured Extraction, Tabular Reasoning)

Ovdje AI **radi posao**.

c) Kontrolni sloj – Provjera

(npr. WRI, Evaluation, Error Detection)

Ovdje AI **provjerava sam sebe**.

d) Izlazni sloj – Finalna isporuka

(strukturiran, jasan, siguran output)

Bez ova četiri sloja, sistem je nestabilan.

5. Standardni Prompt Orchestration Template

Koristi sljedeći orkestrirani pristup:

Faza 1 – Razumijevanje (Context & Clarification): Postavi pitanja dok ne razumiješ cilj, kontekst i ograničenja.

Faza 2 – Izvršenje (Generation): Razloži zadatak na korake i generiši rješenje po fazama.

Faza 3 – Provjera (Evaluation): Provjeri tačnost, logiku i kontekstualnu primjenjivost.

Faza 4 – Poboljšanje (Refinement): Poboljšaj odgovor na osnovu identifikovanih slabosti.

Faza 5 – Finalni output: Isporuči jasan, strukturiran i siguran odgovor.

Ovaj šablon je **kičma svih ozbiljnih AI workflowa**.

6. Primjer orkestracije u praksi

Zadatak: > "Analiziraj laboratorijske podatke pacijenata i pripremi izvještaj."

Orkestracija: 1. **FIP** – AI pita o populaciji, cilju analize, referentnim vrijednostima. 2. **Structured Extraction + Tabular Reasoning** – podaci se čiste, normalizuju i analiziraju. 3. **Data Visualization** – trendovi se prikazuju grafički. 4. **WRI** – izvještaj se piše, reflektuje i poboljšava. 5. **Evaluation Pattern** – provjerava se tačnost i medicinska sigurnost.

→ Rezultat: izvještaj koji je **analitički tačan, čitljiv i siguran za upotrebu**.

Bez orkestracije, isti zadatak bi dao nepouzdan i fragmentiran output.

7. Gdje se Prompt Orchestration koristi

a) AI OS sistemi

Svaki modul (zdravlje, znanje, projekti) ima svoj orkestrirani tok.

b) RAG sistemi

Upit → pretraga → generisanje → evaluacija → finalni odgovor.

c) AI agenti

Agenti bez orkestracije su nepredvidivi.

d) Automatizacije (n8n)

Svaki node je faza u orkestriranom procesu.

e) Profesionalni prompt engineering

Bez orkestracije, nema skalabilnosti.

8. Sažetak lekcije

Prompt Orchestration / Multi-Pattern Control je **razlika između korištenja AI-ja i upravljanja AI-jem**.

Ovaj obrazac: - povezuje sve prethodne lekcije, - daje stabilnost sistemu, - omogućava profesionalnu upotrebu, - sprečava haos u složenim zadacima.

Ako su patterni alati, **orkestracija je arhitektura**.

Bez nje, nema ozbiljnog AI sistema.

Multi-Prompt Orchestration: kada prompti postanu proces

Kako jedan prompt prerasta u kontrolisani tok

Uvod: zašto jedan prompt više nije dovoljan

U prethodnoj lekciji (*Prompt Orchestration / Multi-Pattern Control*) vidio si kako je moguće **orkestrirati više misaonih obrazaca unutar jednog prompta**.

Multi-Prompt Orchestration ide **korak dalje**.

Ovdje više ne govorimo o: - jednom upitu - jednom odgovoru - jednom misaonom toku

Već o **sekvenci promptova** koji zajedno čine **kontrolisani proces**.

Drugim riječima, AI više ne reaguje — **AI izvršava tok**.

Ovaj obrazac čini osnovu za: - AI agente - višefazne analize - RAG sisteme - n8n automatizacije - AI OS arhitekture

Ako želiš stabilne, ponovljive i profesionalne rezultate, **Multi-Prompt Orchestration nije opcija — već nužnost**.

Šta je Multi-Prompt Orchestration

Multi-Prompt Orchestration Pattern je tehnika u kojoj se **više promptova koristi kao koordinisana cjelina**, pri čemu svaki prompt ima: - jasno definisanu ulogu - precizno mjesto u toku - kontrolisan input i output - jasan odnos prema prethodnim i narednim koracima

Svaki prompt predstavlja **jednu fazu procesa**, a ne improvizaciju.

Model ne "razmišlja ispočetka" svaki put, već **nastavlja već započeti tok**.

Zašto je ovaj pattern presudan u praksi

Bez orkestracije više promptova, AI sistemi vrlo brzo pokazuju slabosti: - gubitak konteksta - nedosljedne odgovore - unutrašnje kontradikcije - nepotrebno ponavljanje - nekontrolisan i nestabilan output

Multi-Prompt Orchestration omogućava: - fazno razmišljanje - razdvajanje odgovornosti - evaluaciju svakog koraka - povratne petlje (feedback loops) - stabilne i predvidive AI tokove

Ovo je razlika između: > "AI mi pomaže"

i

> "AI je dio sistema".

Osnovna arhitektura multi-prompt sistema

Tipičan orkestrirani tok izgleda ovako: 1. **Prompt 1 — Analiza i razumijevanje** 2. **Prompt 2 — Obrada i generisanje** 3. **Prompt 3 — Evaluacija i provjera** 4. **Prompt 4 — Korekcija i finalizacija**

Svaki prompt: - prima tačno definisan input - proizvodi kontrolisan output - predaje rezultat sljedećem koraku

AI ne improvizuje — **izvršava unaprijed definisani tok**.

Standardni Multi-Prompt Orchestration Template

Prompt 1 — Context Builder - Razumije cilj - Postavlja pitanja ako je potrebno - Definiše okvir zadatka

Prompt 2 — Core Execution - Izvršava glavni zadatak - Generiše primarni output

Prompt 3 — Evaluation Layer - Provjerava logiku - Identifikuje greške - Procjenjuje kvalitet

Prompt 4 — Improvement Layer - Ispravlja uočene probleme - Poboljšava strukturu i jasnoću - Isporučuje finalni output

Template se može proširivati ili skraćivati, ali **redoslijed se ne preskače**.

Interna logika: zašto ovo funkcioniše

Multi-Prompt Orchestration koristi isti princip kao profesionalni tim: - jedan analizira - drugi radi - treći provjerava - četvrti unapređuje

AI modeli su vrlo dobri u svakoj od ovih uloga **pojedinačno**, ali nepouzdani kada sve pokušavaju raditi istovremeno.

Razdvajanjem u više promptova: - smanjuješ kognitivno opterećenje modela - dobijaš preciznije rezultate - smanjuješ rizik od halucinacija - dobijaš kontrolu nad procesom

Primjer orkestriranog toka (sažeto)

Zadatak: "Napravi analizu tržišta za digitalni AI proizvod u zdravstvu."

Prompt 1 — Analiza - Koja je ciljna publika? - Koji problem se rješava? - Koji je kontekst tržišta?

Prompt 2 — Generisanje - Analiza tržišta - Segmentacija - Prilike i rizici

Prompt 3 — Evaluacija - Jesu li pretpostavke realne? - Postoje li rupe u logici? - Šta nedostaje?

Prompt 4 — Poboljšanje - Preciziranje - Strukturiranje - Finalna verzija

Rezultat je dokument koji izgleda kao da ga je radio **tim stručnjaka**, a ne jedan model.

Gdje se ovaj pattern koristi u realnim sistemima

AI agenti Agent nije jedan prompt, već skup orkestriranih promptova.

RAG sistemi Jedan prompt za retrieval, drugi za sintezu, treći za validaciju.

n8n automatizacije Svaki node predstavlja jednu fazu orkestracije.

AI OS sistemi Dnevni procesi, evaluacije i refleksije oslanjaju se na sekvence.

Profesionalno pisanje i analitika Izvještaji, strategije i tehnička dokumentacija.

Ključna razlika u odnosu na druge pattern-e

- Task Decomposition → razlaže zadatak
- Write → Reflect → Improve → poboljšava jedan output
- Multi-Pattern Control → više patterna u jednom promptu
- **Multi-Prompt Orchestration → kontrolisani sistem promptova**

Ovo je **arhitekturni pattern**, a ne samo prompting tehnika.

Sažetak

Multi-Prompt Orchestration Pattern omogućava izgradnju **stabilnih, ponovljivih i profesionalnih AI sistema**.

Umjesto jednog pametnog prompta, koristiš **pažljivo orkestriran proces**.

Ovaj obrazac predstavlja most između prompt inženjeringa i stvarnih AI sistema.

Part II — Under the Hood: How These Patterns Work in Practice

Output & Structure

Output Format Grounding

1. Uvod u Output Format Grounding

Output Format Grounding je napredna prompt inženjerska tehnika kojom se eksplicitno definiše *format izlaza* koji AI model mora isporučiti.

Umjesto da "pogađa" kako odgovor treba izgledati, model dobija preciznu strukturu koju mora slijediti.

Najčešće korišteni formati su: - **JSON**, - **tabela**, - **bullet list**, - **CSV**, - **kod**, - **sekcionirani tekst**, - **XML ili YAML (rjeđe)**.

Ovaj obrazac je ključan za sisteme gdje je bitna tehnička stabilnost i mogućnost dalje obrade.

2. Zašto je Output Format Grounding ključan?

Bez eksplicitnog definisanja outputa: - odgovor često ima pogrešnu strukturu, - podaci se teško uklapaju u API ili n8n, - JSON se može slomiti (što ruši workflow), - model ubacuje nepotreban tekst, - output zna varirati od primjerka do primjerka.

Output Format Grounding rješava sve ove probleme jer: - garantuje konzistentan format, - omogućava automatsku obradu, - uklanja šum (noise), - čini AI integracije pouzdanim, - daje profesionalne i predvidljive rezultate.

Ovo je temelj svake ozbiljne AI primjene.

3. Standardni prompt šablon za Output Format Grounding

Za sljedeći zadatak generiši output isključivo u ovom formatu:

[navedi tačan format]

JSON format

```
{
  "title": "",
  "summary": "",
  "key_points": [],
  "risk_level": ""
}
```

Tabela (tekstualna)

kolona_1	kolona_2	kolona_3
vrijednost	vrijednost	vrijednost

Bullet list

- ključna tačka 1
- ključna tačka 2
- ključna tačka 3

CSV struktura

```
file_name,width,height,description
value1,value2,value3,value4
```

DODATNA PRAVILA: 1. Ne dodavati tekst van definisanog formata. 2. Ne mijenjati nazive ključeva ili kolona. 3. Ne dodavati objašnjenja prije ili poslije outputa. 4. Popuniti samo tražena polja. 5. Prazne vrijednosti označiti kao `null`.

Ovaj šablon garantuje apsolutnu kontrolu nad izlazom.

4. Interna logika procesa

Kada je format unaprijed definisan, AI model: 1. prepoznaje izlaznu strukturu kao obavezujući okvir, 2. usklađuje sadržaj sa strukturom, 3. filtrira sve što ne pripada formatu, 4. optimizuje svoje zaključke da stanu u zadani dizajn, 5. smanjuje halucinacije jer se fokusira na popunjavanje polja.

Zbog toga Output Format Grounding spada u tehnike koje najdramatičnije povećavaju kvalitet outputa.

5. Primjena u raznim kontekstima

a) n8n workflow-i

JSON output mora biti savršen. Jedna pogrešna zagrada može oboriti cijeli workflow.

b) RAG sistemi

Kvalitet outputa zavisi od dosljednosti formata (citati, izvori, reference).

c) AI OS

Output box-ovi (Complete / Focus / Custom) koriste specifičan format.

d) SEO Scraper

CSV i JSON outputi moraju biti identični svaki put.

e) Poslovni izvještaji

Standardizirana struktura olakšava organizaciju i integraciju.

f) Medicinski sadržaji

Strukturirani izlazi smanjuju rizik od pogrešne interpretacije.

6. Primjer Output Format Grounding primjene

Zadatak: “Napiši sažetak o značaju fizičke aktivnosti.”

Format: JSON

Output:

```
{
  "title": "Znacaj fizicke aktivnosti",
  "summary": "Redovna fizicka aktivnost poboljsava
   ↪  kardiovaskularno zdravlje, regulise tjelesnu tezinu i
   ↪  smanjuje rizik od hronicnih bolesti.",
  "key_points": [
    "poboljsava rad srca",
    "regulise metabolizam",
    "smanjuje upalne procese"
  ],
  "risk_level": "low"
}
```

Ovo je savršeno strukturiran odgovor, spreman za daljnju automatizaciju ili integraciju.

7. Sažetak lekcije

Output Format Grounding omogućava potpunu kontrolu nad AI rezultatima.

Kroz eksplicitno definisanje strukture odgovora, AI modeli postaju predvidljivi, tehnički stabilni i pogodniji za integraciju u realne sisteme.

Ovaj pattern je nezaobilazan u svim profesionalnim aplikacijama, od n8n workflow-a do RAG inženjeringa, AI OS-a i poslovne analitike.

JSON Structuring Pattern

1. Uvod u JSON Structuring Pattern

JSON Structuring Pattern predstavlja tehniku preciznog definiranja načina na koji AI model treba generisati JSON podatke.

JSON (JavaScript Object Notation) je univerzalni standard za: - prenos podataka između sistema, - API komunikaciju, - n8n i ostale no-code alate, - RAG metapodatke, - konfiguracije AI agenata, - strukture memorije u AI OS-u.

Bez strogo definisanog JSON-a, AI nije moguće integrisati u tehnološke sisteme, jer i najmanja greška prekida proces.

Zato se ovaj pattern smatra **apsolutnim temeljem profesionalnog prompt inženjeringa**.

2. Zašto je JSON Structuring Pattern ključan?

AI modeli često generišu: - nepotpun JSON, - slomljene zagrade, - pogrešne tipove podataka, - nizove (array-e) sa sintaksnim greškama, - dodatni tekst koji slomi automatizacije.

JSON Structuring Pattern rješava ove probleme: - garantuje validan JSON, - precizno definira ključeve i vrijednosti, - osigurava čitljivost i dosljednost, - omogućava sigurnu integraciju u n8n, API-je ili aplikacije, - uklanja sve nepredvidive elemente iz outputa.

Ovo je neophodno za svaki tehnički AI projekt.

3. Standardni JSON Structuring Template

Generiši JSON isključivo u sljedećem formatu:

```
{
  "title": "",
  "description": "",
  "tags": [],
  "data": {
    "key_metrics": {},
    "summary": ""
  }
}
```

Pravila: 1. Ne dodavati tekst van JSON strukture. 2. Ne mijenjati nazive ključeva. 3. Ne dodavati komentare unutar JSON-a. 4. Sve vrijednosti moraju biti validni JSON tipovi (string, number, boolean, null, array, object). 5. Ako nema vrijednosti, koristiti `null` ili prazan array `[]`. 6. Output mora biti validan JSON koji se može parsirati.

4. Interna logika JSON Structuring Patterna

Kada model dobije definisanu strukturu:

a) Prepoznaje JSON kao obavezujući okvir

Model zna da ne smije koristiti prirodni jezik van zagrada.

b) Popunjava svaki ključ odgovarajućim tipom podataka

Ovo automatski smanjuje rizik od halucinacija.

c) Provjerava logičku konzistenciju

Struktura ostaje stabilna i predvidljiva.

d) Fokusira se na semantičko punjenje podataka

Pošto je format unaprijed definisan, model se bavi samo kvalitetom sadržaja.

Ovaj proces daje *determinističke* rezultate — ključ za automatizacije.

5. Najčešće JSON strukture u AI sistemima

a) n8n workflow output JSON

```
{
  "status": "ok",
  "result": {},
  "next_step": ""
}
```

b) RAG metadata JSON

```
{
  "chunk_id": "",
  "text": "",
  "source": "",
  "embedding_vector": []
}
```

c) AI OS Memory Record

```
{
  "date": "",
  "category": "",
  "entry": "",
  "insights": []
}
```

d) SEO Scraper entry

```
{
  "url": "",
  "title": "",
  "keywords": [],
  "meta_description": ""
}
```

e) Medical report structuring

```
{
  "pacijent": "",
  "datum": "",
  "parametri": {
    "LDL": null,
    "HDL": null,
```

```
    "EF": null
  },
  "napomena": ""
}
```

Ove strukture su osnova za tvoje praktične projekte.

6. Primjena JSON Structuring Patterna

a) RAG sistemi

Čisti JSON omogućava kvalitetan ingestion i embedding.

b) n8n automatizacije

Workflow-i zavise od stabilne JSON strukture. Greška u zagradi → workflow pada.

c) AI OS

Tvoji output box-ovi (Complete / Focus / Custom) mogu koristiti JSON memoriju.

d) SEO Scraper backend

Scrapovani podaci moraju biti strukturirani u stabilan JSON.

e) API integracije

JSON je jedini prihvaćeni format za većinu API-ja.

f) Bildanje AI agenata

Agent logika se zasniva na JSON porukama između sistema.

7. Primjer primjene JSON Structuring Patterna

Zadatak: "Napiši analizu sadržaja web stranice i vrati je kao strukturirani JSON."

Output:

```
{
  "title": "Analiza web stranice",
  "description": "Sadrzaj stranice fokusiran je na usluge
   ↪  digitalnog marketinga.",
  "tags": ["marketing", "seo", "usluge"],
  "data": {
    "key_metrics": {
      "broj_rijeci": 1280,
      "keyword_density": 2.3,
      "internal_links": 14
    },
    "summary": "Stranica je dobro strukturirana, ali meta
     ↪  opisi su neujednačeni."
  }
}
```

Ovo je profesionalni, stabilan output — spreman za automatizaciju.

8. Sažetak lekcije

JSON Structuring Pattern omogućava potpunu kontrolu nad strukturom podataka koju AI generiše.

Korištenjem precizno definisanih JSON okvira, korisnik dobija predvidljive, tehnički ispravne i pouzdane rezultate koji se mogu direktno uključiti u automatizovane sisteme, aplikacije i procese obrade podataka.

Ova tehnika je neizostavan dio modernog prompt inženjeringa i temelj svakog profesionalnog AI workflow-a.

Structured Extraction Pattern

Kako AI pretvara haotičan tekst u podatke koji se mogu koristiti

1. Uvod u Structured Extraction

Structured Extraction je tehnika kojom se **nestrukturirani sadržaj** — dokument, PDF, paragraf, web-stranica ili tekst iz slike — pretvara u **jasno definisanu strukturu**.

Najčešći oblici te strukture su: - tabela, - JSON objekt, - lista ključnih informacija, - skup kategorija, - uredan CSV dataset.

Ono što ovu tehniku čini ključnom nije sama ekstrakcija, nego činjenica da se složen i nepredvidiv sadržaj **pretvara u nešto što AI sistemi mogu pouzdano koristiti kao podatke**.

Zahvaljujući Structured Extraction patternu, AI prestaje biti samo alat za čitanje i sažimanje — i postaje **univerzalni mehanizam za izdvajanje informacija** u medicini, biznisu, edukaciji i analitici.

2. Zašto je Structured Extraction ključan?

Većina informacija u realnom svijetu **ne dolazi u tabelama**.

One dolaze kao: PDF-ovi, e-mailovi, članci, medicinski izvještaji, upisni formulari, tekstualne bilješke, web sadržaj.

Bez strukturiranja: - podaci su teško pretraživi, - ne mogu se pouzdano povezivati, - ne mogu ući u RAG sisteme, - nisu pogodni za analitiku, - ne mogu se koristiti u automatizacijama (n8n, API, CRM).

Structured Extraction rješava upravo taj problem: on pretvara haotičan input u **precizan, dosljedan i mašinski upotrebljiv format**.

Drugim riječima — bez ove tehnike, AI sistemi nemaju *na čemu* da rade.

3. Standardni prompt šablon za Structured Extraction

Da bi ekstrakcija bila pouzdana, ona mora biti **standardizovana**.

Zato se koristi jasan prompt-šablon koji ograničava model i sprječava nagađanja.

Structured Extraction Prompt Template

> Izdvoji ključne informacije iz sljedećeg teksta i organizuj ih u strukturirani format.
>
> 1. Identifikuj sve relevantne entitete (navedi ih).
> 2. Organizuj ekstrakciju u oblik:
> - **JSON** objekta, ili
> - **tabele sa kolonama**, ili
> - **CSV dataset-a** (navedi format).
> 3. Ako postoje nedostajući podaci, označi ih kao `null`.
> 4. Uključi samo informacije koje se eksplicitno nalaze u tekstu.
> 5. Ne dodaj pretpostavke ili dodatna tumačenja.
> 6. Označi dvosmislene ili nejasne stavke u podacima.

Ovaj šablon služi kao **ugovor** između tebe i AI-ja: model zna šta smije da radi — i, jednako važno, šta ne smije.

4. Interna logika procesa

Structured Extraction se ne dešava u jednom potezu. Proces ima **tri jasne faze**.

a) Identifikacija entiteta

Model prvo odlučuje **šta je važno**: - imena, - datumi, - numeričke vrijednosti, - terapije, - simptomi, - kategorije, - naslovi i oznake.

Ako ova faza zakaže, cijela ekstrakcija je pogrešna.

b) Normalizacija podataka

Zatim se podaci **čiste i standardizuju**: - datumi prelaze u ISO format, - imena se stilizuju konzistentno, - brojevi postaju numerički tipovi.

Cilj nije ljepota — cilj je **dosljednost**.

c) Mapping → struktura

Na kraju se informacije raspoređuju u: - JSON ključeve, - kolone tabele, - CSV polja.

U tom trenutku dokument prestaje biti tekst i postaje **dataset** koji može ući u AI sisteme.

5. Primjena u raznim kontekstima

a) RAG (Retrieval-Augmented Generation)

Dokumenti se moraju strukturirati prije indeksiranja u vektorske baze. Bez toga, pretraga postaje nepouzdana.

b) Medicinski i farmaceutski sadržaji

Ekstrakcija ključnih parametara kao što su: lijek, doziranje, indikacija, EF, LDL, simptomi — direktno iz PDF-ova i izvještaja.

c) AI OS – organizacija znanja

Bilješke, članci i dokumenti se pretvaraju u strukture koje se mogu: pretraživati, sortirati, tagirati i povezivati.

d) Analiza PDF-ova i izvještaja

Automatsko pronalaženje vrijednosti, preporuka i klasifikacija bez ručnog čitanja.

e) No-code sistemi i n8n automatizacije

Strukturirani output se direktno šalje u: Notion, Google Sheets, CRM sisteme i API pozive.

6. Primjer Structured Extraction transformacije

Ulazni tekst: > Pacijent: Marko M. > Datum: 12.2.2024. > Terapija: Rosuvastatin 20 mg. > Kontrola za 3 mjeseca. > LDL: 3.4 mmol/L. > Napomena: smanjiti unos zasićenih masti.

Ekstrakcija (JSON format)

```
{
  "pacijent": "Marko M.",
  "datum": "2024-02-12",
  "terapija": "Rosuvastatin 20 mg",
  "kontrola": "3 mjeseca",
  "ldl": 3.4,
  "napomena": "smanjiti unos zasićenih masti"
}
```

Ekstrakcija (tabela)

Pacijent	Datum	Terapija	Kontrola	LDL	Napomena
Marko M.	2024-02-12	Rosuvastatin 20 mg	3 mjeseca	3.4	smanjiti unos zasićenih masti

Ovaj output je **odmah spreman** za RAG, n8n pipeline, evidencije i CRM sisteme.

7. Sažetak lekcije

Structured Extraction Pattern omogućava da nestrukturirani tekst postane **podatak koji se može koristiti**.

Transformacija teksta u JSON, tabelu ili CSV predstavlja jedan od **temelja modernog AI ekosistema**, posebno u sistemima koji zavise od tačne i dosljedne obrade informacija.

Bez ovog patterna, automatizacija i RAG sistemi nemaju stabilnu osnovu.

Data Transformation Pattern

Uvod

Većina grešaka u radu s podacima **ne nastaje zbog loše analize**, nego zbog loše pripreme podataka. Brojevi su tu, kolone postoje, sve izgleda "u redu" — ali zaključci su pogrešni.

Zašto?

Zato što AI (i ljudi) **razmišljaju drugačije u zavisnosti od oblika podataka**.

Data Transformation Pattern se bavi upravo tim slojem rada, ne analizom, nego **pripremom podataka tako da analiza uopće ima smisla**.

Ako preskočiš ovu fazu, Code Interpreter može savršeno izračunati — **pogrešnu stvar**.

Data Transformation Pattern je standardizovani obrazac koji omogućava precizno upravljanje načinom na koji AI model transformiše, čisti i reorganizuje podatke.

Bez ovakvog obrasca, rezultati su često nedosljedni, nepotpuni ili pogrešno strukturirani.

Cilj ovog patterna je osigurati: - predvidljiv output, - stabilnu logiku transformacije, - kontrolu nad formatom, - mogućnost integracije rezultata u automatizovane sisteme.

Ovaj pristup predstavlja temelj za profesionalni rad sa datasetovima i naprednim AI workflow-ima.

Šta znači "data transformation"

Data transformation znači **svjesno mijenjanje strukture, formata ili organizacije podataka** prije nego što se nad njima radi analiza.

To uključuje: - čišćenje podataka - grupisanje i agregaciju - promjenu formata (datumi, tekst, brojevi) - spajanje ili razdvajanje kolona - filtriranje i selekciju

Važno je razumjeti: > transformacija ne dodaje nove informacije, > ali dramatično mijenja **šta iz postojećih možeš zaključiti**.

Bez jasnih instrukcija, AI model: - ne zna šta treba ukloniti ili zadržati, - ne zna kako strukturirati podatke, - često mijenja format, - gubi informacije ili stvara duplikate.

Data Transformation Pattern uklanja te nedostatke jer modelu daje eksplicitna pravila transformacije.

Primjena je posebno važna u: - analizi Excel fajlova, - pripremi podataka za RAG, - radu sa CSV-ovima i API outputima, - čišćenju tekstualnih podataka, - n8n pipeline-ima za data processing, - SEO scraper backend strukturama.

Zašto je Data Transformation ključna faza workflowa

Bez transformacije, podaci su često: - previše detaljni - neujednačeni - nepogodni za poređenje - pogrešno grupisani

AI tada: - računa ispravno - ali zaključuje pogrešno

Sa dobrom transformacijom: - obrasci postaju vidljivi - trendovi se pojavljuju - anomalije iskaču same

Zato se u ozbiljnoj analitici kaže: > "Ako transformacija nije dobra, analiza je nebitna."

Tipične transformacije koje AI koristi

Čišćenje podataka (Cleaning)

AI uklanja ili ispravlja: - prazne vrijednosti - duplikate - nelogične zapise - pogrešne formate

Bez ovoga, prosjeci i trendovi lažu.

Agregacija (Grouping & Aggregation)

Umjesto pojedinačnih zapisa, AI: - grupiše po datumu, kategoriji, regionu - računa zbir, prosjek, minimum, maksimum

Primjer: - dnevni podaci → mjesečni trend - pojedinačni zapisi → pregled po kategoriji

Ovdje se **pojavljuje značenje**, ne samo brojevi.

Promjena strukture (Reshaping)

AI može: - "okrenuti" tabelu (rows ↔ columns) - razdvojiti jednu kolonu u više - spojiti više kolona u jednu

Ovo je često presudno za vizualizaciju i poređenje.

Filtriranje i selekcija

Ne moraju svi podaci ući u analizu.

AI: - izdvaja relevantne zapise - ignoriše šum - fokusira se na ono što odgovara cilju

Bez ove faze, analiza postaje bučna i nejasna.

Kako izgleda dobar Data Transformation prompt

Dobar prompt **ne traži odmah zaključak**. On prvo traži pripremu.

Primjer:

```
Use Code Interpreter.

1. Load the dataset.
2. Clean the data and explain what was removed or corrected.
3. Transform the data to a monthly aggregated format.
4. Explain why this transformation is appropriate.
5. Only then perform the analysis.
```

Ovim promptom: - tjeraš AI da razmišlja o obliku podataka - dobijaš transparentnost - znaš **zašto** su rezultati takvi kakvi jesu

Data Transformation kao misaoni, ne tehnički pattern

Iako se izvodi kodom, Data Transformation Pattern je prije svega **misaoni obrazac**.

Pitanja koja uvijek trebaš sebi (ili AI-ju) postaviti: - U kom obliku su podaci sada? - U kom obliku trebaju biti da bi odgovor imao smisla? - Šta gubim, a šta dobijam ovom transformacijom?

Ovdje AI postaje: > analitičar koji priprema materijal prije zaključivanja

Kratka analogija

Zamisli da gledaš mapu. Ako je previše detaljna — izgubiš se. Ako je previše pojednostavljena — nemaš kontekst.

Data Transformation je **biranje prave mape za put koji želiš preći**.

Sažetak

Data Transformation Pattern osigurava da AI analizira podatke u pravom obliku. Ne radi se o računanju, nego o pripremi. Kada su podaci pravilno transformisani, zaključci postaju jasni, relevantni i pouzdani. Bez ove faze, i najbolja analiza stoji na klimavim nogama.

Data & Document Handling

From Images to Data: OCR & Visual Understanding

1. Uvod u OCR i vizualno razumijevanje

OCR (Optical Character Recognition) i Image Understanding predstavljaju skup tehnika koje omogućavaju AI modelu da **radi sa vizualnim svijetom na isti način na koji inače radi sa tekstom**.

To uključuje sposobnost da model: - čita tekst sa slike, - prepoznaje objekte, lica, simbole i strukture, - razumije raspored i odnose između elemenata, - interpretira kompoziciju slike, - i na kraju — pretvara vizualne informacije u formalne, mašinski upotrebljive podatke.

Ove dvije komponente zajedno čine temelj za sve AI sisteme koji moraju raditi sa **realnim dokumentima**, fotografijama i snimcima ekrana — onim što ljudi zapravo koriste u svakodnevnom radu, a ne idealiziranim digitalnim formatima.

Drugim riječima: **OCR i Image Understanding su most između stvarnog svijeta i AI sistema.**

2. Zašto je ova tehnika važna?

Većina informacija u stvarnom svijetu **ne nastaje digitalno**.

One postoje kao: - fotografije, - medicinski izvještaji, - laboratorijski nalazi uslikani telefonom, - formulari, - promotivni materijali, - grafikoni i dijagrami, - snimci ekrana aplikacija i sistema.

Bez OCR-a i vizualnog razumijevanja, sav taj sadržaj ostaje **izolovan**: - ne može se indeksirati, - ne može se koristiti u RAG sistemima, - ne može ući u automatizaciju (n8n), - ne može biti dio AI OS memorije, - ne može se analizirati, filtrirati ili kategorizirati.

OCR i Image Understanding rješavaju upravo taj problem: oni omogućavaju da vizualni sadržaj postane **podatkovni resurs**, a ne mrtva slika.

3. Standardizovani prompt šablon za OCR & Image Understanding

Da bi ova sposobnost bila pouzdana, mora biti **standardizovana**.

Zato se koristi jasan prompt-šablon koji vodi model kroz cijeli proces — od identifikacije elemenata do strukturiranja podataka.

OCR & Image Understanding Prompt Template

Analiziraj sliku koju dostavljam i primijeni sljedeće korake:

1. Identifikuj sve vizualne elemente (objekte, tekst, pozicioniranje).
2. Izvuci sav tekst sa slike koristeći OCR.
3. Konvertuj tekst u strukturirani format (tabela ili JSON).
4. Opiši ključne elemente: kompoziciju, osvjetljenje, kontrast i fokus.
5. Identifikuj potencijalno relevantne informacije za analitički ili poslovni kontekst.
6. Ako je potrebno, predloži poboljšanja slike (svjetlo, kontrast, jasnoća).
7. Obradi sliku u dodatnim verzijama ako je potrebna transformacija (crop, resize, rotation).

Ovaj prompt se koristi kao **osnovni alat** za sve vizualne analitičke zadatke i predstavlja ulaznu tačku za dalju automatizaciju.

4. Interna logika procesa

OCR i Image Understanding ne dešavaju se "odjednom".

Proces ima **dva jasno odvojena sloja**.

a) Ekstrakcija (OCR)

U ovoj fazi model: - prepoznaje znakove, brojeve i tekstualne blokove, - normalizuje tekst (format datuma, brojeva, jedinica), - čisti šum i greške nastale uslijed lošeg osvjetljenja ili kvaliteta slike.

Rezultat ove faze je **čist tekst**, ali bez značenja.

b) Interpretacija (Image Understanding)

U drugoj fazi model: - identifikuje objekte i simbole, - razumije raspored elemenata, - prepoznaje kontekst (dokument, grafikon, formular), - analizira kompoziciju, svjetlo i kontrast, - i povezuje elemente u smislenu cjelinu.

Tek kombinacijom ove dvije faze slika postaje **podatak**, a ne samo tekst.

5. Primjena u raznim kontekstima

a) Medicinska i farmaceutska dokumentacija

Ekstrakcija i strukturiranje podataka iz: - laboratorijskih nalaza, - dijagnostičkih izvještaja, - papirnih formulara, - fotografisanih recepata i terapijskih planova.

Ovdje OCR direktno omogućava digitalizaciju procesa koji su inače ručni i nepouzdani.

b) RAG sistemi

Vizualni sadržaj se pretvara u tekstualne embeddinge koje RAG sistemi mogu pretraživati, rangirati i koristiti u odgovaranju.

c) AI OS – vizualna memorija

Fotografije i screenshotovi postaju **indeksirane informacije**, a ne haotična galerija slika.

d) Automatizacija unosa podataka

OCR omogućava automatski unos podataka iz slika direktno u: - Notion baze, - Google Sheets, - CRM sisteme,

putem n8n workflow-a.

e) Analiza grafova i dijagrama

Model može očitati vrijednosti sa grafikona i pretvoriti ih u numeričke tabele spremne za dalju analizu.

6. Primjer OCR ekstrakcije

Ulazna slika sadrži sljedeći tekst: > LDL-Cholesterol > 3.4 mmol/L > Date: 12/02/2024 > Patient: Marko M.

Nakon OCR obrade i strukturiranja, rezultat izgleda ovako:

```
{
  "pacijent": "Marko M.",
  "datum": "2024-02-12",
  "parametar": "LDL-Cholesterol",
  "vrijednost": 3.4,
  "jedinica": "mmol/L"
}
```

Ovaj format se zatim može direktno koristiti: - u AI OS zdravstvenim modulima, - u RAG memoriji, - za analitiku kroz Code Interpreter, - u n8n automatizacijama za praćenje vrijednosti.

7. Sažetak lekcije

OCR i Image Understanding omogućavaju AI-ju da vizualne informacije pretvori u strukturirane podatke.

Time se otvara put ka: - digitalizaciji dokumenata, - automatizaciji unosa, - analitičkoj obradi slika, - i punoj integraciji vizualnog materijala u AI sisteme.

Kada se koriste standardizovani obrasci ekstrakcije, **svaka slika postaje potencijalni podatak**, a ne samo vizualni artefakt.

Tabular Reasoning & Table Operations

Kako AI razumije, čisti i "misli" u tabelama

1. Uvod u Tabular Reasoning

Tabular Reasoning predstavlja sposobnost AI modela da **razumije tabelarne podatke kao strukturu**, a ne kao običan tekst.

Za razliku od narativnog teksta, tabela ima: - kolone sa značenjem, - redove sa kontekstom, - implicitna pravila (tipovi podataka, odnosi, nedostajuće vrijednosti).

AI koji posjeduje tabularno rezonovanje ne "čita" tabelu — on je **analizira kao analitičar**.

Zahvaljujući ovom patternu, model može: - razumjeti šta kolone predstavljaju, - prepoznati greške i nedosljednosti, - izračunavati statistike, - transformisati strukturu, - spajati i filtrirati podatke, - izvoditi zaključke na osnovu brojeva, a ne teksta.

Ovo je ključna sposobnost za sve realne AI sisteme koji rade s podacima.

2. Zašto je Tabular Reasoning važan?

U stvarnom svijetu, **najvažniji podaci gotovo uvijek žive u tabelama**.

Primjeri: - Excel i Google Sheets fajlovi - medicinske i laboratorijske evidencije - CRM i ERP sistemi - CSV datasetovi - SEO scraper outputi - RAG metapodaci - n8n input/output strukture

Bez tabularnog rezonovanja, AI ostaje: - ograničen na tekst, - nesiguran u radu s brojkama, - sklon pogrešnim zaključcima.

Tabular Reasoning omogućava modelu da: - čisti podatke, - vrši agregaciju, - klasificira vrijednosti, - detektuje trendove i anomalije, - priprema podatke za dalju automatizaciju i analitiku.

Drugim riječima: **bez ove sposobnosti, AI nije upotrebljiv u ozbiljnom radu s podacima.**

3. Standardni prompt šablon za Tabular Reasoning

Da bi model radio pouzdano s tabelama, moraš mu jasno reći **da se ponaša kao analitičar**, a ne kao generator teksta.

Tabular Reasoning Prompt Template

Analiziraj sljedeću tabelu i izvrši ove zadatke:

1. Identifikuj sve kolone i njihove tipove podataka.
2. Provjeri da li postoje greške, nelogičnosti ili nedostajući podaci.
3. Izračunaj osnovne statistike (count, mean, median, min, max, standard deviation).
4. Identifikuj trendove, outliere i obrasce.
5. Kreiraj novu tabelu sa standardizovanim kolonama.
6. Transformiši podatke prema navedenim pravilima.
7. Izvezi finalnu tabelu kao CSV ili JSON.
8. Sažmi ključne uvide u maksimalno pet tačaka.

Ovaj šablon predstavlja **profesionalni standard za rad s tabelama** u AI okruženju.

4. Interna logika Tabular Reasoning procesa

AI ne obrađuje tabele "odjednom". Proces se odvija u jasno razdvojenim fazama.

a) Strukturna analiza

U prvoj fazi model identifikuje: - nazive kolona, - tipove podataka (tekst, broj, datum), - prazna polja, - razlike u formatima.

Ovo je faza u kojoj AI razumije **šta tabela zapravo jeste**.

b) Validacija i čišćenje

Zatim slijedi korekcija: - standardizacija datuma, - konverzija numeričkih vrijednosti, - uklanjanje duplikata, - označavanje ili uklanjanje nevažećih zapisa.

Bez ove faze, svaka dalja analiza je nepouzdana.

c) Transformacija i analitika

Nakon čišćenja, AI može vršiti: - agregacije, - filtriranje, - izračune, - klasifikaciju, - trend analizu, - pivotiranje (po potrebi).

Ovdje tabela postaje **izvor znanja**, a ne samo skup brojeva.

d) Prezentacija rezultata

Na kraju, rezultati se organizuju kao: - nova tabela, - CSV ili JSON fajl, - vizualni prikaz, - narativni sažetak za ljudsko razumijevanje.

5. Primjena u raznim kontekstima

a) Excel i poslovne analize

AI može zamijeniti: - ručne pivot tabele, - filtriranje i sortiranje, - osnovne statističke proračune, - generisanje izvještaja.

b) RAG dataset priprema

Prije embeddinga, podaci **moraju biti čisti, konzistentni i strukturirani**. Tabular Reasoning je ovdje obavezna faza.

c) SEO scraper i automatizacije

Scrapovani podaci se pretvaraju u uredne tabele spremne za n8n pipeline i dalju obradu.

d) AI OS – lični analitički sistem

Podaci o: - zdravlju, - treningu, - projektima, - navikama

mogu se analizirati, porediti i optimizirati kroz tabularni obrazac.

e) Medicinske i laboratorijske tabele

Vrijednosti poput LDL-a, HbA1c, EF-a ili CRP-a dobijaju smisao tek kada su: - standardizovane, - usporedive, - pravilno interpretirane.

6. Primjer Tabular Reasoning transformacije

Ulazna tabela:

Ime	Datum	LDL	EF
Marko	12/2/24	3.4	55%
Ana	2024-02-12	5.8	40%
N/A	2024/02/12	—	38%

AI obrada uključuje: - standardizaciju datuma (ISO), - uklanjanje nevažećih zapisa, - konverziju numeričkih tipova, - osnovnu statistiku.

Finalna tabela:

name	date	ldl	ef
Marko	2024-02-12	3.4	55
Ana	2024-02-12	5.8	40

Ključni uvidi: - jedan zapis je neupotrebljiv, - LDL pokazuje visoku varijabilnost, - EF vrijednosti ukazuju na različite stadije rizika, - podaci zahtijevaju konsolidaciju izvora.

7. Sažetak lekcije

Tabular Reasoning omogućava AI modelima da **pouzdano rade s tabelama**, transformišu podatke i izvode analitičke zaključke.

Ovaj pattern je neophodan za: - profesionalni rad s Excelom, - pripremu RAG sistema, - automatizacije, - medicinsku i poslovnu analitiku.

Kada AI pravilno razumije tabele, prestaje biti tekstualni alat — i postaje **analitički asistent**.

Data Visualization Pattern

Kako AI pretvara brojeve u razumljive obrasce

1. Uvod u vizualizaciju podataka

Data Visualization Pattern opisuje sposobnost AI modela da **pretvori numeričke i strukturalne podatke u grafičke prikaze** koji omogućavaju brzo, intuitivno i tačno razumijevanje informacija.

Vizualizacija nije ukras nad podacima. Ona je **most između sirovih brojeva i ljudskog razumijevanja**.

U kontekstu AI sistema, vizualizacija služi da: - pokaže trendove koji nisu očigledni u tabelama, - razotkrije anomalije i greške, - omogući poređenje kroz vrijeme, - pretvori analitiku u odluke.

Bez vizualizacije, podaci ostaju tehnički artefakt. Sa njom — postaju alat za razmišljanje.

2. Zašto je vizualizacija ključna?

Ljudski mozak ne razmišlja u tabelama. Razmišlja u **obrascima, odnosima i promjenama**.

Vizualna reprezentacija podataka omogućava: - trenutno uočavanje trendova, - lakše prepoznavanje outliera, - poređenje više varijabli odjednom, - razumijevanje dinamike kroz vrijeme, - sigurnije donošenje odluka.

U praksi se vrlo često dešava da: - tabela "izgleda u redu", - ali graf odmah otkrije problem, anomaliju ili skriveni obrazac.

Zato je vizualizacija sastavni dio **svake ozbiljne analitike**, a ne završni korak.

3. Standardni prompt šablon za Data Visualization

Ako želiš da AI pravilno vizualizira podatke, moraš ga voditi **kao analitičara**, ne kao dizajnera.

Data Visualization Prompt Template

Na osnovu dostavljenog dataset-a:

1. Identifikuj sve numeričke i kategorijske kolone.
2. Predloži optimalne tipove grafikona za analizu podataka.
3. Kreiraj sljedeće grafičke prikaze:
   - line chart (trend kroz vrijeme),
   - histogram (raspodjela vrijednosti),
   - scatter plot (odnos dvije numeričke varijable),
   - bar chart (poređenje kategorija).
4. Svaki graf opremi jasnim naslovom, oznakama osa i legendom.
5. Izvuci 3–5 ključnih uvida na osnovu vizualizacije.
6. Izvezi grafove kao PNG (ili SVG ako je potrebno).
7. Upozori ako dataset nije prikladan za neku vrstu grafikona.

Ovaj šablon omogućava **standardizovanu i pouzdanu vizualnu analizu**.

4. Interna logika Data Visualization procesa

AI ne crta grafikone nasumično. Proces se odvija u jasno razdvojenim fazama.

a) Analiza dataset-a

U prvoj fazi model utvrđuje: - koje kolone su numeričke, - koje su kategorijske, - da li postoje nedostajući ili nelogični podaci, - da li je dataset vremenska serija ili statički presjek.

Ova faza sprječava pogrešan izbor grafikona.

b) Odabir grafičkog prikaza

Na osnovu strukture podataka, AI bira odgovarajući vizualni oblik: - **line chart** → promjene kroz vrijeme - **scatter plot** → odnos dvije numeričke varijable - **histogram** → raspodjela vrijednosti - **bar chart** → poređenje kategorija - **boxplot** → detekcija outliera

Pogrešan graf = pogrešna interpretacija.

c) Interpretacija i izvještavanje

Vizualizacija nije kraj procesa.

Nakon crtanja grafikona, AI mora: - interpretirati ono što se vidi, - povezati obrasce s kontekstom, - izdvojiti uvide, - upozoriti na ograničenja podataka.

Bez ove faze, graf ostaje samo slika.

5. Primjena u raznim kontekstima

a) Poslovna analitika

Vizualizacija: - KPI trendova, - sezonalnosti, - rasta ili pada performansi, - segmentacije klijenata.

b) Medicinski i zdravstveni podaci

Praćenje: - LDL-a, - glukoze, - EF-a, - CRP-a, - krvnog pritiska,

kroz vrijeme — gdje je **trend važniji od jedne vrijednosti**.

c) RAG evaluacija

Grafički prikaz: - embedding distribucija, - gustoće vektora, - sličnosti upita i dokumenata.

Vizualizacija ovdje često otkriva lošu indeksaciju ili slab embedding.

d) AI OS – lični sistemi

Praćenje: - produktivnosti, - navika, - trening performansi, - zdravstvenih metrika.

Vizualni prikaz omogućava samoregulaciju, ne samo evidenciju.

e) SEO i scraping sistemi

Analiza: - SERP pozicija, - keyword trendova, - autoriteta stranica, - promjena kroz vrijeme.

f) n8n automatizacije

Vizualna evaluacija: - ulaznih tokova, - izlaznih rezultata, - performansi pipeline-a.

6. Primjer Data Visualization primjene

Dataset:

datum	LDL	HDL	trigliceridi
2024-01-01	3.4	1.2	1.8
2024-02-01	3.1	1.3	1.7
2024-03-01	2.9	1.4	1.6

AI generiše: - **line chart** → pad LDL-a kroz vrijeme - **bar chart** → rast HDL-a po mjesecima - **scatter plot** → odnos LDL i triglicerida - **histogram** → raspodjela LDL vrijednosti

Ključni uvidi: - LDL pokazuje stabilan trend opadanja, - HDL ima pozitivan uzlazni trend, - postoji blaga pozitivna korelacija LDL–trigliceridi, - nema ekstremnih outliera u dataset-u.

7. Sažetak lekcije

Data Visualization Pattern omogućava AI modelima da **pretvore sirove podatke u razumljive vizualne obrasce**.

Vizualizacija: - ubrzava razumijevanje, - smanjuje greške u interpretaciji, - povećava kvalitet odluka.

U profesionalnim AI sistemima, vizualizacija nije dodatak — ona je **ključni dio misaonog procesa**.

Tool Usage

Working with Tools: The Code Interpreter Paradigm

Uvod

Kad ljudi prvi put koriste Code Interpreter, često imaju pogrešan mentalni model. Misle da je to "AI koji zna Excel" ili "AI koji zna Python".

To nije tačno.

Code Interpreter nije znanje. On je **workflow**.

Drugim riječima, AI ne postaje pametniji — on dobija **proces** kroz koji može da radi stvarne stvari: da učita podatke, obradi ih, provjeri ih i tek onda donese zaključak.

Ako ne razumiješ taj workflow, Code Interpreter djeluje kao crna kutija. Ako ga razumiješ, postaje **najpouzdaniji alat u cijelom sistemu**.

Šta znači "workflow" u kontekstu Code Interpretera

Workflow znači da AI **ne radi sve odjednom**.

On prolazi kroz jasno razdvojene faze: - prima ulaz (fajl ili podatke) - razumije strukturu - izvršava operacije - provjerava rezultate - tek onda komunicira zaključak

Ovo je ključno: > Zaključak dolazi zadnji, ne prvi.

To je suprotno od klasičnog chata, gdje AI često krene od odgovora pa ga "popunjava".

Standardni Code Interpreter Workflow (korak po korak)

Iako se u interfejsu sve dešava "u jednom razgovoru", u pozadini se uvijek dešava isti niz koraka.

1. Učitavanje (Load)

AI prvo: - učitava fajl (Excel, CSV, image, PDF…) - identifikuje tip i strukturu - provjerava da li su podaci čitljivi

Ako fajl nije jasan ili je pogrešnog formata — workflow staje. Ovdje AI još **ništa ne analizira**.

2. Razumijevanje strukture (Inspect)

Zatim AI: - gleda kolone, redove, tipove podataka - prepoznaje praznine, nelogičnosti, formate - pravi internu mapu podataka

Ovo je faza gdje dobar AI: > prvo pogleda tabelu, pa tek onda računa

Ako preskoči ovu fazu — rezultati su nepouzdani.

3. Obrada (Process)

Tek sada AI: - računa prosjeke, trendove, razlike - filtrira podatke - poredi scenarije - transformiše podatke po zadatku

Ovdje se izvršava stvarni kod. Nema nagađanja. Samo operacije.

4. Provjera (Verify)

Ovo je najvažnija, ali često zanemarena faza.

Dobar Code Interpreter workflow uključuje: - provjeru da li rezultati imaju smisla - poređenje sa očekivanjima - provjeru grešaka (npr. outlieri, nelogične vrijednosti)

Ovdje se često kombinuje sa **CVP (Cognitive Verifier Pattern)**.

5. Izlaz (Output)

Tek na kraju AI: - generiše tekstualni sažetak - pravi tabelu ili grafikon - objašnjava šta brojevi znače

Bitno: > Output je objašnjenje rezultata, ne zamjena za rezultat.

Zašto je razumijevanje workflowa ključno

Ako ne razumiješ ovaj tok: - vjeruješ AI-ju "na riječ" - ne znaš gdje može pogriješiti - ne znaš kako da ga ispraviš

Ako razumiješ workflow: - znaš **šta da pitaš** - znaš **kada da staneš** - znaš **kada da tražiš provjeru**

To je razlika između: > "AI mi je rekao"

i

> "AI mi je izračunao, provjerio i objasnio"

Kako ti utičeš na workflow kroz prompt

Ti kao korisnik **ne pišeš kod**, ali pišeš **instrukcije koje oblikuju workflow**.

Primjer dobrog workflow prompta:

```
Use Code Interpreter.

1. Load the attached Excel file.
2. Inspect the structure and explain what the data represents.
3. Analyze monthly trends.
4. Verify the results and highlight any anomalies.
5. Present a clear summary with one chart.
```

Ovim promptom: - sprečavaš preskakanje faza - prisiljavaš AI na redoslijed - povećavaš pouzdanost rezultata

Kratka analogija

Zamisli laboratoriju. Bez workflowa — neko ti samo kaže rezultat. Sa workflowom — vidiš: - uzorak - mjerenje - kontrolu - zaključak

Code Interpreter workflow je **laboratorijski protokol za podatke**.

Sažetak

Code Interpreter nije magija, nego proces. Razumijevanjem njegovog workflowa — od učitavanja do provjere — dobijaš potpunu kontrolu nad rezultatima. Kada znaš kojim redom AI radi, znaš i kako da ga vodiš, ispraviš i koristiš bez slijepog povjerenja.

Toolformer Pattern: When Models Decide to Use Tools

1. Uvod u Toolformer Pattern

Toolformer je napredni prompt pattern u kojem AI model: - samostalno bira *koji alat treba koristiti*, - odlučuje *kada je alat potreban*, - generiše *ispravan input* za taj alat, - koristi rezultat alata, - spaja sve u finalni odgovor.

Drugim riječima: **AI prestaje biti samo generator teksta. Postaje orkestrator alata.**

Ovo je ključ za: - AI agente - OpenAI Assistants - n8n AI workflows - automatske API pozive - pametne RAG sisteme - AI OS - sve aplikacije koje žele koristiti realne podatke

2. Zašto je Toolformer Pattern važan?

Bez ovog patterna: - AI daje samo tekst - ne zna raditi s alatima - ne zna pozivati API - ne zna raditi realne akcije - ne zna koristiti funkcije i komande - ne zna izvšiti logiku izvan teksta

Sa Toolformer patternom: - AI može pozivati kalkulatore - čitati baze podataka - raspoređivati zadatke - koristiti n8n webhooke - izvršavati kod - pozivati tvoje vlastite API-je - raditi kao backend agent

Ovo je ogroman skok u sposobnostima modela.

3. Standardni Toolformer Prompt Template

Kada korisnik postavi zadatak: 1. Analiziraj da li je potrebno koristiti alat 2. Ako jeste — izaberi odgovarajući alat 3. Generiši input za alat (npr. JSON ili komandu) 4. Pozovi alat i sačekaj rezultat 5. Na osnovu rezultata kreiraj finalni odgovor 6. Ako alat *nije* potreban — objasni zašto

Output strukturiran: - Tool Decision - Tool Input - Tool Output - Final Answer

Ovaj template čini AI *operativnim sistemom*, a ne samo generatorom.

4. Interna logika Toolformer procesa

Toolformer radi kroz pet faza:

a) Task Classification

Da li je zadatak: - matematički? - podatkovni? - API integracija? - dohvat eksternih resursa? - izvršavanje koda? - pretraga baze?

Ako "da" → koristi alat.

b) Tool Selection

AI bira: - kalkulator - API funkciju - n8n webhook - SQL funkciju - File parser - Data transformation skriptu - RAG retrieval

c) Tool Invocation

Model generiše *ispravan format* za alat, npr:

```
{
  "action": "getWeather",
  "location": "Sarajevo"
}
```

ili

```
POST https://n8n.yourdomain.com/webhook/xyz
```

d) Tool Output Interpretation

Rezultat alata → AI obrađuje i interpretira.

e) Final Synthesis

AI kombinuje: - originalni zadatak - output alata - vlastitu logiku

i daje finalni odgovor.

5. Primjena u realnim projektima

a) AI OS

Toolformer omogućava OS-u da: - pozove tvoje API-je - kreira zapise - dohvaća zapise - upravlja navikama - automatski generiše dnevni fokus - poziva n8n workflowe u realnom vremenu

b) n8n agenti

AI u n8n-u može: - odlučiti kada treba koristiti HTTP request - generisati JSON tijela - pozvati webhook - validirati output - kontrolisati tok workflowa

c) OpenAI Assistants API

Toolformer logika određuje: - koji je alat relevantan - kako ga pozvati - kako rukovati pogreškama - kako nastaviti agent ciklus

d) Kodiranje i debugging

AI bira: - code executor - file interpreter - refactoring alat

e) RAG sistemi

AI bira: - retrieval alat - query format - broj dokumenata - način sažimanja

f) Biznis aplikacije

AI automatski poziva: - CRM API - Notion API - Google Sheets API - plaćanja - upise - pretrage

6. Primjer Toolformer procesa

Zadatak: "Koliko je 27,4% od 640?"

AI reasoning: Za ovaj zadatak precizan odgovor zahtijeva alat (calculator function).

Tool Decision:

Koristi kalkulator.

Tool Input:

```
{
  "action": "calculate",
  "expression": "0.274 * 640"
}
```

Tool Output:

175.36

Final Answer:

27,4% od 640 iznosi **175.36**. → Precizno, stabilno, bez greške.

7. Sažetak lekcije

Toolformer Pattern omogućava AI modelu da SAM bira i koristi alate potrebne za rješavanje zadatka.

Time AI: - prestaje biti samo generator teksta, - postaje orkestrator alata, - koristi API-je, funkcije i workflowe, - povećava tačnost, - postaje sposoban za realne operacije i automatizacije.

Ovo je osnova modernog AI agent dizajna i sve napredne AI automatizacije.

Quality & Validation

Evaluating AI Output: Validation, Errors, and Trust

Kako AI prestaje da "zvuči uvjerljivo" i počinje da bude tačan

1. Uvod: Zašto evaluacija mora biti eksplicitna

Jedan od najvećih problema u radu s AI-jem nije to što model griješi. Problem je što **griješi uvjerljivo**.

Bez jasnog obrasca za evaluaciju, AI može: - dati netačne informacije, - propustiti ključne greške, - izgraditi zaključke na pogrešnim pretpostavkama, - "zvučati pametno", a biti pogrešan.

Evaluation / Validation / Error Detection Pattern uvodi **namjernu provjeru kvaliteta** u AI output.

Ovaj obrazac uči AI da: - **procijeni vlastiti odgovor**, - **provjeri tačnost**, - **detektuje greške**, - **označi nesigurnosti**, - i po potrebi — **zaustavi isporuku**.

Drugim riječima: AI više nije samo generator, nego **kontrolisani sistem**.

2. Razlika između generisanja i evaluacije

Važno je razumjeti jednu ključnu razliku: - **Generisanje** = "napravi odgovor" - **Evaluacija** = "da li je ovaj odgovor dobar, tačan i siguran?"

Većina korisnika **nikad ne traži evaluaciju**. Zato AI nema razlog da je radi.

Ovaj pattern uvodi **drugu misaonu ulogu**: > nakon što završi zadatak — AI mora stati i provjeriti sam sebe.

3. Tri nivoa provjere (Evaluation Stack)

Evaluation Pattern se sastoji od tri sloja.

a) Validacija tačnosti (Factual Validation)

Model provjerava: - da li su tvrdnje činjenično tačne, - da li postoje neprovjerene pretpostavke, - da li je koristio nagađanje.

Ako informacija nije sigurna → mora biti označena.

b) Logička validacija (Logical Consistency)

Model provjerava: - da li zaključci slijede iz argumenata, - da li postoje kontradikcije, - da li su koraci preskočeni.

Ovdje se često otkrivaju "tihe greške" koje zvuče razumno.

c) Kontekstualna validacija (Context Fit)

Model provjerava: - da li odgovor odgovara *ovom* korisniku, - *ovom* cilju, - *ovom* domenu (medicina ≠ marketing ≠ edukacija).

Odgovor može biti tačan — ali **neprimjenjiv**.

4. Standardni prompt šablon za Evaluation Pattern

Evaluation / Validation Prompt Template

> Nakon što završiš odgovor, izvrši evaluaciju:
>
> 1. Provjeri činjeničnu tačnost svih tvrdnji.
> 2. Identifikuj sve pretpostavke ili nesigurne dijelove.
> 3. Provjeri logičku konzistentnost odgovora.
> 4. Procijeni da li je odgovor u skladu s kontekstom i ciljem.
> 5. Označi potencijalne greške ili slabosti.
> 6. Ako je potrebno, predloži ispravke ili upozori korisnika.
>
> Na kraju, jasno navedi: - da li je odgovor **siguran za upotrebu**, - ili zahtijeva dodatnu provjeru.

Ovaj šablon pretvara AI u **kontrolisani sistem, ne generator teksta**.

5. Interna logika Error Detection-a

Error Detection nije isto što i poboljšanje (WRI). - **WRI** pita: *"Kako da ovo bude bolje?"* - **Evaluation Pattern** pita: *"Da li je ovo uopšte ispravno?"*

Model mora imati dozvolu da kaže: - "ovdje nisam siguran", - "ovdje postoji rizik greške", - "ovo treba dodatnu validaciju".

Bez toga — AI uvijek ide do kraja, čak i kad ne bi trebao.

6. Primjeri gdje je ovaj pattern kritičan

a) Medicinski i farmaceutski kontekst

Pogrešna doza, pogrešna interpretacija nalaza ili guideline-a **nije prihvatljiva greška**. Evaluation Pattern ovdje djeluje kao "crvena zastavica".

b) RAG sistemi

RAG odgovori moraju biti: - u skladu sa izvorima, - bez halucinacija, - jasno označeni ako izvor nije pronađen.

c) Automatizacije i n8n workflow-i

Greška u jednom koraku može: - prekinuti pipeline, - poslati pogrešne podatke, - napraviti sistemsku grešku.

d) Poslovne i strateške odluke

Uvjerljiv, ali pogrešan savjet je **najskuplja vrsta greške**.

7. Sažetak lekcije

Evaluation / Validation / Error Detection Pattern uvodi **odgovornost** u rad s AI-jem.

Ovaj obrazac: - smanjuje rizik, - sprječava tihe greške, - povećava profesionalni standard, - omogućava sigurno korištenje AI sistema.

Ako WRI uči AI da bude bolji autor, **Evaluation Pattern uči AI da bude odgovoran sistem.**

Bez njega, nijedan AI workflow nije kompletan.

Self-Correction Loops: How Models Fix Their Own Mistakes

1. Uvod u Self-Correction Loop

Self-Correction Loop je obrazac u kojem AI ne isporučuje odgovor kao "gotov proizvod" u jednom prolazu, već radi kroz **kontrolisanu iterativnu petlju**:

1. generiše odgovor,
2. analizira vlastiti output,
3. identifikuje greške i slabosti,
4. pravi plan poboljšanja,
5. isporučuje unaprijeđenu verziju,
6. po potrebi ponavlja proces.

Važno je naglasiti: **ovdje se model ne trenira**.

Model samo *simulira proces samoevaluacije i uredničkog rada* kroz eksplicitne instrukcije.

Ovaj pattern se koristi kada je cilj **maksimalno kvalitetan, pouzdan i profesionalan output**, posebno kod kompleksnih ili osjetljivih zadataka.

2. Zašto je Self-Correction Loop važan

Bez samokorektivne petlje, AI modeli često: - ostave skrivene logičke greške, - preskoče važne elemente zadatka, - generišu površne ili preopćenite odgovore, - naruše format (posebno kod JSON-a, tabela, koda), - propuste vlastite halucinacije.

Self-Correction Loop uvodi **namjernu pauzu između generisanja i isporuke**.

Model dobija: - analitičku distancu, - drugu priliku za razmišljanje, - jasne kriterije kvaliteta, - kontrolu nad vlastitim greškama.

U praksi, ovo je razlika između: > "odgovor koji zvuči dobro"

i

> "odgovor koji je provjeren i pouzdan".

3. Standardni Self-Correction Loop Template

Korak 1 — Draft Output > "Generiši prvi nacrt odgovora. Ne pokušavaj da ga unaprijed optimizuješ."

Korak 2 — Error Review > "Analiziraj svoj odgovor i identifikuj: > – logičke greške > – netačne ili neprecizne tvrdnje > – preskočene elemente > – nejasne ili slabo objašnjene dijelove > – dijelove koji se mogu poboljšati"

Korak 3 — Correction Plan > "Napravi jasan i konkretan plan poboljšanja u tačkama."

Korak 4 — Improved Output > "Generiši novu verziju odgovora primjenjujući sve tačke iz plana."

Korak 5 — Validation Pass > "Provjeri finalnu verziju i potvrdi da ispunjava sve zahtjeve zadatka."

Ovaj template zatvara model u **kontrolisanu petlju samokontrole**.

4. Interna logika Self-Correction Loop-a

Self-Correction Loop funkcioniše kroz tri ključna mehanizma:

a) Metakognitivni sloj

Model ne razmišlja samo *o problemu*, već i *o kvalitetu vlastitog razmišljanja*.

b) Eksplicitna provjera kvaliteta

Umjesto implicitnog "pretpostavljam da je dobro", model mora dokazati da je odgovor dobar.

c) Iterativna optimizacija

Svaka naredna verzija mora biti **bolja, jasnija i preciznija** od prethodne.

Ovim pristupom model se ponaša kao **urednik i revizor**, a ne samo generator teksta.

5. Primjena u stvarnim projektima

Self-Correction Loop se koristi u svim sistemima gdje greška ima cijenu.

Primjeri: - **RAG sistemi** – korekcija pogrešnih citata i loših izvora - **AI OS** – pouzdaniji savjeti i evaluacije - **n8n & API outputi** – sprječavanje slomljenog JSON-a i formatnih grešaka - **Pisanje dokumenata i knjiga** – profesionalna jasnoća i konzistentnost - **Medicinski sadržaji** – terminološka i klinička preciznost - **SEO i marketing** – fina optimizacija finalnog teksta

6. Primjer Self-Correction Loop primjene

Zadatak: "Napiši objašnjenje šta je krvni pritisak."

Korak 1 — Draft

"Krvni pritisak je sila kojom krv djeluje na zidove arterija. Visok krvni pritisak je kada je ta sila povećana."

(površan i nedovoljan odgovor)

Korak 2 — Error Review

Model uočava: - nema razlike između sistoličkog i dijastoličkog pritiska - nema mjernih jedinica - nema kliničkog konteksta - odgovor je prekratak

Korak 3 — Correction Plan

1. Definisati sistolički i dijastolički pritisak
2. Navesti mjerne jedinice (mmHg)
3. Objasniti klinički značaj
4. Dodati kontekst rizika

Korak 4 — Improved Output

"Krvni pritisak predstavlja silu kojom krv pritiska zidove arterija. Mjeri se u milimetrima živinog stuba (mmHg) i sastoji se od dvije vrijednosti: sistoličkog pritiska, koji označava pritisak tokom kontrakcije srca, i dijastoličkog pritiska, koji predstavlja pritisak između otkucaja. Dugotrajno povišen krvni pritisak značajno povećava rizik od moždanog udara, infarkta miokarda i oštećenja ciljnih organa."

Korak 5 — Validation

- tačno
- jasno
- medicinski ispravno
- kontekstualno kompletno

7. Sažetak lekcije

Self-Correction Loop Pattern omogućava AI modelu da: - procjenjuje vlastiti rad, - identifikuje i ispravlja greške, - iterativno poboljšava kvalitet, - isporučuje pouzdane i profesionalne outpute.

Ovaj pattern je temelj ozbiljnog rada sa AI-jem u: - tehničkim sistemima, - medicinskim i analitičkim tekstovima, - poslovnim izvještajima, - AI agentima, - višefaznim automatizacijama.

Bez samokorekcije, složeni AI sistemi postaju nepouzdani. Sa njom — prelaziš u zonu stvarne ekspertize.

Self-Consistency: Stability Across Multiple Reasoning Paths

1. Uvod u Self-Consistency Pattern

Self-Consistency je tehnika u kojoj AI: 1. generiše više različitih mogućih odgovora, 2. upoređuje ih međusobno, 3. analizira razlike i logiku, 4. bira najtačniji i najrazumniji, 5. daje finalnu verziju kao "konsenzus".

Ovaj pristup dramatično povećava: - tačnost zaključaka, - stabilnost, - logičku jasnoću, - pouzdanost u teškim zadacima.

Self-Consistency je posebno moćan u situacijama gdje čak i najbolji modeli mogu pogriješiti ako daju samo jedan odgovor.

2. Zašto je Self-Consistency važan?

Jer AI modeli često: - daju prvi odgovor koji im padne na pamet, - ne razmatraju alternativne mogućnosti, - pogrešno fokusiraju pažnju, - površno riješe kompleksan problem.

Self-Consistency to ispravlja tako da AI: - radi više paralelnih razmišljanja, - testira više strategija rješavanja, - koristi "grupnu inteligenciju", - pravi prosjek ili bira najbolji odgovor.

Ovo je kao da dobiješ "AI tim koji raspravlja i daje zaključak".

3. Standardni Self-Consistency Template

Uradi sljedeće: 1. Generiši **3–5 različitih odgovora** na zadatak, svaki s drugačijom logikom. 2. Uporedi sve generisane odgovore. 3. Identifikuj: - najtačniji element - najkonsistentniju logiku - najbolju metodologiju 4. Kreiraj finalni, **konzistentni odgovor** koji kombinuje najbolje iz svih verzija. 5. Prikaži samo finalni odgovor.

Output Format: - Option A - Option B - Option C - Comparative Analysis - Final Synthesized Answer

Ovaj template je apsolutno top-tier alat za profesionalni rad.

4. Interna logika Self-Consistency procesa

Proces uključuje tri misaona sloja:

a) Divergent Thinking (široko razmišljanje)

Model namjerno generiše različite pristupe, ponekad i kontrastne.

b) Convergent Reasoning (sužavanje logike)

Model poredi pristupe i traži: - gdje se poklapaju, - gdje su najlogičniji, - gdje su greške, - gdje su jaka objašnjenja.

c) Synthesis (konsenzus)

Model kreira finalni odgovor koji: - uzima najbolje elemente - ima najjaču logiku - ispravlja greške - odgovara najbliže istini

To je kao "kognitivna komisija" unutar modela.

5. Primjena u realnim projektima

a) AI OS — donošenje odluka

AI generiše više mogućih interpretacija tvog dana → zatim bira najtačniju.

b) Medicinske analize

AI može kreirati više interpretacija nalaza → zatim sintetizirati najprecizniju i najtačniju.

c) Kodiranje i debugging

AI daje više mogućih rješenja → zatim bira najstabilnije.

d) Kompleksni izračuni i matematika

Posebno važno jer modeli često pogriješe u prvoj verziji. Self-Consistency daje: - stabilnije, - tačnije, - provjerene rezultate.

e) RAG sistemi

Kada je retrieval nejasan → Self-Consistency poboljšava finalni output birajući najbolje objašnjenje.

f) Biznis strategija i analiza

AI kreira više scenarija → zatim bira onaj koji je najracionalniji.

g) Pisanje knjige

AI može: - kreirati 3 verzije poglavlja - pokupiti najbolje dijelove - dati finalnu, savršeno optimizovanu

6. Primjer Self-Consistency u praksi

Zadatak: "Da li je bolji pristup za monetizaciju AI OS-a: prodavati botove, workflowe ili edukaciju?"

Option A: Prodavati botove — skalabilno, automatski prihod.

Option B: Prodavati workflowe — brza monetizacija, ali manje skalabilno.

Option C: Prodavati edukaciju — najveći brand potencijal i dugoročni rast.

Comparative Analysis: - Botovi: najbolji long-term proizvod - Workflowi: najbolji short-term cash-flow - Edukacija: najbolja platforma za brand i autoritet

Final Synthesized Answer: Najefikasnija strategija je kombinacija: 1. edukacija → gradi brand, 2. botovi → skalabilan proizvod, 3. workflowi → brzi prihodi.

→ 3 verzije → analiza → savršen finalni odgovor.

7. Sažetak lekcije

Self-Consistency Pattern je tehnika u kojoj AI generiše više različitih odgovora i zatim bira najlogičniji i najtačniji.

Ovaj pristup omogućava: - bolju tačnost, - stabilnije zaključke, - eliminaciju površnih odgovora, - profesionalni kvalitet, - inteligentno donošenje odluka.

Self-Consistency predstavlja vrhunac naprednog AI razmišljanja — i ključnu tehniku za izgradnju pouzdanih AI sistema.

System Integration

Managing Context: Working Within the Context Window (CWO)

1. Uvod u Context Window Optimization (CWO)

Context Window predstavlja *koliko informacija model može držati u glavi odjednom*. - GPT-ovi imaju određeni broj tokena. - Ako ubaciš previše teksta → važne informacije se izgube. - Ako ubaciš premalo → model nema dovoljno konteksta.

CWO (Context Window Optimization) je tehnika optimizacije sadržaja koji model prima kako bi radio: - pametnije, - tačnije, - stabilnije, - brže.

Bez CWO → AI se zbuni, halucinira, gubi nit, miješa teme, promašuje suštinu. Sa CWO → AI radi kao hirurg — precizno, fokusirano, efikasno.

2. Zašto je CWO važan?

Jer modeli NE RAZUMIJU sav tekst jednako.

Oni rade prema prioritetu: 1. zadnje što pročitaju → NAJVAŽNIJE 2. sredina → veoma lako gubi važnost 3. početak → gubi se na dugim inputima

CWO rješava: - gubitak fokusa AI-ja, - brisanje važnih detalja, - kontradiktorne odgovore, - halucinacije kod dugih promptova, - probleme sa dužinom kod RAG sistema.

Ako praviš: - AI asistenta - AI OS - RAG - Workflowe u n8n - ChatGPT Custom GPT - knjižni sistem u Notionu/Obsidianu - CI/CD dokumentacijski AI

→ ovo je must-have znanje.

3. Standardni CWO Prompt Template

U svim budućim odgovorima koristi sljedeći pristup:

1. **Identifikuj ključne informacije** (najvažnijih 10–20%).
2. **Sažmi nebitne informacije** u kratke linije.
3. **Strukturiraj sadržaj** (sekcije, liste, tabele).
4. **Prioritiziraj kritične elemente** na početku prompta.
5. **Ukloni redundantne dijelove**.
6. **Dodaj short-memory recap** za dugotrajne procese.
7. **Razdvoji prompt u logičke blokove.**

Ovo omogućava modelu da razumije *tačno ono što treba*, a ne sve što si mu poslao.

4. Interna logika CWO procesa

CWO se zasniva na tri glavna mehanizma:

a) Hierarchical Relevance (hijerarhija važnosti)

Model rangira informacije: 1. najvažnije → imperativ 2. srednje → kontekst 3. nebitne → ignorira

Ti kao autor moraš kontrolisati ovaj rang.

b) Compression (kompresija)

Duge sekcije se skraćuju u: - bullet list - sažetak - ključne stavke - numeričke vrijednosti - ultrasažeti format

c) Chunking (segmentacija)

Informacije se dijele u male "pakete" koje model može pratiti. Svaka ozbiljna AI arhitektura mora koristiti chunking.

5. Kada se koristi CWO?

Obavezno u sljedećim situacijama: - RAG sistemi (dokumenti + AI) - n8n workflowi (dugi JSON-i, tehnički inputi) - AI OS daily logs (mnogo teksta svaki dan) - veliki Notion/Obsidian fajlovi - knjige, eBookovi, dugi eseji - ChatGPT custom GPT memorija - kod debug output (predugački logovi) - svi sistemi gdje AI treba razumjeti historiju događaja

Ako AI daje kontradiktorne ili zbunjene odgovore → kriv je loš CWO.

6. Primjer CWO u praksi

Zadatak: "Daj mi strategiju za AI OS zasnovanu na mojim 20 prethodnih dnevnih zapisa."

Bez CWO → AI će: - izgubiti fokus, - pogrešno grupisati informacije, - ignorisati polovinu zapisa.

Sa CWO → AI će: 1. Sažeti svaki zapis na 3–5 ključnih stavki 2. Identificirati obrasce u ponašanju 3. Rangirati probleme po prioritetu 4. Donijeti strategiju u 5 tačaka 5. Napraviti akcijski plan

CWO mijenja sve.

7. CWO u RAG sistemima (ultra važno)

Ako uneseš: - previše teksta → model ignorira pola - premalo teksta → model nema dovoljno informacija - loše odabrane fragmente → model griješi

CWO omogućava: - pravilno biranje chunkova - pravilnu dužinu chunkova - pravilno spajanje konteksta - optimalno korištenje embeddinga

Najbolji RAG-ovi nisu oni sa najviše podataka — nego oni sa NAJBOLJE OPTIMIZOVANIM kontekstom.

8. Sažetak lekcije

Context Window Optimization (CWO) je tehnika upravljanja i optimizacije količine informacija koje AI model može obraditi istovremeno.

Omogućava: - bolju tačnost, - manje halucinacija, - stabilnije odgovore, - efikasnije RAG sisteme, - pametnije AI asistente.

CWO je ključno znanje u modernom AI dizajnu i jedan od temelja svake ozbiljne primjene.

Bez CWO — AI je nepouzdan. Sa CWO — AI je profesionalac.

Multi-Modal Reasoning: Combining Text, Images, and Data

1. Uvod u Multi-Modal Prompting

Multi-modal prompting je tehnika u kojoj AI model prima **više vrsta inputa istovremeno**, kao što su: - tekst - slike - audio - video - PDF dokumenti - tabele - screenshotovi - grafovi - skice - rukopis

Umjesto "samo teksta", AI postaje **sistem koji vidi, sluša, analizira i razumije složene podatke**.

Ovo otvara nove mogućnosti u: - analitici - edukaciji - medicini - OCR-u - n8n automatizacijama - vizualnim RAG sistemima - izgradnji AI OS funkcija - pripremi knjiga, kvizova, prezentacija - izradi portfolio projekata

2. Zašto je Multi-Modal Prompting važan?

Jer tekst je samo jedan dio stvarnog svijeta.

Sa multimodalnim pristupom AI može: - čitati grafove - analizirati laboratorijske nalaze - segmentirati objekte na slici - opisati scenu - raditi tabelarnu analizu iz screenshotova - pretvoriti PDF u strukturirane podatke - razumjeti rukom pisane bilješke - parsirati slike recepata, projekata, računa - generisati nove slike iz starih

Multimodal je *druga dimenzija AI-ja* — gdje postaje stvarni inteligentni asistent.

3. Standardni Multi-Modal Prompt Template

1. Opis ulaznih podataka: - vrsta slike/video/audio - šta se nalazi na njima - šta je cilj analize

2. Zadatak: - jasno navedi šta želiš da model uradi

3. Ograničenja: - stil odgovora - format outputa - preciznost - fokus na ključne elemente

4. Output Format: - JSON, tabela, bullet points, analiza, preporuka

5. Posebne instrukcije: - ignorisati nebitne vizualne elemente - fokusirati se na tekst na slici - pretvoriti sliku u strukturirane podatke - napraviti rezime ili kalkulaciju

4. Interna logika multimodalnog procesiranja

Model razmišlja kroz četiri koraka:

a) Vision Encoding

Slika ili video se pretvara u vektore koje model može "razumjeti".

b) Semantic Linking

Vizualni elementi se povezuju sa tekstualnim objašnjenjima: - objekti - pozicije - odnosi - tekst na slici - brojke - obrasci

c) Task Interpretation

Model određuje šta zapravo treba uraditi: - analiza? - pretvaranje u tabelu? - dizajn? - zaključak? - opis?

d) Output Generation

Model kreira finalni rezultat u traženom formatu.

5. Primjena u stvarnim projektima

a) AI OS — analiza dnevnih screenshotova

Tvoj OS može uzeti sliku: - ekrana - bilježnice - tabele - kalendara - jelovnika - trening aplikacije

i pretvoriti je u strukturirane podatke.

b) Medicina i farmacija

AI može: - čitati laboratorijske nalaze - interpretirati EKG slike (uz ograničenja) - rezimirati PDF smjernice - analizirati tabelarne podatke sa screenshotova

c) RAG sistemi sa slikama

Integracija vizualnih dokumenata u baze znanja: - PDF grafikoni - prezentacije - slike sa medicinskih konferencija - edukacijski materijali

d) n8n workflow

n8n može: - primiti sliku preko webhoka - poslati je AI-ju - dobiti strukturirani JSON nazad - spremiti ga u Notion ili Google Sheets

Savršen za: - scraping screenshotova - analizu računa - inventure - marketing pipelinee

e) ChatGPT + Code Interpreter

Multi-modal = vizualna analiza + kod: - ekstrakcija teksta sa slike - detekcija objekata - generisanje dataset-a - analiza grafikona - prepoznavanje obrasca

6. Primjer multimodalnog prompta

Zadatak: "Analiziraj laboratorijski nalaz koji je fotografisan telefonom."

Prompt: "Na osnovu priložene slike laboratorijskog nalaza: 1. Ekstrahuj sve vrijednosti u tabelarnom formatu 2. Usporedi ih sa referentnim rasponom 3. Identificiraj odstupanja 4. Napiši kliničku interpretaciju stručnim tonom 5. Output format: JSON + rezime"

→ model generiše: - tabelu - interpretaciju - preporuku - uredan JSON

Ovo je nivo profesionalne primjene.

7. Napredne tehnike multimodalnog promptovanja

1. Region-of-Interest Prompting

"Fokusiraj se SAMO na donji desni ugao slike."

2. Visual Grounding

"Objasni šta radi osoba označena žutom strelicom."

3. Diagram-to-Text

“Pretvori ovaj graf u objašnjenje za laik korisnike.”

4. Image-to-Data

Pretvaranje slike tabele u CSV/JSON.

5. Multi-image reasoning

Upoređivanje dvije slike: - progresije - promjene - razlike - napretka - situacionih faktora

8. Sažetak lekcije

Multi-Modal Prompting Pattern omogućava modelima da rade sa slikama, audio, videom i drugim tipovima podataka, čineći AI sistem **pametnijim, sposobnijim i bližim stvarnom životu**.

Ovaj pattern otključava: - bolju analitiku - preciznije razumijevanje - profesionalne workflowe - napredne RAG sisteme - vizualne agente - stvaranje multimodalnih aplikacija

Bez multimodala — AI je ograničen. Sa multimodalom — AI postaje univerzalni analitičar.

Retrieval-Augmented Prompting: When AI Needs Memory (RAP)

1. Uvod u Retrieval-Augmented Prompting (RAP)

Retrieval-Augmented Prompting (RAP) je tehnika u kojoj se AI modelu *prvo* dostavi relevantna informacija iz baze znanja, a *tek onda* model odgovara na upit.

Drugim riječima: **AI + pretraga memorije**

umjesto

AI + nagađanje

RAP je osnova svakog ozbiljnog: - RAG sistema, - AI asistenta sa memorijom, - internog chatbot-a, - enterprise AI rješenja, - workflowa koji koristi tvoje dokumente, - baze znanja u Notionu, Obsidianu, Pinecone-u, ChromaDB-u, Elasticsearchu.

Bez RAP-a, modeli halucinirаju. Sa RAP-om, modeli daju *tačne i verifikovane* odgovore.

2. Zašto je RAP ključan?

Bez retrievala (pretraživanja): - modeli se oslanjaju na opštu obuku, - ne znaju tvoje dokumente, - lako izmišljaju podatke, - ne razumiju specifičan kontekst.

Sa retrieval-om: - odgovori su bazirani na konkretnim izvorima, - smanjuje se rizik od halucinacija, - AI zna *tačno ono što ti želiš*, - možeš izgraditi lične i poslovne AI asistente, - radi kao produžetak tvoje baze znanja.

RAP je temelj za sve napredne AI sisteme — posebno za AI OS i RAG projekte.

3. Standardni RAP Prompt Template

Koristi sljedeći proces: 1. **Primaj upit** korisnika. 2. **Identifikuj ključne pojmove** u upitu (entity extraction). 3. **Kreiraj pretraživački query** zasnovan na tim pojmovima. 4. **Pretraži bazu** i vrati *najrelevantnije fragmente*. 5. **Sažmi i strukturiraj fragmente** u koherentan odgovor. 6. **Referenciraj izvor** (ako je potrebno). 7. **Napravi finalni odgovor** koristeći samo pronađene informacije. 8. **Dodaj confidence score.**

Ovo je osnovni mentalni tok svakog RAP sistema.

4. Interna logika RAP procesa

RAP se sastoji iz tri koraka:

a) Retrieval (pretraga)

AI traži najrelevantnije dijelove baze: - embedding upit → - vektorska pretraga → - vraća se top-k rezultata.

b) Augmentation (dopuna)

Ti rezultati se ubacuju u prompt kao kontekst.

Na primjer: > “Based on the following retrieved information: [tekst], answer the question…”

c) Generation (generisanje)

AI sada odgovara *samo* na osnovu dostavljenog teksta → drastično smanjuje halucinacije.

5. Primjena RAP-a u realnim projektima

a) AI OS memorija

RAP omogućava OS-u da "zna" tvoje prethodne dane, navike, odluke, lekcije.

b) RAG projekti

Svaki odgovor dolazi iz tvoje baze, ne iz modelove improvizacije.

c) n8n integracije

Workflow može: - primiti pitanje → - pretražiti Notion/Obsidian → - poslati fragmente GPT-u.

d) Medicinski upiti

AI koristi samo validne izvore (smjernice, protokole, članke).

e) SEO i scraping sistemi

Analiza koristi sadržaj koji ti scrapuješ, a ne opšte znanje modela.

f) Korporativni asistenti

AI odgovara prema internim PDF-ovima, procedurama, SOP-ovima.

g) ChatGPT sa tvojim dokumentima

Pretraga + kontekst = tačan odgovor.

6. Primjer RAP procesa

Zadatak: "Koje su ključne prednosti OpenAI API-ja?"

Pretpostavimo da je baza znanja sadržavala dokument: > "OpenAI API omogućava integraciju LLM-ova u aplikacije. Prednosti uključuju jednostavno skaliranje, pouzdanu obradu teksta i mogućnost kreiranja agenata."

AI radi sljedeće:

1. Extraction: - prepoznaje ključne pojmove "OpenAI API", "prednosti"

2. Retrieval: → pronalazi relevantan fragment iz baze

3. Augmentation: → ubacuje ga u prompt

4. Response: "Glavne prednosti OpenAI API-ja su skalabilnost, pouzdana obrada teksta i mogućnost razvoja naprednih AI agenata." → 0% halucinacija, 100% bazirano na stvarnom izvoru.

7. Sažetak lekcije

Retrieval-Augmented Prompting (RAP) je tehnika kojom se AI-u dostavlja relevantan sadržaj iz baze znanja prije generisanja odgovora.

Time se: - smanjuje haluciniranje, - povećava tačnost, - AI povezuje sa tvojim podacima, - stvaraju RAG sistemi, - omogućavaju profesionalni AI asistenti.

Ovo je temelj modernog AI dizajna.

Reasoning, Reflection & Self-Correction

Meta-Prompting Pattern

1. Uvod u Meta-Prompting

Meta-prompting je tehnika u kojoj AI: - procjenjuje vlastiti output, - uočava logičke greške i praznine, - daje preporuke za poboljšanje, - optimizira odgovor kroz više iteracija, - djeluje kao *vlastiti editor*, a ne samo generator.

Drugim riječima: **AI ne daje samo odgovor — nego razmišlja O odgovoru.**

To je vrhunac napredne AI upotrebe, posebno u: - pisanju stručnog sadržaja, - medicinskim interpretacijama, - razvoju koda, - dizajniranju AI sistema, - RAG validaciji, - AI OS analitici, - kritičkom razmišljanju, - strategijskom donošenju odluka.

2. Zašto je Meta-Prompting ključan?

Jer AI, kao i ljudi, može: - pogriješiti, - previdjeti detalje, - prebrzo zaključiti, - biti površan, - dati nepotpune odgovore.

Meta-prompting uvodi **samoanalizu**, tako da model: - preispituje logiku, - traži nedostatke, - poboljšava koherentnost, - povećava tačnost, - daje visoko kvalitetne odgovore.

Ovo je tehnika profesionalnih prompt inženjera.

3. Standardni Meta-Prompt Template

Prvo uradi **inicijalni odgovor** na zadatak.

Zatim uradi **meta-evaluaciju** svog odgovora: 1. Identifikuj nedostatke u logici 2. Provjeri jesu li svi dijelovi zadatka ispunjeni 3. Prepoznaj potencijalne greške 4. Predloži kako se odgovor može poboljšati 5. Napiši finalnu, poboljšanu verziju odgovora

Output format: - “Initial Answer:” - “Meta-Evaluation:” - “Improved Answer:”

4. Interna logika Meta-Prompting procesa

Meta-prompting se odvija u tri faze:

a) Generation (inicijalni odgovor)

Model generiše svoj prvi koncept.

b) Evaluation (analiza odgovora)

Model kritički analizira svoj output: - logičke greške - nedostatke - preširoke tvrdnje - površne dijelove - kontradikcije

c) Improvement (poboljšanje)

Model kreira *drugu, bolju verziju* odgovora.

Ovaj ciklus je sličan profesionalnom uređivanju knjige ili akademskog rada.

5. Primjena Meta-Promptinga u realnim projektima

a) Pisanje poglavlja u knjizi

Model može: - napisati lekciju, - sam sebe evaluirati, - poboljšati stil, strukturu i logiku, - isporučiti finalnu verziju visokog kvaliteta.

b) AI OS

Meta-prompting daje: - tačniju dnevnu analitiku, - bolje fokus preporuke, - stabilnije zaključke, - korekciju površnih odgovora.

c) RAG sistemi

Model može: - provjeriti da li je koristio relevantne izvore, - ispraviti greške, - dodati preciznije informacije, - povećati tačnost odgovora.

d) Kodiranje i debugging

AI postiže: - analizu vlastitog koda, - korekciju grešaka, - optimizaciju performansi.

e) Medicina i nauka

Model može: - preispitati medicinsku interpretaciju, - provjeriti dosljednost s preporukama, - ukloniti neprecizne tvrdnje.

f) Business & Strategy

Model može: - revidirati poslovnu logiku, - procijeniti rizik, - eliminirati nepotrebne pretpostavke, - ojačati argumentaciju.

6. Primjer Meta-Promptinga

Zadatak: "Objasni ulogu HDL-a u kardiovaskularnom zdravlju."

Initial Answer: HDL se smatra "dobrim" holesterolom jer pomaže u uklanjanju LDL-a iz arterija i smanjuje rizik od bolesti srca.

Meta-Evaluation: - Objašnjenje je tačno, ali previše pojednostavljeno. - Nedostaje preciznija uloga u reverznom transportu holesterola. - Nedostaje objašnjenje funkcionalnosti HDL čestica. - Trebao bi koristiti stručniji ton.

Improved Answer: HDL holesterol učestvuje u reverznom transportu holesterola, preuzimajući suvišne lipide iz perifernih tkiva i vraćajući ih u jetru. Funkcionalne HDL čestice doprinose antioksidativnim, antiinflamatornim i endotelno-zaštitnim efektima, čime smanjuju aterosklerotski rizik.

→ Druga verzija je mnogo kvalitetnija, preciznija i stručnija.

7. Sažetak lekcije

Meta-prompting je tehnika u kojoj AI model procjenjuje, kritikuje i poboljšava vlastite odgovore.

Ovo omogućava: - veću tačnost - dublje razumijevanje - logički snažnije odgovore - profesionalni stil - manje grešaka - veću pouzdanost

Meta-prompting predstavlja prelazak sa “AI generatora” na “AI mislioca” — i ključ je za napredne AI aplikacije i agente.

Error-Correction Pattern

1. Uvod u Error-Correction Pattern

Error-Correction Pattern je tehnika u kojoj AI: - pregleda svoj odgovor, - traži greške ili nedosljednosti, - objašnjava gdje je pogriješio, - nudi korekciju, - generiše *finalnu, ispravljenu verziju*.

Ovo je ključna tehnika za profesionalne AI sisteme jer smanjuje: - halucinacije, - netačne tvrdnje, - logičke greške, - neprecizne kalkulacije, - kod koji ne radi, - pogrešne pretpostavke.

Error-Correction je kao da model dobije **internog AI QA inženjera**.

2. Zašto je Error-Correction važan?

Bez ove tehnike: - AI često ne uočava sopstvene greške, - odgovori mogu biti netačni, - kod može imati bugove, - medicinske informacije mogu biti pogrešno interpretirane, - RAG odgovori mogu biti slabo utemeljeni.

Sa Error-Correction patternom: - odgovori postaju precizni, - logički korektni, - stručni, - stabilni i pouzdani, - idealni za poslovnu primjenu.

3. Standardni Error-Correction Template

Učini sljedeće: 1. Generiši svoj inicijalni odgovor 2. Pregledaj ga kritički 3. Identifikuj sve greške (logičke, faktografske, stilske, matematičke) 4. Objasni zašto su to greške 5. Ponudi ispravljenu verziju odgovora 6. U finalu prikaži samo ispravljeni odgovor

Output format: - Initial Answer - Error Analysis - Corrected Answer

Ovaj template daje profesionalne rezultate koje možeš direktno koristiti u knjigama, dokumentima, workflowima ili kodu.

4. Interna logika Error-Correction procesa

Error-Correction radi kroz tri ključna misaona mehanizma:

a) Disassembly (rastavljanje odgovora)

Model dijeli svoj odgovor na dijelove kako bi ih analizirao.

b) Error Detection (analiza grešaka)

Traži: - nepreciznosti - nedostatke - kontradikcije - pogrešne pretpostavke - netačne podatke - nekonzistentan stil - previše ili premalo detalja

c) Reconstruction (ponovna izgradnja)

Model pravi novu, poboljšanu verziju odgovora.

5. Primjena u realnim projektima

a) Pisanje stručnog sadržaja

AI može: - ispraviti stil, - ukloniti greške, - poboljšati konzistentnost, - povećati preciznost objašnjenja.

b) AI OS dnevni izvještaji

Greške u zaključivanju se eliminišu.

c) n8n debugging

AI detektuje: - pogrešne ulaze - pogrešne node konfiguracije - loše JSON strukture - nedostajuće polja - nekonzistentne vrijednosti

d) Kodiranje

Model može: - naći bug, - objasniti zašto postoji, - dati ispravku - optimizovati kod.

e) Medicina i nauka

Presudno za tačnost: - interpretacija nalaza - objašnjenje farmakoloških mehanizama - analiza studija - razlikovanje činjenica od netačnih tvrdnji

f) RAG sistemi

AI otkriva: - koji dio odgovora nije podržan izvorom - gdje je kontekst pogrešno interpretiran - da li nedostaju ključne informacije

g) Poslovna i finansijska logika

AI može: - provjeriti kalkulacije - ispraviti rizike - detektovati nelogične ekonomske pretpostavke

6. Primjer Error-Correction procesa

Zadatak: "Koja je uloga triglicerida u metaboličkom zdravlju?"

Initial Answer: Trigliceridi su vrsta holesterola koja povećava rizik od kardiovaskularnih bolesti.

Error Analysis: - Netočno: trigliceridi nisu holesterol, već vrsta masnih molekula. - Nedostaje objašnjenje njihove metaboličke uloge. - Stil previše pojednostavljen.

Corrected Answer: Trigliceridi su skladišni oblik masti i izvor energije. Povišeni nivoi triglicerida povezani su sa inzulinskom rezistencijom, disfunkcijom lipoproteina i povećanim kardiovaskularnim rizikom. Uloga triglicerida je ključna u procjeni metaboličkog zdravlja.

→ Ovo je mnogo preciznije, tačnije i profesionalnije.

7. Sažetak lekcije

Error-Correction Pattern omogućava AI modelu da: - procijeni sopstveni odgovor, - identifikuje greške, - objasni zašto su greške nastale, - ponudi ispravljenu verziju, - kreira profesionalni finalni odgovor.

Ovo je tehnika za *sigurnost, tačnost i profesionalni nivo kvaliteta*.

Bez Error-Correction-a AI ostaje površan. Sa njim — AI postaje samokorektivni sistem visoke tačnosti.

Socratic Prompting Pattern

1. Uvod u Socratic Prompting

Socratic Prompting je tehnika inspirisana Sokratovim načinom poučavanja: **umjesto da daje odgovor → AI postavlja pitanja koja te vode do pravog rješenja.**

To stvara: - dubinsko razumijevanje, - bolje odluke, - kritičko razmišljanje, - lični razvoj, - preciznije rezultate, - veću mentalnu jasnoću.

AI prestaje biti generator teksta — postaje **kognitivni mentor**.

2. Zašto je Socratic Prompting važan?

Bez ove tehnike: - AI može dati odgovor koji ne razumiješ, - model "skraćuje put", - korisnik ostaje pasivan, - problemi se rješavaju površno.

Sa Socratic patternom: - korisnik aktivno razmišlja, - AI te vodi kroz proces, - dobijaš uvid, a ne samo rezultat, - rješenja su 2–5x kvalitetnija, - mentalni modeli se razvijaju.

Savršen za: - AI OS unutrašnji rad - coaching i samorefleksiju - biznis strategiju - complex thinking - tehničke procese i debugging - donošenje odluka - učenje koncepta kroz primjere

3. Standardni Socratic Prompt Template

Kada korisnik postavi pitanje ili problem: 1. Ne daj direktan odgovor 2. Postavi 3–7 preciznih, logičkih pitanja 3. Pomozi korisniku da sam otkrije greške u razmišljanju 4. Razjasni pretpostavke koje možda nisu tačne 5. Vodi ga korak po korak do rješenja 6. Tek na kraju — ponudi finalnu sintezu (ako je potrebno)

Output format: - Clarifying Questions - Reasoning Path - Guided Insight - Optional Final Answer

Ovo transformiše interakciju iz "daj mi odgovor" → "nauči me misliti".

4. Interna logika Socratic procesa

Socratic pattern radi kroz četiri kognitivna mehanizma:

a) Problem Deconstruction (razbijanje problema)

AI razdvaja problem na male, razumljive dijelove.

b) Assumption Testing (testiranje pretpostavki)

AI te vodi da vidiš gdje možda polaziš od pogrešne premise.

c) Guided Reasoning (vođeno razmišljanje)

AI te usmjerava pitanjima, ali ne daje gotove odgovore.

d) Insight Formation (formiranje uvida)

Rješenje dolazi iz tvog vlastitog razumijevanja — ne iz modelove komande.

To gradi *pravo znanje*, a ne samo kopiranje odgovora.

5. Primjena Socratic patterna u realnim projektima

a) AI OS introspekcija

Umjesto "Uradi X", AI te pita: - Zašto misliš da želiš to? - Koji je pravi motiv? - Koji rizik ovdje ne vidiš? - Šta je najvažnije u ovom trenutku?

To te vodi ka dubljem razumijevanju sebe i svojih ciljeva.

b) Biznis strategija i donošenje odluka

Savršeno za: - startup ideje - monetizaciju - MVP planiranje - AI proizvode - strateške dileme

AI te pita: - Koji je ključni value-proposition? - Koji problem zapravo rješavaš? - Ko su korisnici? - Koje su alternative?

c) n8n debugging

Umjesto da "pogađa" grešku: AI ti postavlja pitanja: - Kako izgleda ulazni JSON? - Koja tačno node vraća error? - Da li je format validan? - Šta kaže Execution log?

I uz to te vodi da sam lociraš problem.

d) Učenje tehničkih koncepata

AI te pita: - Šta misliš da je glavna razlika između few-shot i chain-of-thought? - Gdje bi ovaj pattern mogao biti greška? - Šta ovdje model pokušava uraditi?

e) Medicina i analitika

AI te vodi: - Koji su glavni simptomi? - Koje su moguće etiologije? - Koji nalazi to potvrđuju? - Šta se isključuje?

Savršen za edukaciju i razumijevanje kliničkih procesa.

6. Primjer Socratic interakcije

Zadatak: "Da li je dobra ideja da odustanem od SEO scrapera i fokusiram se samo na AI OS?"

AI (Socratic):

Clarifying Questions: 1. Koji je tvoj motiv za AI OS u odnosu na scraper? 2. Koji proizvod donosi brži cash-flow? 3. Koji donosi dugoročnu vrijednost? 4. Koji je tvoj trenutni kapacitet vremena? 5. Šta bi se desilo ako završiš scraper kao portfolio projekat?

Guided Insight: Pitanja te vode da sam vidiš logičnu strukturu: - scraper = brzi output + portfolio - AI OS = dublji projekat + strateški cilj - ne mora biti ili/ili → može biti sekvencijalno

Optional Final Answer: Najbolji pristup može biti završiti scraper kao "shipping win", pa onda AI OS.

→ Socratic pattern te vodi do toga, umjesto da ti naređuje.

7. Lekcija u jednoj rečenici

Socratic Prompting pretvara AI u mentora koji te vodi kroz razmišljanje pitanjima, umjesto da ti daje gotove odgovore — što rezultira dubljim razumijevanjem i kvalitetnijim odlukama.

Reflection Pattern

1. Uvod u Reflection Pattern

Reflection Pattern je napredna tehnika u kojoj AI **naknadno razmatra vlastiti odgovor, odluku ili proces**, s ciljem da: - procijeni kvalitet rezultata, - prepozna slabosti, - identifikuje propuštene tačke, - unaprijedi buduće odgovore, - poboljša strategiju razmišljanja.

Za razliku od Error-Correction (ispravke grešaka), Reflection ide **korak dalje**: > Ne pita se samo "Da li sam pogriješio?" > već *"Kako mogu razmišljati bolje sljedeći put?"*

Ovo je **metakognitivni pattern** — razmišljanje o vlastitom razmišljanju.

2. Zašto je Reflection Pattern ključan?

Jer u realnim projektima: - nema savršenog prvog odgovora, - problemi su višeslojni, - kontekst se mijenja, - odluke imaju posljedice, - učenje dolazi kroz iteraciju.

Reflection omogućava AI-ju da: - uči iz prethodnih interakcija, - poboljšava kvalitet donošenja odluka, - postane stabilniji sistem, - gradi dugoročnu konzistentnost, - djeluje kao **mentor, a ne samo izvršilac**.

Ovo je ključna tehnika za: - AI OS - agente - coaching sisteme - dugoročne projekte - strateško planiranje - profesionalno učenje

3. Razlika: Reflection vs. Error-Correction

Error-Correction	Reflection
Ispravlja greške	Analizira kvalitet razmišljanja
Fokus na tačnost	Fokus na proces
Kratkoročna korekcija	Dugoročno poboljšanje
"Šta je pogrešno?"	"Kako mogu bolje razmišljati?"

U profesionalnim sistemima **oba patterna se koriste zajedno**.

4. Standardni Reflection Prompt Template

Nakon završenog zadatka ili odgovora: 1. Procijeni kvalitet rješenja 2. Identifikuj šta je urađeno dobro 3. Identifikuj šta može biti bolje 4. Prepoznaj koje pretpostavke su korištene 5. Analiziraj da li su te pretpostavke bile opravdane 6. Izvuci lekcije za buduće slične zadatke 7. Predloži kako bi se pristup unaprijedio sljedeći put

Output format: - What Worked Well - What Could Be Improved - Key Assumptions - Lessons Learned - Improved Strategy (for next iteration)

Ovaj template je idealan za kontinuirano poboljšanje kvaliteta AI outputa.

5. Interna logika Reflection procesa

Reflection Pattern se oslanja na četiri kognitivna sloja:

a) Outcome Review (pregled rezultata)

Da li rezultat zaista rješava problem?

b) Process Review (pregled procesa)

Kako se došlo do rješenja?

c) Assumption Awareness (svijest o pretpostavkama)

Koje su pretpostavke bile implicitne?

d) Forward Optimization (optimizacija unaprijed)

Kako unaprijediti pristup u budućnosti?

Ovo čini AI **učećim sistemom**, ne statičnim alatom.

6. Primjena Reflection Patterna u realnim projektima

a) AI OS (lični operativni sistem)

Reflection omogućava OS-u da: - analizira prethodne dane, - procijeni odluke, - prepozna obrasce ponašanja, - prilagodi preporuke, - poboljša fokus i energiju.

Bez reflectiona → OS je samo dnevnik. Sa reflectionom → OS postaje mentor.

b) Prompt Engineering i učenje

AI reflektuje: - da li je prompt bio jasan, - da li je struktura bila optimalna, - šta bi se promijenilo sljedeći put.

c) RAG sistemi

AI može analizirati: - da li je retrieval bio relevantan, - da li su korišteni pravi izvori, - da li je kontekst bio preširok ili uzak.

d) Biznis i strategija

Nakon odluke, AI reflektuje: - da li su rizici dobro procijenjeni, - da li su alternative bile razmotrene, - kako poboljšati donošenje odluka.

e) Coaching i lični razvoj

Reflection omogućava: - dublju introspekciju, - jasnije uvide, - emocionalnu inteligenciju, - dugoročnu promjenu ponašanja.

7. Primjer Reflection Patterna

Zadatak: "Predloži strategiju za monetizaciju AI OS-a." *(Nakon što je strategija već data)*

Reflection Output:

What Worked Well: Strategija je jasno razdvojila kratkoročne i dugoročne prihode.

What Could Be Improved: Nedovoljno pažnje posvećeno kapacitetu vremena i operativnom opterećenju.

Key Assumptions: Pretpostavljeno je da postoji spremna publika za edukaciju.

Lessons Learned: Potrebno je validirati publiku ranije.

Improved Strategy: Dodati pilot-proizvod za testiranje tržišta prije pune implementacije.

→ Ovo nije korekcija greške — ovo je **učenje**.

8. Kako se Reflection koristi sa drugim patternima

Reflection se često kombinuje sa: - Meta-Prompting - Error-Correction - Self-Consistency - Socratic Prompting

Zajedno čine **zatvorenu petlju učenja**: > Generate → Evaluate → Reflect → Improve → Repeat

Ovo je osnova **autonomnih AI sistema**.

9. Sažetak lekcije

Reflection Pattern omogućava AI-ju da analizira ne samo *šta* je odgovorio, već *kako* je razmišljao i *kako može biti bolji u budućnosti*.

Time AI prelazi iz statičnog alata u **učeći, adaptivni sistem**.

Reflection je završni sloj profesionalnog Prompt Engineeringa — i ključna komponenta naprednih AI agenata, AI OS arhitektura i dugoročnih inteligentnih sistema.

ReAct Pattern (Reasoning + Acting)

1. Uvod u ReAct Pattern

ReAct (Reasoning + Acting) je napredni prompt inženjerski obrazac koji kombinuje: 1. **razmišljanje (Reasoning)** 2. **akciju (Acting)** — pozivanje alata, API-a, upita, pretraživanja, parsiranja podataka

Ovaj pattern omogućava AI modelima da: - planiraju korake, - razmišljaju kao agenti, - pozivaju alate, - izvršavaju radnje, - koriste memoriju, - iterativno rješavaju kompleksne zadatke.

ReAct je *osnovna arhitektura* iza modernih AI agenata, kao što su: - OpenAI Assistants - LangChain agenti - AutoGPT - n8n AI agent sistemi - RAG agent pipeline-i

Ako želiš da AI radi "kao agent", moraš ovladati ReAct metodologijom.

2. Zašto je ReAct Pattern ključan?

Bez ReAct-a: - AI samo "priča", ali ne *radi*. - Model može dati odgovor, ali ne može donošenje odluka razložiti. - Kompleksni zadaci se raspadaju. - U integracijama (n8n, Lovable, RAG) AI ne može samostalno praviti korake.

Sa ReAct-om: - AI postaje problem solver, - pametno bira akcije, - koristi alate kad treba, - planira unaprijed, - validira rezultate.

To je temelj inteligentnog ponašanja u AI sistemima.

3. Standardni ReAct Prompt Template

Kada rješavaš zadatak, koristi sljedeću strukturu:

1) Thinking (Reasoning) Opiši svoj tok misli, plan i logiku. (Ne otkrivati korisniku — samo interni proces.)

2) Action Navedi koju radnju preduzimaš: - pretraga - čitanje podataka - računanje - pozivanje API-ja - parsiranje JSON-a - izvršavanje koda - RAG upit

Format:

```
Action: [ime_alata_ili_radnje]
Input: [podatak koji se šalje alatu]
```

3) Observation Zabilježi rezultat akcije.

4) Final Answer Nakon završetka svih akcija, isporuči finalni odgovor korisniku.

Ovaj obrazac simulira način na koji ljudi rješavaju probleme: mislimo → djelujemo → gledamo rezultat → nastavljamo.

4. Interna logika ReAct procesa

ReAct funkcioniše u pet koraka:

a) Interpretacija cilja

Šta korisnik želi? Šta je konačni cilj?

b) Plan razmišljanja

AI pravi mapu koraka (internal chain-of-thought).

c) Izbor akcije

AI bira odgovarajuću radnju (alat, API, kalkulaciju, kod).

d) Posmatranje rezultata

AI vidi šta se desilo nakon akcije.

e) Iteracija ili finalizacija

Nastaviti ili dati finalni odgovor.

Ovaj ciklus kreira inteligentno ponašanje.

5. Primjena u realnim projektima

a) AI OS

Tvoj AI OS koristi "thinking → action → result" kako bi: - generisao dnevni fokus, - provjerio stare zapise, - analizirao navike, - gradio preporuke.

b) RAG sistemi

ReAct je idealan za: - analiziranje pitanja, - pravljenje RAG upita, - provjeru relevantnosti, - sastavljanje finalnog odgovora.

c) AI agenti (OpenAI Assistants)

Agenti koriste ReAct da: - pozovu alat, - pretraže bazu, - upišu podatke, - izvrše kalkulacije, - rješavaju višekoračne zadatke.

d) n8n AI Action nodes

Savršeno se poklapa sa multi-step workflowima gdje model: - parsira - odlučuje - poziva akcije - validira output

e) Programiranje i kod

ReAct omogućava: - generisanje koda, - provjeru koda, - izvršavanje, - korekciju grešaka.

f) SEO Scraper

ReAct agent može: 1. scrape → 2. clean → 3. extract → 4. analyze → 5. export

Sve u jednoj orkestriranoj sekvenci.

6. Primjer ReAct procesa

Zadatak: "Pronađi prosjek LDL vrijednosti u ovoj tabeli i napiši medicinsku interpretaciju."

Thinking: Treba izračunati prosjek → analizirati → dati medicinski zaključak.

Action 1:

```
Action: Calculate
Input: [2.8, 3.1, 3.4, 3.0]
```

Observation 1: Prosjek = 3.075

Action 2: Interpretiraj vrijednost kao kardiolog.

Observation 2: LDL iznad ciljnih vrijednosti.

Final Answer: "Prosječna LDL vrijednost iznosi 3.08 mmol/L, što je iznad preporučenog cilja za većinu pacijenata sa povišenim KV rizikom. Preporučuje se optimizacija terapije i reevaluacija lipida za 6–8 sedmica."

→ Ovo je *AI agent behavior*, a ne običan prompt.

7. Sažetak lekcije

ReAct Pattern je napredni obrazac koji kombinuje razmišljanje i akciju u jedinstveni inteligentni ciklus.

Omogućava AI modelima da: - planiraju, - koriste alate, - razmišljaju u koracima, - validiraju rezultate, - i rješavaju kompleksne zadatke efikasnije nego jednim promptom.

Ovaj pattern je osnova modernih AI agenata, RAG sistema, n8n integracija i svih višenamjenskih AI aplikacija.

Bez ReAct-a AI ostaje generator teksta. Sa ReAct-om → postaje inteligentan sistem.

Deliberate Reasoning Pattern

1. Uvod u Deliberate Reasoning

Deliberate Reasoning Pattern je tehnika u kojoj se model eksplicitno instruiše da: - razmišlja sporije, - analizira problem u više faza, - razmatra alternativne opcije, - razlaže logiku, - minimizira greške, - izbjegava "brze zaključke".

Umjesto automatskog odgovora → model prolazi kroz proces *dubinske evaluacije*.

Ovo je ekvivalent ljudskom: "Sačekaj. Razmisli. Preispitaj. Tek onda odgovori."

To je apsolutno ključno za: - planiranje, - strategiju, - medicinu, - matematiku, - optimizaciju workflowa, - troubleshooting n8n-a, - RAG evaluaciju, - AI OS donošenje odluka.

2. Zašto je Deliberate Reasoning ključan?

Bez ovog patterna: - modeli daju prebrze, površne odgovore, - preskaču ključne korake, - logičke greške su česte, - halucinacije se provuku, - kompleksni problemi ostaju neriješeni.

Sa Deliberate Reasoning-om: - odgovori su precizniji, - pogrešni zaključci se izbjegavaju, - predlažu se alternative, - logika se provjerava, - rezultati su 2–4× kvalitetniji.

Ovo je tehnika koja razdvaja "običan chat" od *profesionalne AI inteligencije*.

3. Standardni Deliberate Reasoning Template

Prije nego što odgovoriš na zadatak: 1. **Identificiraj glavno pitanje.** 2. **Razloži problem na ključne elemente.** 3. **Analiziraj svaku komponentu posebno.** 4. **Razmotri alternativna rješenja (min. 2 pristupa).** 5. **Procijeni prednosti i slabosti svakog pristupa.** 6. **Sastavi logički sekvenciran zaključak.** 7. **Tek onda generiši finalni odgovor.**

Ovaj template tjera model da "radi prave mentalne korake".

4. Interna logika procesa

Deliberate Reasoning se zasniva na tri misaona principa:

a) Decomposition (razlaganje)

Model razbija kompleksan problem na potprobleme.

b) Comparative reasoning (uporedno razmišljanje)

Model razmatra više mogućih rješenja, umjesto jednog.

c) Structured evaluation (strukturalna evaluacija)

Model procjenjuje prednosti i mane različitih pristupa.

Ovo dovodi do *maksimalno racionalnih zaključaka*.

5. Primjena u stvarnim projektima

a) AI OS — donošenje kvalitetnih odluka

Kada AI OS generiše dnevni fokus ili analizira tvoj život → Deliberate Reasoning daje dubinsku logiku i istinsku korist.

b) n8n debugging

Složene greške se rješavaju analizom: - ulaza, - izlaza, - dependency-ja, - edge caseova.

c) Medicinski i farmaceutski slučajevi

Analiza simptoma → diferencijalna dijagnoza → zaključak.

d) Biznis strategija

Poređenje scenarija: - ulozi, - rizici, - benefiti, - ograničenja.

e) RAG optimizacija

Odabir najboljeg retrieval pristupa.

f) Kodiranje

Odabir arhitekture, optimizacija, refactoring.

Deliberate Reasoning je neophodan svugdje gdje je logika temelj.

6. Primjer Deliberate Reasoning u praksi

Zadatak: "Da li je bolji pristup za SEO scraper — n8n pipeline ili dedicated backend?"

Model radi sljedeće:

1. Identifikacija glavnog pitanja

Koje rješenje je skalabilnije, stabilnije i isplativije?

2. Razlaganje

- Složenost
- Skalabilnost
- Trošak
- Održavanje
- Integracije

3. Analiza svake komponente

n8n prednosti: - No-code - Brzo testiranje - Lak upgrade

n8n slabosti: - Ograničenja u loopovima - Trošak cloud izvršenja - Manje kontrole

Backend prednosti: - maksimalna fleksibilnost - moguća integracija u WordPress i druge sisteme - brži scraping

Backend slabosti: - zahtijeva kodiranje - duži razvoj

4. Poređenje i evaluacija

N8n → najbolji za MVP + portfolio Backend → najbolji za skaliranje i prodaju SaaS-a

5. Zaključak

N8n za početak, backend kada rasteš.

→ mnogo logičnije nego “instinktivni odgovor”.

7. Sažetak lekcije

Deliberate Reasoning Pattern je tehnika koja modelu omogućava **duboko, strukturirano i racionalno razmišljanje**, umjesto brzog generisanja površnih odgovora.

To je ključna vještina za: - rješavanje kompleksnih problema, - dizajn AI sistema, - medicinske i naučne analize, - strateško planiranje, - debugging i optimizaciju.

Ovaj pattern daje AI-ju sposobnost koja liči na *kritičko mišljenje*.

Bez njega — modeli često griješe. Sa njim — AI postaje logičan, konzistentan i pouzdan.

About This Book

Operativni Sistem za rad sa AI — *Mentalni obrasci za rad sa vještačkom inteligencijom*

Most people approach language models informally. This book approaches prompting as a discipline — one built on structure, constraints, and repeatable patterns.

Part I establishes the foundation: mental models for controlling conversations, a command framework for shaping model behavior, step-by-step reasoning techniques, and the transition from single prompts to agent workflows. Part II focuses on applied patterns: output structuring, data and document handling, tool usage, validation methods, and self-correcting reasoning systems.

The material is organized by pattern rather than by use case. Each lesson isolates a specific technique, defines its purpose, and shows how it integrates with other components. By the end, you gain a working vocabulary and a structured methodology — not a collection of disconnected tips.

Prompting is not guesswork. It is a structured discipline with logic, constraints, and predictable failure modes — and it can be learned systematically.

This book is organized as a two-part system. Part I builds the cognitive layer: framing requests precisely, verifying understanding, decomposing complex tasks, and maintaining control across extended interactions. It also introduces the transition from isolated prompts to multi-step agent workflows, including role definition, memory limitations, tool usage, guardrails, and production-readiness considerations.

Part II shifts to applied patterns — reusable structures for recurring classes of work: output formatting and structured extraction, reasoning over tables and documents, tool-driven workflows, validation and self-correction loops, multi-modal input handling, and retrieval-augmented prompting. Each pattern is named, isolated, and described in terms of when to use it and how it interacts with others.

Across both parts, the organizing principle remains consistent: a prompt is not a request — it is a specification. When roles, constraints, output formats, verification criteria, and fallback logic are explicitly defined, results become more stable and processes become repeatable.

The techniques are platform-agnostic and apply wherever instruction-following language models are used.

Who this is for. Professionals who rely on AI systems and want consistency rather than intuition. Builders of workflows, analysts, researchers, and product teams who need predictable model behavior.

What this is not. A beginner overview, a template collection, or a book about model training. It is a focused study of prompting as a system.

Two parts. Eleven blocks. Fifty-seven lessons. A structured methodology.

Goran Crljenica — metaxis.io